Linköping University | Department of Computer and Information Science
Master's thesis, 30 ECTS | Computer Science
2025 | LIU-IDA/LITH-EX-A--25/034--SE

# Optimizing and Evaluating Language and Vision Models for Swedish Government Document Classification

*Optimering och utvärdering av språk- och visionsmodeller för klassificering av svenska myndighetsdokument*

**Vilgot Åström**
**Albin Norén**

Supervisor : Krzysztof Bartoszek
Examiner : Marcel Bollmann

External supervisor : Hans Jönsson

LINKÖPING UNIVERSITY

Linköpings universitet
SE–581 83 Linköping
+46 13 28 10 00 , www.liu.se

## Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida `http://www.ep.liu.se/`.

## Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: `http://www.ep.liu.se/`.

© Vilgot Åström
Albin Norén

**Abstract**

This thesis investigates document classification strategies for Swedish government documents using transformer-based models. We explore both unimodal approaches leveraging either textual or visual information and multimodal fusion architectures that combine both modalities. Through comprehensive experimentation, we demonstrate that fine-tuning pre-trained models significantly outperforms using general-purpose embeddings, indicating successful incorporation of domain-specific terminology. Our results show that the KB-SBERT model serves as an excellent foundation for Swedish document classification, while DeBERTa offers strong performance for multilingual applications. We introduce a novel complex early fusion architecture that achieves the highest classification accuracy (90.56%) by effectively integrating the strengths of both text and image modalities, outperforming all unimodal approaches. We also evaluate decoder-based models, finding that while they show promising results, their computational demands present significant deployment challenges. The study further examines the trade-off between model accuracy and computational efficiency, providing practical insights for real-world implementation in government settings. Our findings contribute to the understanding of multimodal document classification in low-resource languages and offer guidance for developing efficient classification systems that balance performance with computational constraints.

# Acknowledgments

We would like to especially thank our supervisor, Krzysztof Bartoszek, and examiner, Marcel Bollmann, for guidance and feedback in the process of writing this thesis. Additionally, we would like to thank the Natural Language Processing group at Linköping University for their input and ideas. We would also like to thank Hans Jönsson at Ida Infront for his contributions and interest, along with the entire Ida Infront Linköping office for hosting us during the study. Finally, we would like to thank Inge Hansson and Fredrik Nääs for reviewing our thesis and providing valuable comments.

# Contents

**Abstract** iii

**Acknowledgments** iv

**Contents** v

**List of Figures** vii

**List of Tables** viii

**1 Introduction** 1
- 1.1 Motivation . . . 1
- 1.2 Aim . . . 1
- 1.3 Research Questions . . . 1
- 1.4 Approach . . . 2
- 1.5 Delimitations . . . 2
- 1.6 Contributions . . . 2

**2 Theory** 4
- 2.1 Practical Background . . . 4
- 2.2 Related Work . . . 5
- 2.3 Natural Language Processing Pipeline . . . 8
- 2.4 Base Models . . . 13
- 2.5 Fine-Tuning . . . 17
- 2.6 Fusion . . . 19
- 2.7 Evaluation Metrics . . . 21

**3 Method** 24
- 3.1 Dataset . . . 24
- 3.2 Model Comparison . . . 27
- 3.3 Fine-Tuning . . . 28
- 3.4 Fusion Model Construction . . . 30

**4 Results** 35
- 4.1 Compiled Results . . . 35
- 4.2 Confusion Matrices . . . 36
- 4.3 Per-Class Accuracy . . . 38
- 4.4 Model Scores . . . 41
- 4.5 Training and Fine-Tuning Times . . . 42

**5 Discussion** 43
- 5.1 Dataset . . . 43
- 5.2 Method Discussion . . . 46

5.3 Results . . . . . 48
5.4 The Work in a Wider Context . . . . . 50

**6 Conclusion** **52**
6.1 Future Work . . . . . 53

**Bibliography** **54**

**A Appendix A - Confusion matrices** **61**

**B Appendix B - Hyperparameters** **76**

# List of Figures


2.1 Attention weights for each head at the final transformer layer. . . . . 11
2.2 ViT architecture . . . . . 16
2.3 How a ViT changes during fine-tuning. . . . . 18
2.4 Comparison between early fusion and late fusion. . . . . 20
2.5 Confusion matrix example for binary classifier. . . . . 22

3.1 Distribution of successfully extracted files . . . . . 26
3.2 Label distribution of extracted files . . . . . 26
3.3 First screen for manual classification . . . . . 28
3.4 During classification interface . . . . . 29
3.5 Results screen . . . . . 29
3.6 Complex early fusion architecture . . . . . 32
3.7 Hierarchical approach for document classification. . . . . 34

4.1 ViT confusion matrix . . . . . 36
4.2 Complex early fusion confusion matrix . . . . . 37
4.3 Human Benchmark confusion matrix . . . . . 38
4.4 Scores for each model as the importance of accuracy against throughput increases. 41
4.5 Rank of each model as the importance of accuracy against throughput increases. . 42

A.1 Confusion Matrix, Precision, Recall and F1 for ViT . . . . . 62
A.2 Confusion Matrix, Precision, Recall and F1 for KB-SBERT . . . . . 63
A.3 Confusion Matrix, Precision, Recall and F1 for DeBERTa . . . . . 64
A.4 Confusion Matrix, Precision, Recall and F1 for Late fusion model . . . . . 65
A.5 Confusion Matrix, Precision, Recall and F1 for Fully trained Early fusion model . . 66
A.6 Confusion Matrix, Precision, Recall and F1 for Simple Early fusion model . . . . . 67
A.7 Confusion Matrix, Precision, Recall and F1 for Complex early fusion model . . . . 68
A.8 Confusion Matrix, Precision, Recall and F1 for Hierarchical model with text concatenation . . . . . 69
A.9 Confusion Matrix, Precision, Recall and F1 for Hierarchical model with feature engineering . . . . . 70
A.10 Confusion Matrix, Precision, Recall and F1 for Decoder model (Qwen2.5-3B) . . . . 71
A.11 Confusion Matrix, Precision, Recall and F1 for CLIP model . . . . . 72
A.12 Confusion Matrix, Precision, Recall and F1 for Frozen KB-SBERT model with 2-layer MLP classification head . . . . . 73
A.13 Confusion Matrix, Precision, Recall and F1 for Naive Bayes classifier . . . . . 74
A.14 Confusion Matrix, Precision, Recall and F1 for Human benchmark . . . . . 75

# List of Tables

2.1 Example Confusion Matrix for Three-Class Classification . . . . . 23
2.2 Matrix with different aggregation strategies . . . . . 23

4.1 Performance comparison of different classification models on 15-class dataset. . . . 36
4.2 Per-class Performance Metrics KB-SBERT . . . . . 39
4.3 Per-class Performance Metrics ViT . . . . . 39
4.4 Per-class Performance Metrics Complex early fusion . . . . . 40
4.5 Per-class Performance Metrics Human Benchmark . . . . . 40
4.6 Training details . . . . . 42

B.1 Training Hyperparameters KB-SBERT . . . . . 76
B.2 Training Hyperparameters ViT . . . . . 76
B.3 Training Hyperparameters Complex Early fusion . . . . . 77
B.4 Training Hyperparameters Qwen2.5-3B . . . . . 77

# 1 Introduction

This thesis aims to investigate the use of large language models in Swedish government document classification. It is carried out at the company Ida Infront in Linköping, Sweden. This chapter introduces the problem in a wider context and then poses the research questions.

## 1.1 Motivation

In today's digitalized society, Swedish government agencies handle an extensive volume of documents daily. These documents must be correctly classified to ensure proper handling, archiving, and accessibility. This creates challenges for public administration, including ensuring consistent classification across diverse document types and addressing the scalability required to handle large volumes of documents efficiently [59].

Although Large Language Models (LLMs) represent a promising tool for automating document classification, existing implementations are predominantly optimized for the English language. Moreover, domain-specific knowledge such as administrative procedures and legal terminology remains a significant gap, further complicating their adoption in this context.

Addressing these gaps presents an opportunity to not only improve the efficiency and accuracy of document handling in Swedish government agencies but also to contribute to the broader development of language models for low-resource languages and specialized domains.

## 1.2 Aim

This project aims to optimize and evaluate LLMs specifically for classifying Swedish government documents and assessing model performance for local deployment within government agencies. The goal is to provide Swedish agencies with scalable, accurate classification solutions that can be deployed locally, minimizing computational demands while ensuring reliable performance.

## 1.3 Research Questions

The thesis seeks to answer the following research questions.

1. How can domain-specific terminology and contextual knowledge be incorporated into pre-trained language models to improve classification accuracy for Swedish government documents?
2. What strategies can balance model accuracy and computational efficiency for deploying language models locally in Swedish government agencies?
3. How does a multimodal approach combining visual and textual elements enhance document representation compared to unimodal approaches relying solely on either visual or textual information?

## 1.4 Approach

To address the research questions, this project begins with conducting a thorough literature study to identify classification techniques successfully employed in similar domains. This investigation focuses particularly on methods for domain-specific adaptation of predictors. The subsequent findings, along with theoretical background, can be found in Chapter 2.

When a sufficient understanding of the task has been acquired, the creation of a robust dataset derived from files submitted to a Swedish government agency follows. The labeled dataset is constructed by using file names as class indicators, establishing a minimum threshold for class inclusion to ensure sufficient representation. The steps taken towards creating the dataset is described in Chapter 3.

Based on this research, we select appropriate classification models for our task. We first evaluate baseline (non-fine-tuned) models to establish performance benchmarks and identify specific limitations. Using the results from these tests, informed decisions can be made in fine-tuning and creating novel model architectures that try to improve on the limitations of the base models. A comprehensive view of the findings is presented in Chapter 4.

The results are then analyzed to provide a detailed assessment of the predictors' performance and the dataset's inherent challenges, which is found in Chapter 5. Finally, Chapter 6 summarizes the review to directly address the research questions posed in this study.

## 1.5 Delimitations

To maintain a manageable project scope, certain limitations have been established. Firstly, the dataset used is a reduced version of the full dataset provided by the client of Ida Infront. Since data pre-processing is an entire field in and of itself, it would be very time-consuming to structure the entire dataset into a workable format with fitting labels. By initially limiting the dataset into a manageable size, more focus can be directed towards creating and comparing models.

Secondly, this project will leverage pre-trained models rather than attempting to develop an entirely new model architecture. While designing a novel architecture would be interesting, it would demand a substantially larger dataset and require more powerful computing resources than are currently available.

Finally, this project will be narrowed to the domain of documents written in Swedish and English within Swedish government contexts. If any of the solutions presented are applied to a different domain or in another language, the results cannot be guaranteed.

## 1.6 Contributions

This project aims to deliver practical insights in optimizing LLMs for Swedish government document classification. The primary contributions of this project include:

- New detailed insights into how to use language models to accurately classify documents in the specific context of Swedish public administration.

- A systematic analysis of the trade-off between computational efficiency and predictor accuracy across various classification techniques.
- An investigation into the resources required to effectively implement an efficient automatic classification system for a company's document management infrastructure.
- An analysis of how multiple modalities can be used together in the context of document classification for specific domains.

# 2 Theory

Before exploring document classification methods, it is important to comprehend the types of documents handled by Swedish municipalities, as well as the various model architectures that are relevant for this project. This chapter will provide an overview of both.

## 2.1 Practical Background

This section provides an overview of the company where this thesis was conducted, along with the documents used by its customers.

### 2.1.1 Ida Infront

Ida Infront is a Swedish company providing IT solutions primarily to government and municipality agencies in Sweden. Their service, IIPAX, is a tool used for secure information exchange, case management, and electronic archiving [33]. It is in the company's and their clients' best interest to implement an easily deployable and scalable solution for classifying and organizing large document datasets.

### 2.1.2 Documents in Swedish Government Agencies

Swedish government agencies manage a diverse range of documents in their daily operations. These documents serve various administrative, procedural, and legal functions, and can look very different. A list describing the labels that this project uses is presented below. A detailed description of how these labels were acquired can be found in Section 3.1.

- **Agreement** Formal agreements between parties, such as contracts for services, partnerships, or legal settlements. These documents clearly define terms and conditions.
- **Application** Submitted by individuals or organizations to notify authorities about specific actions, incidents, or requests. It may include reports of issues or applications for services.
- **Cover Letter** A cover letter or forwarding document used to introduce and explain attached reports, decisions, or referrals.

- **Correspondence** General communication between individuals, organizations, or departments. Includes letters and emails addressing various subjects.
- **Decision** Official determinations or resolutions made by authorities, documenting final outcomes of deliberations, approvals, or rejections.
- **Inquiry** A formal or informal inquiry seeking clarification, information, or opinions from authorities or organizations.
- **Invoice** A billing document detailing amounts due for goods or services provided. It includes payment terms, deadlines, and issuer details.
- **Order** A request to procure goods or services. This document formalizes orders and specifies quantities, pricing, and delivery terms.
- **Photo** Visual documentation used as evidence, record-keeping, or illustrative material in various municipal processes.
- **Police Report** A formal report submitted to the police regarding incidents requiring legal attention or investigation.
- **Receipt** Documents confirming the receipt of funds, goods, or services. Typically brief and used for financial and administrative tracking.
- **Statement** A statement or opinion provided in response to a request, often regarding legal, administrative, or procedural matters.
- **Terms and Conditions** Documents outlining terms and conditions for agreements, services, or regulatory compliance.
- **Text Message** Short text messages used for communication between authorities and individuals.
- **Ticket** Grants access to a service or transportation. Tickets typically include validity details, conditions, and identification information.

All of the documents that this project will be working with were sent to a single Swedish government agency in the span of 6 months. Some of the classes share the same document template, making them visually similar, while others may be textually similar but visually different. Since the agency does not want to be mentioned, in addition to the sensitive nature of the documents, no examples of how the documents look will be included in this report, since all of their content would have to be masked out.

## 2.2 Related Work

This section examines relevant literature addressing similar domains as in this project, alongside pre-trained classification models that have not undergone fine-tuning. These models will serve as motivation for and comparisons against our implementations.

### 2.2.1 BERT Models for Swedish Text Classification

In 2020, a work comparing various BERT models for binary text classification in Swedish was presented [29]. Although their work focused on raw text classification, their findings are relevant to our context as well. The project examined multiple models trained on Swedish corpora, as well as multilingual variants, and found that BERT models specifically pre-trained on Swedish consistently outperformed general-purpose multilingual models. Furthermore, their best results were achieved with the BERT model released by the Swedish National Library, motivating our choice to use this model in our thesis.

### 2.2.2 DeBERTa Models for Text Classification

Multiple previous works have focused on text classification in specialized domains. For instance, one study from 2024 applied fine-tuned DeBERTa models for phishing detection [40], while another explored entailment classification in the medical domain of clinical trials [72]. Despite the significant differences in these tasks, both achieved SOTA results using DeBERTa models. This strong performance over diverse contexts motivates us to explore their potential for our task in this thesis.

### 2.2.3 Fusing Modalities for Document Classification

Due to the varying structure of documents, where some have images, handwritten text or just plain computer text, Bakkali et al. investigates how multi-modal fusion could enhance the document classification process by handling both textual and visual information from a document [5]. In this project three text models where tested: Glove, FastText and BERT, and two visual models: NasNet and Inception-ResNet-v2. Two different fusion strategies where also tested; an equal concatenation method where the two unimodal features are concatenated, and average ensembling fusion where the two representations are simply added together. By testing these fusion methods and comparing them to uni-modal models the authors proved that multi-modal fusion can lead to an increase in accuracy.

This paper inspired us to test if multi-modal fusion could improve classifier accuracy as well. Since this project was for English document classification plus the fact that it was conducted in 2020, we will not be using the same base models as they used. Apart from that we will also look into some more advanced fusion strategies.

### 2.2.4 Multi-modal Fusion in Sentiment Analysis

An area where multi-modal fusion has been widely adopted and tested is sentiment analysis, where the task is to identify what tone a sentence is written or spoken in [20, 68]. In some cases, visual or audible cues can be detrimental for interpreting the sentiment of a sentence since elements such as irony can be hard to identify in their textual form, especially if taken out of context.

Many variations of networks for multi-modal fusion have been explored, one very similar to this project uses a BERT and ViT model to predict the sentiment of a student chat message. Here, the text from the chat plus emojis are used as input, where emojis serve as the visual modality [68]. The dataset used in this paper was very small, with only 262 labeled instances, where each instance belongs to one of the following labels: positive, negative, or neutral.

During prediction, the two modalities are sent to their respective models and the CLS tokens are then extracted as representations of the input. Due to the small size of the dataset, no additional fine-tuning is performed. The paper tested two fusion methods, one using the dot product of the two CLS tokens and the other through tensor product. Both methods resulted in an increase in accuracy compared to using the base models by themselves. However, the increase was not substantial ( $< 1\%$).

The scope of our project differs significantly. While that paper relies on a limited dataset with short texts that prevents model fine-tuning, our thesis leverages a comprehensive dataset with extensive texts well-suited for fine-tuning purposes. Furthermore, the paper only explores two basic fusion methods, whereas our thesis examines a wider range of fusion techniques. However, this project utilizes a ViT for their visual representation which is easy to combine with a BERT-based model since both are transformer-based, this has inspired us to also use a ViT.

### 2.2.5 Decoder-Based Classifiers

Decoder-based models have become the most prominent AI models in mainstream discourse, with examples like ChatGPT representing this architectural approach [1]. Due to their success in natural language tasks it is of interest to investigate how they perform on more specific tasks like classification.

A study aiming to classify public documents using language models has been explored in Spanish-language datasets [45]. This study compared four models, including three encoder-based models (different RoBERTA versions) and one decoder-based model (GPT-2), concluding that encoder-based architectures are more effective for document classification tasks. The authors attribute this to encoder-based models being optimized for text understanding, while decoder-based models are designed for text generation.

Another study investigates how decoder and encoder based models compare for intent classification and sentiment analysis [8]. For encoders they used BERT, RoBERTa and DistillBERT while for decoders they used LLaMa 2, GPT-3.5 and GPT-4. Here they could deduce that encoder models consistently outperformed decoder models, especially for datasets containing numerous classes. Encoder models have a significantly slower processing times, for example a fine-tuned Llama 2 model proved to have 108x longer inference time then a RoBERTa model. The authors conclude that decoder-based models can serve as an alternative to encoder-based models when fine-tuning is nonviable but otherwise encoder-based models are still preferred.

However, many studies have still attempted to adapt decoder-only models for classification tasks. One study found that after fine-tuning a LlaMa 2 model for classifying abusive text it was able to achieve competitive results to BERT-based architectures [44]. This indicates that decoder-based classifiers can be a viable choice in certain domains.

Although the results from these studies do not favor decoder-based classifiers, this project will also include one decoder-based models to explore whether our classification task is viable for this model type. While the previous projects only looked at GPT and LLaMa versions, we will examine Qwen 2.5, which is more current, open source, and multilingual [69]. This will hopefully provide insights into how the current landscape for decoder-based classification looks and if these model types are a feasible alternative in our domain.

### 2.2.6 Text Classification Using Naive Bayes

A common theme in previous works in text classification is the inclusion of a Naive Bayes classifier as a baseline. For example, one study published in 2024 showed its competitiveness across diverse datasets [54], while another study from the same year presents its value as a lower-bound reference in an evaluation of text classification methods [61]. Incorporating a Naive Bayes classifier makes potential gains from more modern models easily quantifiable against a well-established standard. This motivates the decision to use this method in this thesis.

### 2.2.7 DocBERT

In 2019, a BERT model for document classification, DocBERT [2], was introduced as a fine-tuned version of BERT. It was shown to outperform the state-of-the-art methods for document classification at the time and is still used as a baseline in many cases.

DocBERT's success, although mainly demonstrated in English datasets, shows the value of fine-tuning language models for specific document classification tasks. This aligns with the goal of this project of optimizing models for Swedish government documents, where language-specific characteristics and domain knowledge require similar customization to ensure accurate and adaptable solutions.

The main difference between DocBERT's goal and this thesis lies in the dataset scope. This thesis addresses both photographed document images and digitally created documents,

whereas DocBERT was only evaluated on the latter. Additionally, this thesis primarily focuses on Swedish documents, while DocBERT's datasets were exclusively in English.

### 2.2.8 Zero-Shot Classification Using CLIP

An alternative to constructing a dataset with correctly labeled files and then fine-tuning a classifier on said dataset is to use a zero-shot classifier, which does not require any sort of fine-tuning or labeled data. This model have not been altered or further trained, meaning minimum overhead is required to get them to start making predictions.

Contrastive Language-Image Pretraining (CLIP) combines image embeddings with text embeddings to perform zero-shot classification of images [47]. The architecture utilizes a transformer for embedding the text [62]. In contrast to BERT, CLIP does not use the CLS token for representing a sentence, but instead uses the final value of the EOS (end of sequence) token.

In the original paper, two different approaches were tested for embedding the images: one using a ResNet and another using a ViT [24][16]. For training the model, a dataset consisting of 400 million image-text pairs was created called WIT (WebImageText). Since the model handles two inputs (text embedding, image embedding) in batches of size N, the input embedding can be represented as a 2-dimensional similarity matrix of size NxN. The training objective now becomes to maximize the similarity between corresponding image-text pairs (the diagonal elements of the NxN similarity matrix) while minimizing all other pairs. This teaches the model to connect visual concepts with their linguistic descriptions.

CLIP displayed better accuracy than previous state-of-the-art classification models for datasets with a limited number of labeled images, which is reasonable since a zero-shot model does not require any dataset-specific training. However, when it comes to more complex tasks like satellite image classification, CLIP under-performs compared to a fully supervised classifier.

Although the CLIP architecture performs image classification through a combination of visual and textual context much like this project, there are some differences between the CLIP project and this paper. The objective of the CLIP project was to create a generalized zero-shot classifier, which leads to a very different pre-training task compared to this project, where the dataset and domain are already determined. The scope of the classifier also drastically changes the size of the dataset and the required compute to get a feasible result.

## 2.3 Natural Language Processing Pipeline

In this section, a typical NLP pipeline using a Transformer-based approach for a text classification task is described. This includes the initial tokenization, the embedding, the transformer encoding, and a final classification head. Each step involves both conceptual and technical details, supported by relevant scientific sources. To illustrate, a sentence extracted from a Swedish governmental document is employed as a running example. This section will consider the sentence

*"Ministern presenterar regeringens planering."* (2.1)

and examine how it is processed through the pipeline.

### 2.3.1 Tokenization

Tokenization refers to the process where text is split into smaller units (tokens). This could be done by splitting the text into words, sub-words, or characters. Word tokenization often leads to massive vocabularies and is plagued by the out of vocabulary (OOV) problem [55]. On the other hand, character-level tokenization often fails to capture any meaningful context

and can lead to very long inputs [38]. For these reasons, most modern pipelines use subword tokenization [15, 48, 69]. One of the most common tokenization algorithms is Byte Pair Encoding (BPE). First introduced as a data compression algorithm [19], it was later modified to tokenize text [55]. Starting with a vocabulary of every single character in the corpora, the algorithm starts by finding the most common pair of symbols in the vocabulary and merges them into a new symbol in the vocabulary. This process repeats until a fixed vocabulary size is reached. For our example sentence 2.1, the most common bigram is 'er', which is added to the vocabulary. After eight iterations, the sentence is divided as

`Minister n presenter ar regering ens planering .`

In practice, tokenizers are trained on large corpora to identify frequent patterns to construct a vocabulary. Here is how the sentence is tokenized by Alibaba Cloud's Qwen [69] model:

`Min ist ern presenter ar regering ens planering .`

Another type of tokenization is WordPiece tokenization [57, 67]. It works in a similar way to BPE, but chooses new merges based on the maximum likelihood instead of solely focusing on frequency [9]. This is the tokenization used in BERT [15], which tokenizes the example sentence as follows:

`Minister n presenter ar regering ens plane ring .`

As seen, the tokenizer splits the word 'planering' into 'plane' and 'ring'. This seems good in an English context, but might not be optimal for Swedish. A BERT tokenizer trained on multiple languages splits the sentence in the following way:

`Minister n presenter ar regeringens planering .`

This better captures the morphemes of the Swedish language. To take this further, the Swedish national library, KBLab, trained a BERT model on Swedish corpora [52]. It tokenizes the sentence as follows:

`Mini stern presenterar regeringens planering .`

At the tokenization step, additional special tokens are added. In the example above, a [CLS] token is added to the start of the sequence and a [SEP] token to the end. The [CLS] token is a classification token, which at the end of the encoding step will represent the whole sequence. The [SEP] token is added to indicate the end of the sentence. After tokenization, each token receives a unique id, which is what gets passed into the model. For the example above, it is tokenized in the following manner:

`[2 , 32744, 10760, 8996, 5743, 7116, 7, 3 ]`

where 2 represents the [CLS] token and 3 represents the [SEP] token.

### 2.3.2 Embedding

An embedding is a mapping from each token in the vocabulary to a dense vector of real numbers, capturing semantic and syntactic information [42]. By embedding language into this continuous vector space, models can perform arithmetic-like operations on word meanings (e.g. `king - man + woman = queen`). An embedding layer can be viewed mathematically as a lookup table $E \in R^{|V| \times d}$, where $|V|$ is the vocabulary size and $d$ is the embedding dimension. For each token $t$, the embedding is given by

$$e_t = E[t] \in \mathbb{R}^d.$$

Additionally, positional embeddings are added to inject the model with knowledge of the position of each token in the sequence [15]. The output of the embedding stage is a matrix of shape $(\textit{sequence length} \times \textit{embedding dimension})$. For the example sentence 2.1 tokenized previously, the embedded vector is constructed as

$$\begin{bmatrix} | & | & | & | & | & | & | & | \\ \mathbf{e}_2 & \mathbf{e}_{32744} & \mathbf{e}_{10760} & \mathbf{e}_{8996} & \mathbf{e}_{5743} & \mathbf{e}_{7116} & \mathbf{e}_7 & \mathbf{e}_3 \\ | & | & | & | & | & | & | & | \end{bmatrix} \in \mathbb{R}^{8 \times 768}.$$

where 8 is the sequence length $N$, and 768 is the embedding dimension $d$ used in BERT architectures [15].

### 2.3.3 Transformer Encoding

Transformer encoding utilizes self-attention to weigh each part of a sequence according to their relevance. Introduced in 2017 [62], it revolutionized language modeling by enabling models to capture dependencies across the entire sequence, regardless of length. At the core of the transformer lies a mechanism called attention [4]. The transformer encoding block takes an embedded sequence $X$ as input, which is projected into three matrices; query $Q$, key $K$, and value $V$. This is done using learned weight matrices

$$Q = XW^Q, K = XW^K, V = XW^V,$$

with $W^Q$, $W^K$, $W^V \in \mathbb{R}^{d\times d}$. For multi-head attention, these projections are computed separately for each head. For example, suppose $h = 12$ heads are used. Then for each head $i$, projection matrices $W_i^Q$, $W_i^K$, and $W_i^V \in \mathbb{R}^{d\times d_h}$, where $d_h = \frac{d}{h} = \frac{768}{12} = 64$. For head $i$, the projections are

$$Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V,$$

with $Q_i$, $K_i$, $V_i \in \mathbb{R}^{N\times d_h}$. For each head, attention scores are computed as

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_h}}\right)V. \tag{2.2}$$

Let's focus on head $i$. The product $Q_iK_i^T$ computes a score matrix $S \in \mathbb{R}^{N\times N}$ where each entry $S_{a,b}$ measures the similarity between token $a$ (as a query) and token $b$ (as a key). To be explicit, if $q_a \in \mathbb{R}^{d_h}$ is the $a$-th row of $Q_i$ and $k_b \in \mathbb{R}^{d_h}$ is the $b$-th row corresponding to $K_i$, then

$$S(a, b) = \frac{(q_a \cdot k_b)}{\sqrt{d_h}}.$$

After computing $S$, the softmax function is applied row-wise to obtain attention weights $A \in \mathbb{R}^{N\times N}$ as

$$A(a, b) = \frac{exp\big(S(a, b)\big)}{\sum_{b'=1}^{N} exp\big(S(a, b')\big)}. \tag{2.3}$$

Finally, the head's output is computed as

$$head_i = AV_i,$$

where $head_i \in \mathbb{R}^{N\times d_h}$. After processing all $h$ heads independently, their outputs are concatenated along the feature dimension $Concat(head_1, head_2, \ldots, head_h) \in \mathbb{R}^{N\times(h\cdot d_h)}$, and the concatenated matrix is then linearly transformed using another learned weight matrix $W^O \in \mathbb{R}^{d\times d}$ as

$$Z = Concat(head_1, \ldots, head_h)W^O.$$

Thus $Z \in \mathbb{R}^{N\times d}$ is the output of the multi-head self-attention sub-layer, which is then passed through a residual connection adding the original $X$ and a layer normalization step

$$Z' = LayerNorm(X + Z).$$

Following the attention sub-layer, each transformer layer also includes a feed-forward layer, operating on each token independently. It typically contains two layers with a ReLU activation function in between. The output $Y$ is computed as

$$Y = FFN(Z') = max(0, Z'W_1 + b1)W_2 + b_2,$$

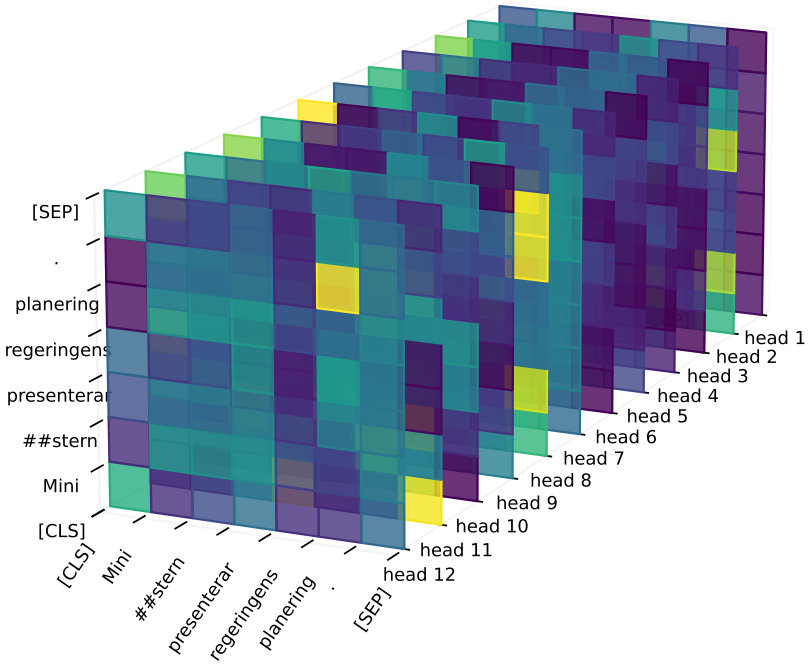



Figure 2.1: Attention weights for each head at the final transformer layer.

where $W_1, W_2$ are weight matrices and $b_1, b_2$ are the bias vectors. While the ReLU activation function is used in the original architecture, many popular models implement the GeLU function instead [28, 15, 48]. The first layer projects from $d$ to a higher-dimensional space $4d$. The second layer is then used to project back into the original space $d$. Again, a residual connection is added along with a layer normalization step, producing the final output $Y'$ of the transformer layer as

$$Y' = LayerNorm(Z' + Y).$$

This concludes one transformer layer. In the original Transformer paper [62], the authors proposed 6 layers in the encoder. The BERT models use 12 layers for their base variants and 24 layers for their large models [15].

Returning to the example sentence 2.1, we can now understand how it gets encoded by the transformer. Figure 2.1 shows the attention weights for each head after it has been passed through the twelfth and final transformer layer of a BERT model. Each matrix is the one obtained in Equation 2.3. The Y-axis represents the queries, and the Z-axis represents the keys. After this step, the process continues until the final output $Y'$ is obtained as

$$Y' = \begin{bmatrix} | & | & | & | & \cdots & | \\ \mathbf{y}_{2\,[\text{CLS}]} & \mathbf{y}_{32744\,[\text{Mini}]} & \mathbf{y}_{10760\,[\text{stern}]} & \mathbf{y}_{8996\,[\text{presenterar}]} & \cdots & \mathbf{y}_{3\,[\text{SEP}]} \\ | & | & | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{8\times 768},$$

a context-aware encoding of the sentence.

### 2.3.4 Final Classification Head

While the transformer encoding layers produce context-aware token embeddings, the classification head transforms these embeddings into a single decision, e.g., assigning a label or category. Typically, this is done using the `[CLS]` token at the beginning of the sequence, designated to pool information about the entire input, although some architectures may integrate different pooling strategies (e.g., mean pooling of all token embeddings) instead. The

classification head is usually implemented as a Multi-layer Perceptron (MLP), outputting a probability distribution over the possible classes through a softmax layer. This MLP typically consists of one or more fully connected layers with nonlinear activation functions (e.g., ReLU or GELU), followed by a final linear layer that maps to the number of output classes. The weights of this classification head are fine-tuned during training to minimize a chosen loss function, such as cross entropy [15].

Continuing with the example sentence 2.1, let's assume the task is to classify the sentence into categories *Agreement*, *Announcement*, or *Budget*. The classification head might receive the [CLS] token from the final transformer layer, such that the input to the classification head is a vector $y \in \mathbb{R}^{768}$. This token is then processed through a linear layer to produce a prediction $p$ as

$$p = argmax(softmax(W_c y + b_c)),$$

where $W_c \in \mathbb{R}^{3\times 768}$ is a weight matrix and $b_c \in \mathbb{R}^3$ is a bias vector. The $softmax$ function transforms the logits from the layer to a distribution over the classes, and the $argmax$ function outputs the most probable class.

### 2.3.5 Decoding

The Transformer-decoder is designed for autoregressive sequence generation, meaning it produces the output sequence token by token. A typical Transformer decoder block consists of three main sub-layers: masked multi-head self-attention, encoder-decoder cross attention, and a position-wise feed-forward network. Residual connections and layer normalization are applied around each sub-layer.

Since the decoder must not attend to future tokens, a causal mask $M \in \mathbb{R}^{T\times T}$ is introduced, where

$$M(i,j) = \begin{cases} 0, & \text{if } j \leqslant i, \\ -\infty, & \text{if } j > i. \end{cases}$$

The mask is then applied before the softmax function in the self-attention as

$$MaskedAttention(Q,K,V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Similarly to the encoder, the $Q$, $K$, and $V$ matrices are usually split into multiple heads for which the attention weights are computed separately and then concatenated along the feature dimension and transformed with a weight matrix to get the output $Z^{mask} \in \mathbb{R}^{T\times d}$, where $T$ is the sequence length so far and $d$ is the embedding dimension. This output is added to the input $X$ to create a residual connection and the sum is normalized to produce $\widetilde{Z}$ as

$$\widetilde{Z} = LayerNorm\Big(Z^{mask} + X\Big).$$

Traditionally, the transformer decoders then consist of a cross-attention block, which attend to the sequence encoded by the transformer encoder. This works by introducing another attention block, where the queries are generated from the previous sub-layer, but the keys and values come from the encoder outputs $Y'$ such that

$$\begin{aligned} Q^{\text{cross}} &= \widetilde{Z}W^{Q,\text{cross}} \\ K^{\text{cross}} &= Y'W^{K,\text{cross}} \\ V^{\text{cross}} &= Y'W^{V,\text{cross}} \end{aligned}$$

and

$$Z^{\text{cross}} = softmax\Big(\frac{Q^{\text{cross}}(K^{\text{cross}})^\top}{\sqrt{d_k}}\Big)V^{\text{cross}}.$$

Again, this is done separately for all heads and concatenated and linearly transformed with a weight matrix. It is then added to a residual connection and normalized to produce the output $\widetilde{Z}^{\text{cross}} \in \mathbb{R}^{T \times d}$. Finally, each row of $\widetilde{Z}^{\text{cross}}$ is processed independently by a two-layer feed-forward network and again added to a residual connection and normalized. This produces the final output $\widetilde{Y}$ of the decoder block as

$$\begin{aligned}\widetilde{Y} &= LayerNorm\Big(\widetilde{Z}^{\text{cross}} + \text{FFN}(\widetilde{Z}^{\text{cross}})\Big) \\ &= LayerNorm\left(\widetilde{Z}^{\text{cross}} + \Big(max(0, \widetilde{Z}^{\text{cross}} W_1 + b_1)W_2 + b_2\Big)\right),\end{aligned}$$

where $W_1, W_2$ are weight matrices and $b_1, b_2$ are the bias vectors.

Many modern text-generation models, such as the GPT-series, utilize a decoder-only architecture. In such cases, the cross-attention block is simply excluded from the decoder layer [48]. This concludes one Transformer-decoder block. In the original encoder-decoder transformer architecture, the authors proposed using six decoder layers [62]. Most modern architectures utilize more. For example, OpenAI's largest GPT-3 model utilizes 96 decoder layers [10].

## 2.4 Base Models

In this section, we present the foundational models upon which this project builds, as well as the architectural predecessors that have contributed to the core framework.

### 2.4.1 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes Theorem [6], which states how to combine prior information with new evidence to determine a conditional probability. Given a document $D$ and a class $C$, the theorem states

$$P(C \mid D) = \frac{P(D \mid C)\, P(C)}{P(D)},$$

where $P(C \mid D)$ is the posterior probability of class $C$ given document $D$, $P(C)$ is the prior probability of class $C$, $P(D \mid C)$ is the likelihood of observing document $D$ given class $C$, and $P(D)$ is the evidence, which is constant across classes. Since $P(D)$ is the same for all classes and does not affect the outcome of the classification, the formula can be simplified to

$$P(C \mid D) \propto P(D \mid C) P(C).$$

The prior probability $P(C)$ is the initial probability of encountering class $C$ before considering any data. It is computed as the fraction

$$P(C) = \frac{N_C}{N},$$

where $N_C$ is the number of documents belonging to class $C$ and $N$ is the total number of documents. Under the (naive) assumption that words in the documents are conditionally independent given the class $C$, the likelihood is computed as

$$P(D \mid C) = P(w_1, w_2, \ldots, w_n \mid C) = \prod_{i=1}^{n} P(w_i \mid C).$$

For each word $w_i$, its probability $P(w_i \mid C)$ can be estimated from the training data by counting how often it occurs in class $C$ and dividing it by the total number of words in the class.

Implementing additive smoothing to prevent division by zero in cases where a word doesn't appear in training data for a given class, the conditional probability is computed as

$$P(w_i \mid C) = \frac{N_{w_i,C} + 1}{W_C + V}.$$

where $N_{w_i,C}$ is the number of times word $w_i$ appears in class $C$, $W_C$ is the total word count in the class $C$, and $V$ is the total number of unique words. With these smoothed likelihoods and computed priors, Naive Bayes efficiently estimates the posterior probability for each class, making it a robust baseline for text classification tasks despite its simplifying assumptions.

### 2.4.2 BERT

BERT (Bidirectional Encoder Representations from Transformers) is a Transformer encoder model designed for natural language understanding [15]. The developers of BERT discovered that the one-way attention mechanism found in leading pre-trained language models of the time, significantly constrained model performance on tasks requiring comprehensive contextual understanding, such as sentence comprehension and question answering systems. For example, the pre-training objective for the first GPT model was to find the most probable token in a sequence of tokens, which in mathematical terms can be explained as trying to maximize the following likelihood

$$L_1(U) = \sum_i \log P(u_i | u_{i-k}, ..., u_{i-1}; \Theta)$$

where $u_i$ is the new token we are trying to predict given previous tokens $u_{i-k}, ..., u_{i-1}$, for a given context length $k$ and the network parameters $\Theta$ [48]. Since the original GPT architecture utilized the decoder part of the transformer which, as described above, masks out future tokens, the pre-training task fits according to the utilized architecture. However, since the training objective is only looking at the previously seen tokens, the context from the tokens coming after the current token is left out.

#### Pre-Training

BERT is pre-trained on two distinct tasks simultaneously: masked language modeling (MLM) and next sentence prediction (NSP). MLM masks out a random set of tokens from the training set and trains the model to predict these masked-out tokens. This training objective allows the model to check the context of the text before and after the masked out token. The second pre-training task trains the model to make predictions at a sentence-level instead of a token-level. Here, two sentences are provided and the model is tasked to predict if the second sentence follows the first one. In the training dataset, 50% of the sentences follow each other while the other 50% does not.

BERT utilizes the transformer encoder architecture described in Section 2.3.3. Since this architecture attends to all tokens in a sequence (in contrast to the decoder part, which masks out future tokens), the encoder architecture is inherently bi-directional. This trait, combined with the MLM pre-training task, allows BERT to consider all tokens (past and future) of a sentence when making a prediction.

#### Special Tokens

Several special tokens also have to be included in the input text: [SEP] is used to differentiate between segments of the text, [MASK] is used to mask out tokens during pre-training and [CLS] is appended to the beginning of every sequence as a way to represent the entire sequence in a fixed vector dimension. It is ensured that the CLS token effectively represents the entire sequence through strategic architectural design. Since the CLS token is exclusively

fed to the classification head during training, the backpropagation of errors creates a natural pressure for this token to capture comprehensive sequence information. The model learns that the only way to minimize classification loss is by transforming the CLS token into a vector that encapsulates the most task-relevant aspects of the entire input.

#### Results & Fine-Tuning

BERT manages to beat GPT in popular benchmarks such as GLUE (General Language Understanding) and SQuAD v1.1 (Stanford Question Answering Dataset), which shows that encoder-based models have strong capabilities for a wide range of tasks [64, 49]. What makes the introductory BERT paper groundbreaking is its approach to fine-tuning, as BERT's architecture was constructed with downstream adaptability in mind. By leveraging pre-training on datasets beyond the reach of individual developers, BERT provides robust token embeddings, position encodings, transformer encoder layers, and the CLS representation scheme as transferable foundations. The design allows developers to simply discard BERT's original classification head and attach their own task-specific layer to the pre-trained model, simplifying the application of advanced NLP to specialized domains with modest labeled datasets.

### 2.4.3 KB-SBERT

KB-SBERT is an extension of SBERT (Sentence-BERT), which is a version of BERT specifically designed for sentence-embeddings. SBERT uses two "siamese" BERT encoders with shared weights to encode sentences separately, and then combines the encodings to one sentence embedding. During training, the model learns to minimize the distance between the embeddings of semantically similar sentences and maximize the distance between the embeddings of dissimilar ones. This allows the SBERT models to learn semantic representations of sentences [51]. KB-SBERT uses this architecture and is trained on the Swedish language specifically. It was trained by using a teacher model and a student model. The teacher model is a high-performing English sentence embedding model, and the student model is a Swedish BERT encoder [52]. During training, the student model receives and embeds a Swedish sentence, while the teacher model does the same with the English translation of the sentence. The goal is to minimize the difference between the student model's and the teacher model's outputs. This way, the embedding quality of the English model is transferred to the Swedish model [50]. This project uses the second iteration of the KB-SBERT model, released in 2023 [53].

### 2.4.4 DeBERTa

DeBERTa is a BERT-based model using disentangled attention and a new masking strategy. Instead of combining the token embedding with the positional embedding before the attention mechanism, it keeps them separate and changes the attention formula 2.2 to

$$Attention(Q^C, Q^P, K^C, K^P, V^C) = softmax\left(\frac{Q^C(K^C)^T + Q^C(K^P)^T + K^C(Q^P)^T}{\sqrt{3d_h}}\right)V^C,$$

where $K^C, Q^C, V^C$ are content-based key, query and value vectors, while $K^P, Q^P$ are the position-based key and query vectors. This allows the model to consider relative positions dynamically instead of relying on static positional embeddings [26].

This project utilizes the latest iteration of the DeBERTa models, DeBERTaV3, which changes its pretraining approach described in 2.4.2 to use a generator model trained in the same way BERT-models are trained, and a discriminator network trained with the RTD (Replaced Token detection) objective. The generator model receives sentences with masked out tokens, which it replaces with plausible alternatives, while the discriminator learns to predict

if each token is original or if it was replaced by the generator. This GAN-style Transformer pretraining approach was first introduced with the ELECTRA models [12], and later implemented in DeBERTa [25].

### 2.4.5 Vision Transformer

Vision transformer (ViT) utilizes the transformer mechanism for computer vision with the objective of classifying images [16] [62]. ViT handles an image by splitting it into a set of equally sized patches. For example, an image of size 224x224 might be split up into 196 patches of size 16x16. Since the transformer encoder requires as vector of constant size D, i.e. a one dimensional input, these patches (which are of three dimensions) need to be flattened and then projected to size D. If a patch has an initial size of 16x16x3 it is flattened to a vector of size 768, which is done deterministically.

The projection introduces the first learnable parameter, since the 768 long vector needs to be projected to size D without losing context. In practice, this embedding is performed by a linear layer of size [768,D], called the patch embedding. The positional embedding is added to each patch embedding. In the same way as the original transformer architecture uses positional embeddings with tokens, this vector makes patches differentiable and allows the Transformer to learn how they are correlated.

Before sending the vector through the transformer block, a CLS token is appended, resulting in 196+1 vectors of size D. The CLS token is initially random and interacts with the patch embeddings through self-attention as in the BERT architecture [15]. The final CLS token, outputted by the transformer layer, represents the entire image in one vector.

The last component of the ViT is a multi-layer perception (MLP) head which is appended to the transformer block. The size of this is entirely dependent on the classification task. For example, 150 labels to classify means the final layer of the MLP head will have an output dimension of 150. A prediction from the classifier can be extracted by taking the maximum value of this output layer. This component is also entirely modular; when fine-tuning a ViT for another classification task, this head is discarded and a new one is created and trained. The pipeline of the entire process can be seen in Figure 2.2.

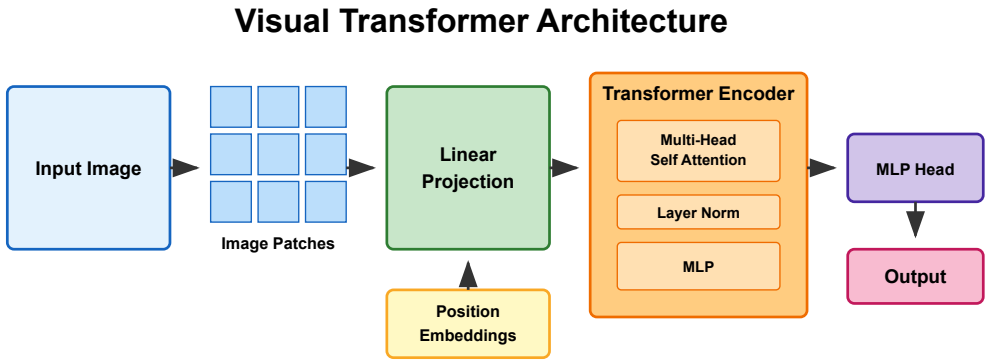



Figure 2.2: ViT architecture

The pre-training task for the ViT was to classify images from the pre-train dataset, which was made possible due to the size of the labeled datasets, where the largest dataset, JFT-300M, contained 303 million labeled images [16]. JFT-300M was constructed through web-crawling and an automatic labeling algorithm. Due to the sheer size of the dataset it is difficult to get a comprehensive view of all of the labels in the dataset. However, in the paper describing the dataset all of the example labels are of objects in the images and not for example document types or text-based categories, suggesting that the dataset primarily focuses on visual object recognition rather than document classification or textual content understanding [60].

In the original ViT paper, the model was pre-trained on different-sized datasets spanning from 1.3M images up to 303M images. This resulted in multiple models that were compared with the most popular architecture for computer vision at the time, the residual net (ResNet) [24]. The authors conclude that while ResNet outperforms ViT for smaller-sized datasets, ViT improves at a much higher rate when the training set increases, displaying promising capabilities for the ViT architecture.

### 2.4.6 Qwen

Developed by Alibaba Group's Qwen Team, Qwen is a series of large language and multimodal models. The Qwen series has undergone significant evolution, with the latest iteration, Qwen 2.5, introduced in 2024. This version encompasses models ranging from 0.5B to 72B parameters, demonstrating robust performance across various benchmarks [69].

Qwen excels in English and Chinese and supports an additional 27 languages, covering regions including Western and Eastern Europe, the Middle East, and Asia [69]. There are no performance metrics in the technical report detailing the model's knowledge of Swedish. However, an independent investigation has shown that Qwen 2.5 demonstrates strong proficiency in both understanding and generating Swedish text [66].

In the context of text classification, Qwen models have demonstrated strong zero-shot capabilities, effectively handling classification tasks without the need for task-specific fine-tuning. When fine-tuned, these models adapt well to domain-specific requirements, enhancing their performance [73]. The models are released under the Apache 2.0 license, making them open source and accessible for modification and integration into various applications [69].

## 2.5 Fine-Tuning

Fine-tuning has become an integral part of AI research and development, partly due to the increasing amount of hardware required to create and train a model from scratch [46]. At its core, fine-tuning is an application of transfer learning, which is a technique that assumes that it is more efficient to teach a model that already understands the broad concepts of domain-specific tasks within that domain, rather than training a specialized model from scratch without any foundational knowledge. In simpler terms, it is easier to teach a model that already "knows" about language to perform specific language tasks (such as sentiment analysis) than to build a sentiment analyzer with no prior understanding of language structure [65].

The concept was popularized in 2019 with the introduction of the language model BERT, since its architecture was pre-built with fine-tuning in mind [15]. However, fine-tuning predates that and has, for example, been used in the popular Alex-net, published in 2012, which was pretrained on the 2011 Imagenet dataset and then fine-tuned on the ILSVRC-2012 dataset, which is an annual image classification competition [36].

### 2.5.1 Full Fine-Tuning

There are multiple ways that a model can be fine-tuned for a new task, some being more computationally demanding than others. Traditional full fine-tuning is a common way to calibrate a pre-trained model, such as BERT or ViT, into a more specific domain.

Using ViT as an example, the model was initially trained on a very large dataset like the JFT-300M set, which contains 303M images [16]. The size of the datasets directly affects the computational costs, for example the largest ViT version took 2.5k TPUv3core days to be pre-trained on the JFT-300M dataset. Using the Google Cloud TPU cost calculator, this would cost an estimated $130 000 to train, for 1 month [21].

After the model has been pretrained on the JFT-300M dataset its final output layer would be of size 18,000, corresponding to the 18,000 different classes in the dataset. When a ViT is

fine-tuned for another dataset, this final layer is discarded and replaced with a zero-initialized linear layer with the same output size as a number of classes in the new dataset [16]. The full fine-tuning process now involves training the weights of this linear layer to accurately predict the new labels while also updating the weights of the base model to better fit the fine-tune domain. This process is less computationally demanding and does not require as much data as pre-training a model [11]. An example of how a ViT changes after fine-tuning can be seen in figure 2.3

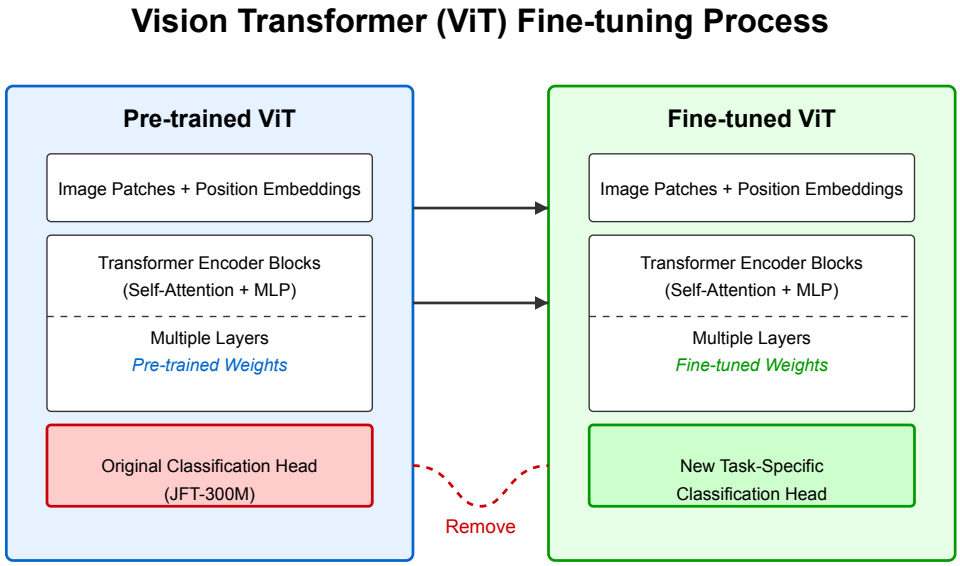



Figure 2.3: How a ViT changes during fine-tuning.

### 2.5.2 PEFT-Methods

When adapting large language models, it is often computationally impractical to update all the model parameters. Parameter-efficient fine-tuning (PEFT) methods address this issue by restricting the number of learnable parameters during fine-tuning, thereby reducing both memory and computational overhead [30].

One of the most commonly used PEFT methods is Low-Rank Adaptation (LoRA). In LoRA, rather than updating a full-weight matrix $W \in \mathbb{R}^{d\times k}$, the update is restricted to a low-rank decomposition. That is, the change in the weight is represented as

$$\Delta W = AB, \quad A \in \mathbb{R}^{d\times r}; B \in \mathbb{R}^{r\times k},$$

so that the adapted weight is

$$W' = W + \Delta W = W + AB.$$

Here, $r \ll \min(d,k)$ is the rank constraint that significantly reduces the number of trainable parameters while still allowing the model to capture important task-specific variations [31].

QLoRA improves on LoRA by using 4-bit quantization (NF4) with a logarithmic scale, enhancing memory efficiency without sacrificing performance. This method provides better precision for smaller values in model weights. QLoRA also employs double quantization, where both weights and quantization factors are quantized. This keeps a small set of scaling

factors in full precision, while the rest are stored in lower precision, reducing memory usage while maintaining performance. Furthermore, QLoRA uses paged optimizers to offload memory-intensive optimizer states (such as gradients) from GPU VRAM to CPU RAM or disk, making it possible to fine-tune models on GPUs with limited memory. These improvements often lead to faster training times without reducing model performance significantly [14].

For larger models where even low-rank approaches may be resource-intensive, an alternative method known as BitFit can be used [7]. BitFit restricts fine-tuning to only update the bias terms in the model. Consider a typical linear transformation in a transformer layer:

$$Z = WX + b.$$

In BitFit, the weight matrix $W$ remains fixed while only the bias vector $b$ is updated as

$$Z = WX + (b + \Delta b),$$

with $\Delta b$ denoting the trainable bias adjustments. Despite its simplicity and extremely low parameter count, BitFit has been shown to yield competitive performance on several downstream tasks [7].

## 2.6 Fusion

An issue with the models described above is their uni-modality, meaning their limited capabilities of only being able to handle one dimension. While the BERT model is good at interpreting textual information, some tasks require other dimensions as well. Take, as an example, a model for medical diagnosis; if the model only can rely on textual information like patient-reported symptoms, family history, etc., it will miss all the visual information, like bruises, x-rays, and CT scans. This limits the model's diagnostic capabilities, as visual data provides crucial information that text alone cannot capture. It is therefore very important to evaluate what sort of information would help a classifier increase its accuracy, and how information from different modalities can be incorporated to construct a multi-modal representation of information.

A way to incorporate information from different modalities is to utilize two separate pre-trained models, for example, a text model and a vision model, encode both representations and then combine them. The technique of combining representations from different modalities is called fusion. Fusion techniques can be reduced to two variations: early fusion and late fusion [70].

### 2.6.1 Late Fusion

In late fusion, two unimodal models that work well individually make their separate predictions on the same input and then combine these predictions by, for example, voting or taking the most confident prediction [18]. While a late fusion module is easier to construct, it fails to combine the information from two modalities. Take, as an example, sentiment analysis of a sentence where text is combined with the facial expression of the person saying the sentence. If the text is "Nice shoes!" but the facial expression is "smirking", a late fusion approach would struggle to capture the true sentiment.

### 2.6.2 Early Fusion

The alternative to late fusion is to extract the modality representations before the final output from the models. The rationale behind this approach is that by extracting a representation of the input (for example, the CLS token in BERT/ViT) before a prediction is made, a numerical encoding of how the pre-trained model views the input is obtained [68]. This numerical

encoding can then be combined with the other numerical encoding to create a representation of an input spanning across different modalities. The differences between the two techniques can be seen in Figure 2.4.

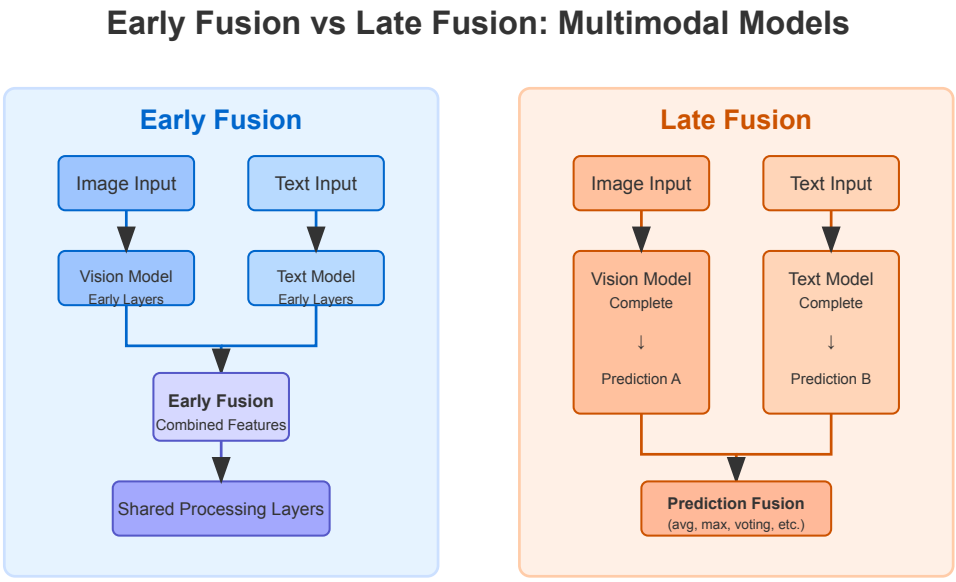



Figure 2.4: Comparison between early fusion and late fusion.

### Simple Operation Fusion

There are multiple ways to combine vector representations from different modalities [70]. Simple operation fusion combines two modalities by concatenating or adding them together. For two vectors $X_1 \in \mathbb{R}^{d1}, X_2 \in \mathbb{R}^{d2}$ where $d_1 \neq d2$ is permitted

$$F_{concat}(X_1, X_2) = [X_1; X_2].$$

The issue with this fusion technique is that it fails to balance the two modalities, so if one is dominant over the other, this method expects that the subsequent layers in the fusion model will fix this. Another issue is that the dimension space can become high $(d_1 + d_2)$, which may result in the "curse of dimensionality".

Another simple operation fusion technique is to calculate the weighted sum. Using the same vectors as in the previous example, this method can be described as

$$F_{wsum}(X_1, X_2) = \alpha_1 X_1 + \alpha_2 X_2,$$

where $\alpha_i$ can either be a learnable or constant scalar. An issue with this method is that $d_1 = d_2$ must hold. If $d_1 \neq d2$ we will have to transform each vector

$$T_i(X_i) = W_i X_i + b_i$$

resulting in the equation

$$F_{wsum}(X_1, X_2) = T_1(\alpha_1 X_1) + T_2(\alpha_2 X_2).$$

Weighted sum fusion can introduce a lot of learnable parameters, but avoids increasing the dimensionality of the combined vector to a size larger than any of the two single vectors.

It also allows the model to decide which modality is the most important, and should therefore contribute the most to the final fused representation.

An issue with weighted sum fusion is that even if the weight used to balance modalities is learnable, when the model is finished training, this weight becomes fixed. Different data points may require different priorities between the modalities, but due to the fixed weight, this dynamic adaptation is not possible.

#### Complex Fusion Methods

Another fusion technique is attention-based fusion, where the attention mechanism from the transformer block is utilized to perform fusion of modalities [62, 70]. An example of a model that uses this technique is ViLBERT, which can be seen as a multimodal extension of BERT [39]. The word input is tokenized, encoded and sent through a transformer much like the original BERT. For visual input, ViLBERT uses a CNN that extracts regions of interest from an image and converts them to a vector, which is projected to a fitting size. To this vector, a special IMG token is appended, which acts as the visual representation of a CLS token. After sending this vector through a transformer, it is sent through a cross-attention transformer layer, which lets the textual part attend to the visual input and vice versa. This allows the modalities to exchange information. The output is then a sequence of contextualized representations, including the CLS token and IMG token, which can be used for downstream tasks such as classification.

Another approach to attention-based fusion is to send the textual and visual representations to a single transformer, encouraging the model to discover a new joint representation of the modalities. This approach was proposed and implemented in the VisualBERT paper and is a more lightweight alternative to using multiple transformers for fusion [37].

## 2.7 Evaluation Metrics

Once predictions have been made, it is essential to analyze the results to get an understanding of the benefits and shortcomings of the predictors. This section introduces the fundamental metrics commonly encountered in classification projects that help quantify performance and guide improvement efforts.

### 2.7.1 Computational Efficiency

In data science, there are a lot of different metrics that can be used for calculating computational efficiency. The issue with computing efficiency is the variation of computing power in a single unit. While there exist metrics like time complexity, which only rely on the algorithm that is measured, this may be difficult or even impossible to calculate for large and complex projects. Another popular metric is FLOPS (floating point operations per second), which counts the number of floating point operations per second required to perform some computation. This metric is often used in scientific papers that present or compare AI models [58][16].

Another approach is measuring the training and inference time of a model. While this metric can vary significantly across different hardware configurations, it remains valuable when system specifications are clearly documented or when the primary goal is comparing relative performance between different model architectures.

### 2.7.2 Classification Performance Metrics

In machine learning, it is essential to quantify the results of a classifier using metrics like accuracy, recall, precision, and F1-score.

### Accuracy

Accuracy is calculated by dividing the number of correct predictions by the total number of predictions made. This is a very simple metric and can at times be misleading. For example, if there is a high class imbalance, a good accuracy may only indicate that the predictor is good at making predictions for a frequent class, but miss some classes that are not represented as well.

### Confusion Matrix

Using a confusion matrix, it is possible to calculate all other metrics that are presented in this section. An example of a confusion matrix for a binary classifier can be seen in Figure 2.5. For a perfect classifier, all predictions should be on the left diagonal of the matrix. The matrix size increases with the number of classes to predict, so for a dataset with 15 classes, the matrix size would increase to 15x15.

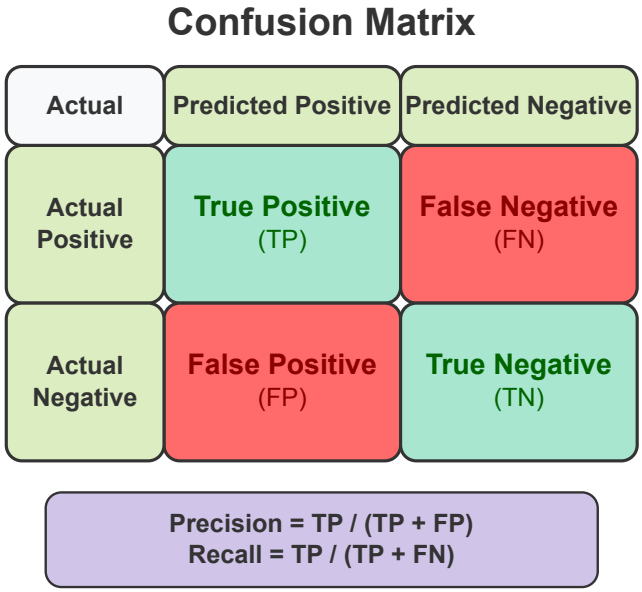



Figure 2.5: Confusion matrix example for binary classifier.

### Precision & Recall

Precision and recall provide deeper insights into the predictions [23]. Precision describes how precise the predictor is: if a model says that something belongs to class A, how often is it correct? Meanwhile, recall tells us how complete the predictions are: out of all actual instances of class A, how many did the predictor find?

An example of how to calculate these two metrics for a binary classifier can be seen in Figure 2.5. For a multi-class classifier, there are three ways to calculate precision and recall depending on what information you are interested in: micro-averaging (which aggregates contributions of all classes to compute overall metrics), macro-averaging (which calculates metrics for each class independently and then takes their unweighted mean), and weighted-averaging (which calculates metrics for each class and takes their average weighted by the number of true instances). Both precision and recall span between 0 and 1, where 1 is a perfect score.

### F1 Score

F1 score represents a balanced metric between precision and recall and is calculated through the formula $f_1 = 2\frac{p \cdot r}{p+r}$, where $p$ is the precision and $r$ is the recall. F1 score ranges between 0 and 1, where 1 indicates perfect precision and perfect recall. As with precision and recall, F1 score can be calculated as either micro average, macro average, or weighted average.

### Perspective on Aggregation Strategies

To better understand how precision, recall, and F1 score can vary depending on which aggregation strategies are used, an example is provided using the confusion matrix seen in Table 2.1.

| | **Predicted A** | **Predicted B** | **Predicted C** |
|---|---|---|---|
| **Actual A** | 800 | 150 | 50 |
| **Actual B** | 30 | 60 | 10 |
| **Actual C** | 20 | 5 | 25 |

Table 2.1: Example Confusion Matrix for Three-Class Classification

The total accuracy of this predictor is the sum of the diagonal divided by the sum of all values, resulting in 0.769. Table 2.2 displays the micro average, macro average, and weighted average for recall, precision, and F1 score based on the confusion matrix in Table 2.1.

As can be seen, the micro averaging gives the same value as the accuracy, which is always the case when calculating the micro average for values from a single-label multiclass classification task (when an instance can only belong to one of multiple labels). For the other aggregation strategies, the values are noticeably different. When the weighted average is significantly higher than the macro average, as in this case, it indicates that the model is better at predicting frequent classes over infrequent classes. This is true in this example case since the accuracy for predicting the most frequent class A is $800/1000 = 0.8$ while the accuracy for the most infrequent class C is $25/50 = 0.5$. If the values had been switched, it would indicate that the predictor was better at predicting infrequent classes than frequent ones.

| | **Micro Average** | **Macro Average** | **Weighted Average** |
|---|---|---|---|
| **Recall** | 0.7696 | 0.6333 | 0.7696 |
| **Precision** | 0.7696 | 0.5048 | 0.8555 |
| **F1-score** | 0.7696 | 0.5387 | 0.8013 |

Table 2.2: Matrix with different aggregation strategies

# 3 Method

This chapter outlines the process of going from an unstructured dataset to testing and evaluating classifiers. During this process, a lot of choices and assumptions are made that may directly affect the outcome of the project. Because of this, it is important for replicability and future work that these decisions are thoroughly documented and motivated.

## 3.1 Dataset

A crucial first step in any machine learning project is establishing a properly structured, labeled dataset of sufficient size. The quality and preparation of this element often determines the ultimate success or failure of the model. Beyond just labeling, the data must be in a format that models can effectively process and learn from. This section outlines the process of going from an unstructured collection of documents into a robust, labeled dataset.

### 3.1.1 Preprocessing

The dataset that this project uses is a compilation of documents sent to a Swedish public agency during the span of 6 months. The total size of the dataset is 140,338 files in varying file formats, including: pdf, docx, pptx, png, mov, mp3, mp4, etc. Since this project only handles images and text data, file formats like mov, mp3 and mp4 can be excluded since they represent a different modality. PowerPoint files and Excel files are also excluded partly because they are difficult to work with, but also because of their low quantity. It was deemed unnecessary to try to include them through a separate pipeline when these two filetypes make out 4 of the total 140,338 files. Finally, this leaves us with pdf files and the different image formats. These images are converted to pdfs leaving us with a total of 139,854 files.

The files were manually annotated according to their corresponding category by the agency this project is working with. However, this naming convention is not uniform and can include misspellings or synonyms that refer to the same label. This project assumes that, apart from misspellings, the labels are correct and can be used as a ground truth during training and testing models. Filenames that are deemed similar enough are manually grouped together under the same label. For example, files under the label "text message" can be named "chat", "chat conversation", or "text message".

Since this dataset will be used for training, testing, and validation, it is important to ensure an adequate number of instances for each class. What counts as adequate varies depending on how advanced the classification task is and how specialized the domain is [71, 56]. In this project, labels that have at least 1000 instances of them are included, which results in 15 classes consisting of a total of 55,892 files. This is a large drop form the original 139,854 but since the aim of this project is not to accurately classify the entire dataset of a Swedish government but to evaluate the classifiers, dataset pre-processing was not the primary focus.

### 3.1.2 Text Extraction

To apply the data to different text models, raw text must first be extracted from the pdf files. This is done through Optical Character Recognition (OCR) using the Tesseract engine [22]. To save on memory and time, this project assumes that the first page contains enough information to classify it, and discards the rest. This decision was motivated by investigating a set of files for each label. Here it was deduced that most files only consisted of one page, and in the cases where more pages were present, the first page contained enough information to be identifiable.

The first page of each file is loaded with the PyMuPDF package [3] and converted to an image using a dpi value of 150. The image is then passed through the Tesseract OCR engine, and the text is saved to a txt file with the same name as the pdf file. The smaller txt format enables smoother downstream data handling. Since Tesseract would sometimes fail to capture any content, the text extraction step resulted in 52,185 labeled text files.

The process of first converting the files to images and then using OCR to extract the text can seem counterintuitive since pdf files often contain embedded text in them, which can be extracted directly. However, since the dataset also contained scanned images of documents, the embedded texts could at times only contain date marks or other types of meta data that is not relevant to the classification task. By only extracting this text, important information that is not embedded in the file would be missed. When treating all documents as images we ensure that all visible text on the first page is parsed and that all files are treated equally, disregarding wether they have embedded text in them or not.

### 3.1.3 Image Conversion

When investigating the dataset it was noted that many files could be manually classified before reading the text in them. For example, the "text message" class is very easy to spot since the chat bubbles often follow a similar format. The same goes for the "receipt" class, which in many cases is a picture of a printed-out receipt. Finally the "photo" class is a very broad class and can contain any sort of picture which in most cases does not contain any textual information at all.

Because of this, it was of interest to investigate visual classifiers as well. These classifiers take an image of a specific size as input, which means that all pdfs in the dataset need to be converted to pngs. The pdfs are converted through a Python script using the PyMuPDF library [3]. Since visual classification models only accept a single image as input and pdf files often contain multiple pages, the same assumption made during the text extraction step was taken. During the conversion process, about 10% of the images failed to convert due to their resolution, resulting in 51,102 labeled images.

### 3.1.4 Dataset Analysis

When extracting text and images from the original dataset, some files were unable to be processed. For example, if a document does not contain any text, the text extraction process fails. During the image conversion process, some images were too large, causing a decompression bomb error. To be able to fairly evaluate the classifiers and not the effectiveness of the text

parser or image converter, the extraction error needs to be accounted for. The distribution of extracted files can be seen in Figure 3.1

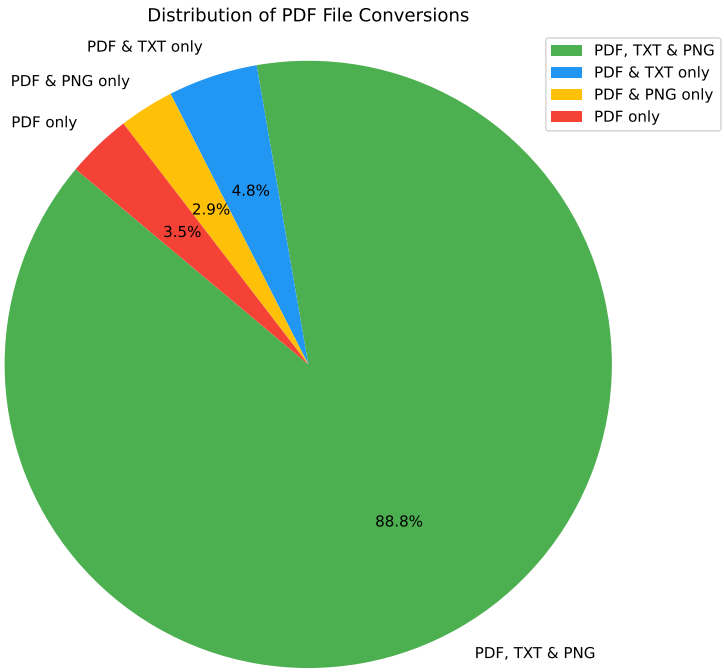



Figure 3.1: Distribution of successfully extracted files

Figure 3.2 displays how the classes are distributed and the results of the file conversion process for all individual classes.

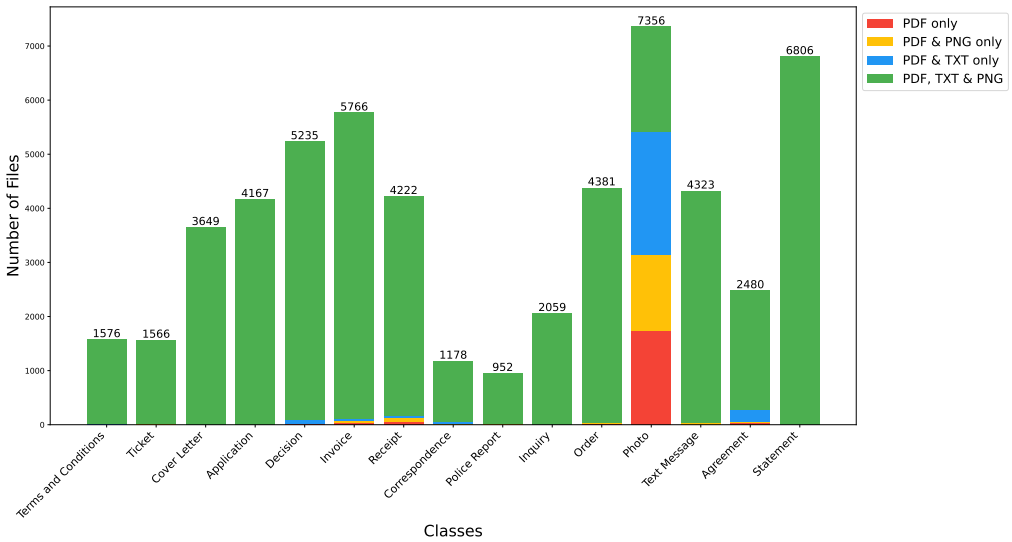



Figure 3.2: Label distribution of extracted files

## 3.2 Model Comparison

This section presents the document classification methodology, beginning with the model selection process, followed by motivating the choice of evaluation metrics to assess performance across different modalities, and concluding with the development of a human benchmark to contextualize the results.

### 3.2.1 Model Selection

Due to the impressive capabilities of popular proprietary language models like Claude and ChatGPT, it would be valuable to compare their performance with the models presented in this paper. However, the sensitive nature of the documents being classified makes it problematic to send them to external servers where control over the data would be lost. This study will therefore utilize models that can be directly loaded and used on a local network.

All models selected for this project are publicly available through the HuggingFace Hub. They were chosen based on either their superior performance or because their training objectives closely align with the goals of this project.

Since the domain of the classification task is in Swedish, it is beneficial to look at encoders pre-trained on a Swedish dataset [52]. Due to a lack of existing fine-tuned or pre-trained vision models in the Swedish domain, this project will only be able to use base versions of vision models. A vision model that had been pre-trained on a large corpus of Swedish texts in images may have performed better at classification in this domain. However, since most vision models do not have the capability to read text, the improvement would not be substantial without specific text recognition capabilities.

### 3.2.2 Choice of Metrics

Since we have separate dataset versions for each modality, calculating accuracy solely within each modality-specific dataset would provide an incomplete picture. Dividing correct predictions by the total files in each modality's dataset fails to account for files that could not be processed in that modality. Even though a failure to extract text from an image is not inherently the fault of the text model, it is still a file that the text model could not handle. Therefore, when evaluating the models, two accuracy metrics will be reported: model accuracy, which considers only files where the model successfully made predictions, and total accuracy, which includes all files in the original dataset, treating extraction failures as incorrect predictions.

This report will also use F1 score since it aggregates both recall and precision in a single metric. F1 score will be calculated both as weighted average and macro average. By comparing these two metrics, it is possible to gain valuable insights into the model's performance across both under-represented and over-represented classes.

Furthermore, confusion matrices and per-class accuracy for the classifiers will also be presented for deeper analysis.

Since one of the research questions in Section 1.3 concerns the trade-off between classification accuracy and inference time, a combination of these measures would become a useful metric to aid the discussion. We combine these measures into an operational value of a single classification solution, which we call model score. Since different situations can favor speed or accuracy, a tunable score can be constructed as

$$S(\lambda) = \lambda \cdot A + (1 - \lambda) \cdot \frac{\log(T) - \min(\log(T))}{\max(\log(T)) - \min(\log(T))},$$

where $A$ is the accuracy, $\lambda \in [0, 1]$ is a parameter determining the weight on accuracy, and $T$ is the throughput, which is normalized to account for larger relative changes than in accuracy. This creates an opportunity to evaluate which model performs best depending on the accuracy-throughput trade-off of the application.

### 3.2.3 Human Benchmark

To provide context for the results from all classifiers and assess the quality of the dataset, a human benchmark will be presented. This was constructed by having two people manually classify documents from the dataset. To ensure that the class distribution was preserved, the number of files in each label matches the distribution of the actual labels. The annotators were given time to "pre-train" on the classes beforehand.

Since manually classifying documents like this is a tedious task, an application was constructed using HTML to enable faster classification. An image of the starting page can be seen in Figure 3.3

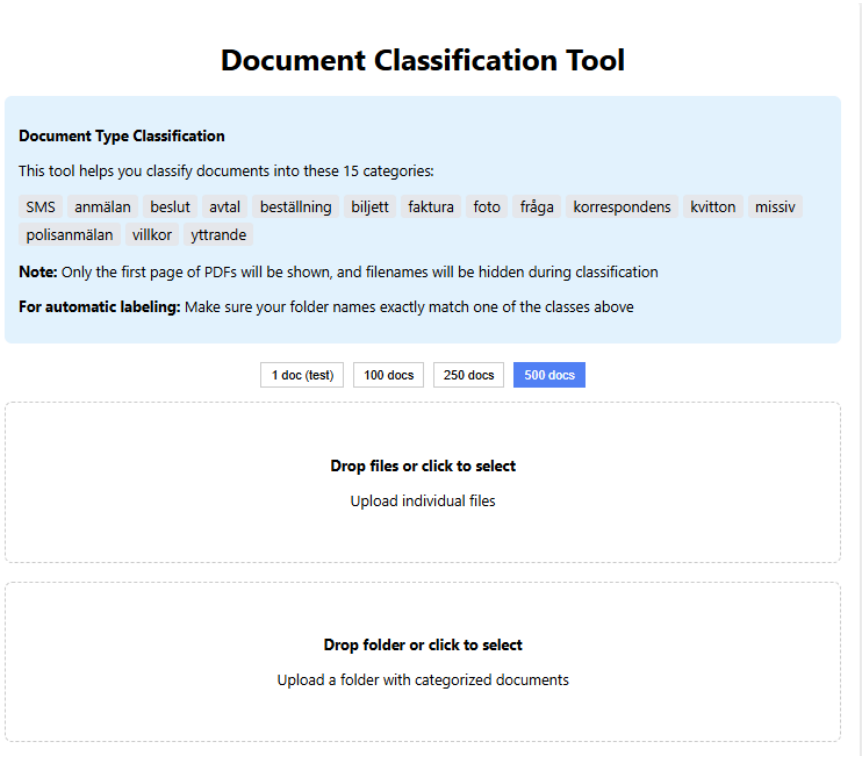



Figure 3.3: First screen for manual classification

After selecting the number of files to classify and uploading a folder where each sub-folder corresponds to a label, users are directed to the page seen in Figure 3.4.

Here, a view of the pdf is presented as well as the 15 label options and a progress bar. When all documents have been classified, a report is displayed showing the total accuracy and a confusion matrix. This can be seen in Figure 3.5

## 3.3 Fine-Tuning

This section outlines the process of fine-tuning a pre-trained model to a specific dataset. Three models were fine-tuned in this project: two encoder-based and one decoder-based.

### 3.3.1 KB-SBERT and DeBERTa

The process of fine-tuning a BERT model is very simple since it is an aspect built into the architecture of the model, as discussed in the background section. Using the parsed texts, described in Section 3.1.2, a custom dataset from the torch.utils.data package is constructed where each text is mapped to a label. Since most BERT models have a context window of

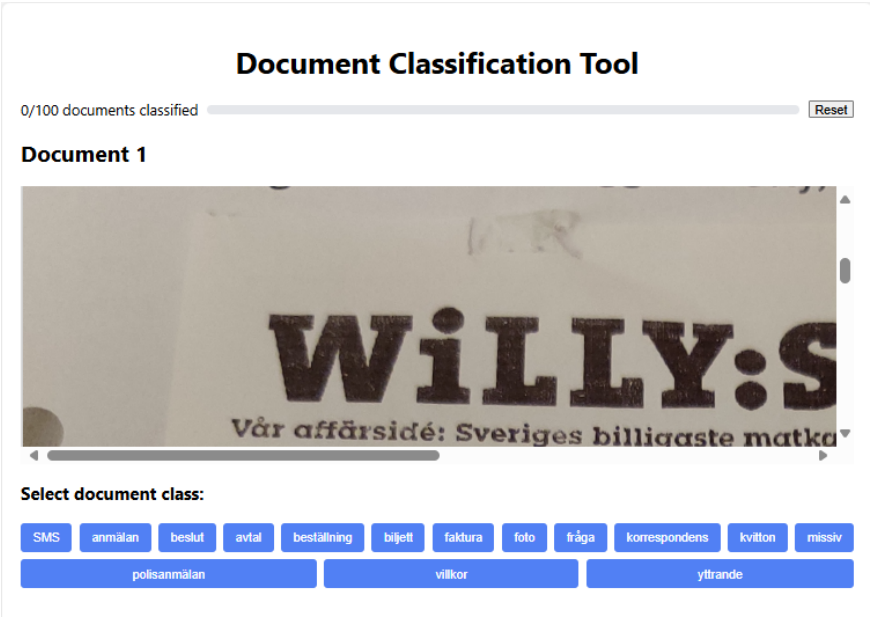



Figure 3.4: During classification interface

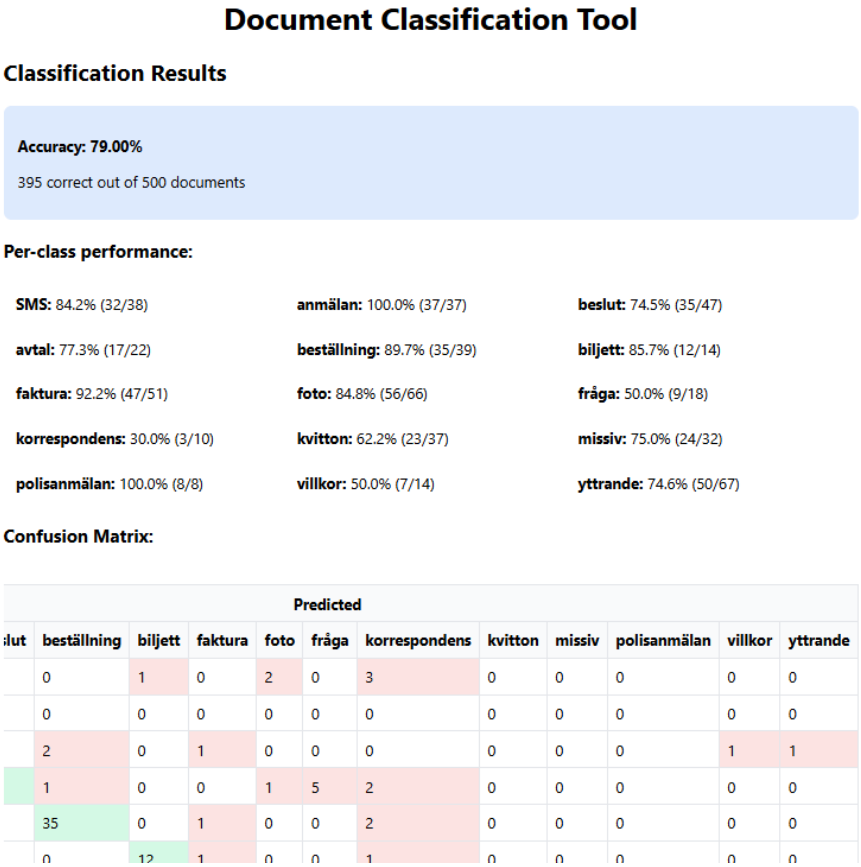



Figure 3.5: Results screen

512 tokens, which means that the model can only handle 512 tokens at a time, each text is truncated to that length during the tokenization process.

The base model "KBLab/sentence-bert-swedish-cased"[1] and "microsoft/deberta-v3-base"[2] is fine-tuned on 20% of the total dataset, using the Transformers Trainer with cross-entropy loss as loss function; this is the standard for AutoModelForSequenceClassification [35, 27]. A comprehensive list of hyperparameters can be seen in the Appendix B.

### 3.3.2 Vision Transformer

The vision transformer is fine-tuned in a similar fashion to KB-SBERT and DeBERTa. Here, "google/vit-base-patch16-224"[3] is used, which is the standard ViT version [17]. Patch 16 means that each image is split into 16 patches. By increasing this number the patches get smaller, resulting in a higher computational demand. 224 refers to the image size, in pixels that the vision transformer handles.

Before fine-tuning the ViT the images need to be pre-processed. Firstly, the image dataset described in Section 3.1.3 is converted to an .arrow dataset, which is a compressed version of the dataset that can be directly accessed using the transformer trainer. Each time an image is fetched from the dataset, it is resized to 224x224, which is done using the a built in torchvision transformers function. This function randomly crops and resizes an image each time it is called, which means that the model is shown a different segment of the image for each epoch. After resizing, the image has a 50% probability of being flipped on the horizontal axis. The process of data augmentation is common for training image models and is used to reduce overfitting and improve generalization [36, 16].

The model is then fine-tuned using the transformer Trainer on the same split as KB-SBERT and evaluated using cross-entropy loss. Hyperparameters used for fine-tuning can be found in the Appendix B.

### 3.3.3 Decoder-Based Classifier

There are many ways of adapting a decoder-only model for a classification task, such as using prompting strategies or constraining output vocabulary [13]. This project will use a simple classification head on top of the final decoder output, following previous work [34]. We remove the LM head from a Qwen2.5-3B[4] model, and add a classification head in its place. Since the model uses masked attention, as described in Section 2.3.5, only the final token is fed into the classification head, which contains information about the entire input sequence. Using QLoRA, 0.06% of the total model parameters are trained and updated for five epochs. More details on the hyperparameters used can be seen in Appendix B.

## 3.4 Fusion Model Construction

Some of the classes in the labeled dataset may be difficult to classify by only relying on one modality. Files in the "photos" class, for example, often do not contain any textual information that would be relevant for a text classifier like BERT. The same goes for only relying on visual information, where some classes like "report" and "statement" both have the same template (provided by the government agency), meaning that the visual context may not be enough information. Due to this, it is of interest to see if the modalities could be combined to create a meaningful fused representation of the document content. This is done using the fusion techniques described in the background chapter where the visual modality comes from a ViT and the textual modality comes from a SBERT.

[1] https://huggingface.co/KBLab/sentence-bert-swedish-cased
[2] https://huggingface.co/microsoft/deberta-v3-base
[3] https://huggingface.co/google/vit-base-patch16-224
[4] https://huggingface.co/Qwen/Qwen2.5-3B

### 3.4.1 Complex Early Fusion

This model is built on the premise that meaningful representations from two distinct modalities can be extracted from separately fine-tuned pre-trained models prior to making predictions (in the form of CLS-tokens). These representations are then combined in a balanced way that utilizes the strengths of both representational formats

#### Dataset

For this version, the dataset is split into four parts: train fine-tune (40%), test fine-tune (10%), train fusion (40%), test fusion (10%). Half of the dataset is used to fine-tune the two base models, and the other half is used to train and test the fusion module.

Since the base-model weights are frozen when training the fusion module, half of the dataset (used to train the fusion part) is directly converted into two sets of CLS tokens, one from the fine-tuned KB-SBERT and one from the fine-tuned ViT. By doing this, the dataset used to train the fusion module shrinks substantially since we now only have to work with CLS tokens instead of images or texts, which takes less disk space. Apart from shrinking the dataset size, the training of the fusion module becomes less computationally demanding since the conversion from an image or text to a CLS token is skipped. The tokens are initially of dimension [batch_size, embedding_dim] where the embedding dimension is 768.

#### Architecture

The two CLS tokens are first projected separately in two linear layers of the same dimensions. The hope is that this teaches the model to extract key information about each separate modality. After projecting the two tokens they are stacked together, resulting in the shape [batch_size, 2, embedding_dim]. This entire representation is then sent through a transformer block which has 8 heads and 6 encoder layers, the same as in the original paper [62]. Increasing these would make the model more complex and possibly more accurate, but could also run the risk of making the model too computationally demanding. The fusion technique used in this architecture can be seen as a combination of ViLBERT and VisualBERT since the model uses a single transformer for fusing (like VisualBERT) and processes each modality separately before concatenating (like ViLBERT) [39, 37]. However, this technique is still quite different since this implementation only handles the CLS tokens from both base models, making it more streamlined. Apart from that, the two original CLS tokens are appended to the transformer CLS output, acting as a residual connection step. This is not employed by either VisualBERT or ViLBERT. This final CLS representation is then sent through a classification head with the shape [embedding_dim, num_classes] where the max value is the final prediction.

These design choices are motivated by a combination of theoretical background and testing. If two transformers for each modality had been used, like the ViLBERT architecture, the model would become too complex for our hardware. By using the final representations from the modalities (CLS tokens), it is easier to fall back in cases where one modality could not be processed, for example due to a lack of text. The residual connection step was motivated by the fact that it is often used in other popular architectures where it could be beneficial to "remind" the forward pass of the original representation [16, 36, 15]. The main thought behind the architecture is for it to be as similar as possible to the original BERT and ViT architectures, but where we send the concatenated ViT and BERT CLS-tokens to the transformer block instead of token or image embeddings.

Creating a novel architecture gives much more control over all hyperparameters and the entire classification process. This allowed us to switch the uni-modal feature extractors (for example, to use KB-SBERT instead of a base BERT model). The same goes for using a ViT instead of a CNN for the visual modality, which is not as explored in previous works, and apart from CLIP, we could not find any multi-modal model that had undergone extensive

pre-training which used a ViT [47]. This control is very useful when working with a low-resource language in this domain since the already existing pre-trained multi-modal models are optimized for high resource languages which may result in non-optimal results when applied to our domain.

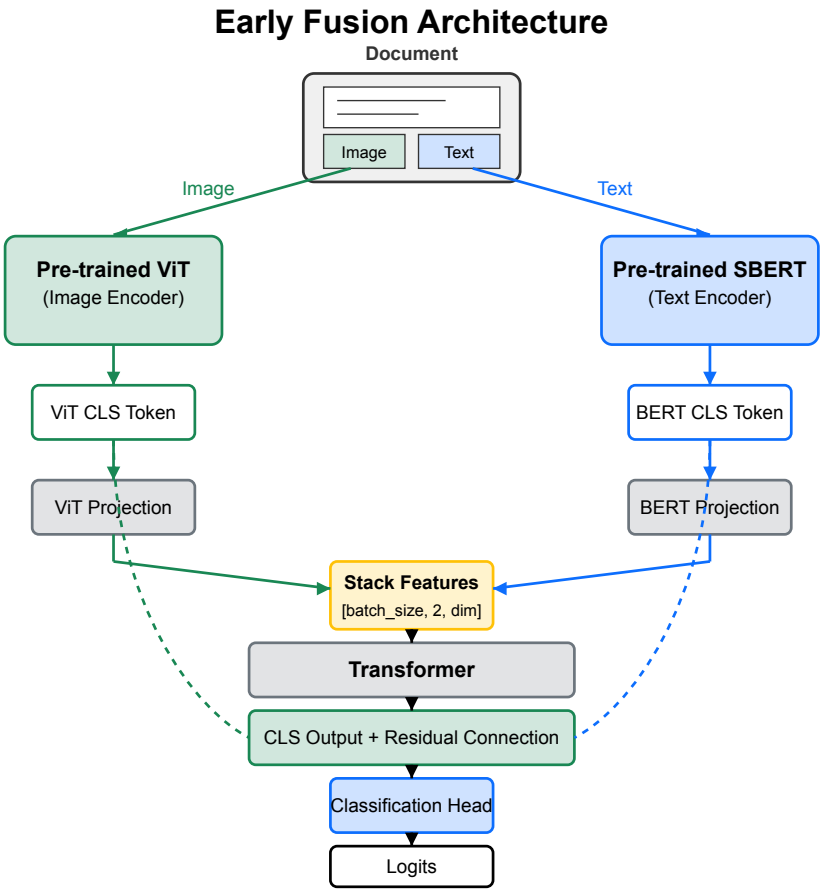



Figure 3.6: Complex early fusion architecture

Figure 3.6 shows the entire architecture for the early fusion module. Cross-entropy loss is used to calculate the loss and is propagated to the ViT/BERT projection blocks.

### 3.4.2 Simple Operation Early Fusion

A second, simpler version of the pre-fine-tuned early fusion architecture is constructed where weighted sum fusion is used instead of attention-based fusion. This uses the same dataset split as complex fusion. Instead of transformer-based fusion, the two embeddings are stacked and then multiplied by a vector of shape [1,2,1], which acts as the weight. After multiplication, the new vector is summed along the modality dimension, resulting in a vector of size [batch_size, embedding_dim] which is sent through a classification head for predictions. The reasoning behind constructing and testing this model is to compare it to the results of the complex fusion architecture to verify whether the complex technique results in a better predictor.

### 3.4.3 Fully Trained Early fusion

Finally, a third early fusion version is constructed, which follows the same architecture as in Figure 3.6 but where the base models are included in the training loop. What this means is that all parameters of both models are updated during training and not just the new parameters under the fusion block. Since the vision and text models are included during backpropagation, they are not fine-tuned beforehand like the previous versions.

This version does not fall back on any single classification head if a modality is missing, but instead sets that missing CLS token to a zero vector of size embedding_dim. The previous fusion model relied on the fine-tuned "google/vit-base-patch16-224" that was substituted with "google/vit-base-patch32-224-in21k"[5] which is smaller, to reduce training time.

Since the entire architecture is trained during backpropagation, the original dataset is split into two parts: a training set (80%) and a test set (20%). When fine-tuning the ViT separately, data augmentation was performed to enable generalization, but since we now want to ensure that both modalities are receiving similar information, no data augmentation is performed. Instead, each image is pre-processed using the built-in feature extractor function, which resizes and rescales the images automatically.

### 3.4.4 Late Fusion

For the late-fusion architecture, an alternative approach for deciding which prediction to use is tested. Early testing has shown that the KB-SBERT outperforms the ViT as an independent classifier. By taking this into account, the late fusion architecture relies on the predictions from the fine-tuned KB-SBERT model (used for the complex early fusion architecture) for all documents except in cases when text could not be extracted. In this case, the predictor is switched to the fine-tuned ViT. As stated before, the late fusion architecture is very simple but fails to actually combine the context of two modalities. The reason for constructing and testing this model is to be able to compare its results with the early fusion architecture to see if early fusion is able to combine textual and visual information.

### 3.4.5 Hierarchical Fusion Module

Another approach is to let the models make predictions sequentially, starting with coarse predictions that become more fine-grained as more details are examined. This mimics how humans perceive and process the world [43]. In the context of this project, the first coarse prediction is made with a ViT and its output is used as a feature for the final BERT classifier. This architecture can be seen in Figure 3.7. This is preferred over a tree-like structure where different models are used depending on the first prediction, since this is better suited to handle situations where the ViT makes an incorrect prediction.

Since the BERT model lacks support for handling additional features, this is handled implicitly. One solution is to represent the coarse predictions as features in the classification head. This is done through a concatenation of a prediction and the CLS token from the BERT model. Another approach is to simply prepend the input text with the coarse classification, before the BERT model encodes it. This study examines both approaches.

[5] https://huggingface.co/google/vit-base-patch32-224-in21k

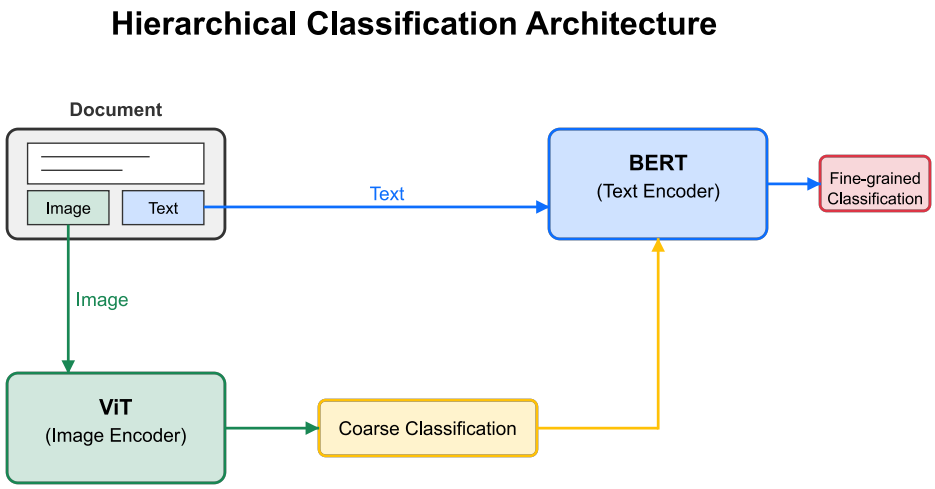



Figure 3.7: Hierarchical approach for document classification.

# 4 Results

In this chapter, the results of the different tests are presented. First, performance metrics for each model tested are presented. This is then followed by confusion matrices and per-class accuracies for specific models. Then, the results using the Score metric presented in Section 3.2.2 are presented, followed by training and fine-tuning metrics.

## 4.1 Compiled Results

A compilation of all the tested models can be seen in Table 4.1. Model Accuracy refers to the accuracy reported by the model, correct predictions divided by all predictions, while Dataset Accuracy refers to the correct predictions divided by all files in the dataset. This means that Dataset accuracy will be lower, since the denominator includes files that failed to go through preprocessing, as described in Section 3.2.2. Throughput refers to the measure at inference time, and is presented in files per second. All tests were conducted on a machine with 4GiB of VRAM. It includes tokenization but does not include preprocessing time, such as OCR. The best results are highlighted in bold.

The table shows that the fine-tuned models generally perform better than other models. The complex early fusion model is the most accurate, both in terms of model accuracy and dataset accuracy. The fine-tuned text-only models perform well on the data it could handle, reaching similar model accuracies while dataset accuracies decrease. Looking at model throughput, the decoder-based classifier is shown to be the slowest, comparable to the human benchmark. The fusion models are generally an order of magnitude faster, and the text-only models are themselves an order of magnitude faster than the fusion models. The Naive Bayes classifier reaches the highest throughput by far, two orders of magnitude faster than the encoder models.

| Model | Model Acc. (%) | Dataset Acc. (%) | Macro avg F1 | Weighted F1 | Throughput (files/s) |
|---|---|---|---|---|---|
| **Fine-tuned Encoder Models** | | | | | |
| Fine-tuned ViT | 85.10 | 75.77 | 0.8290 | 0.8496 | 6.1 |
| Fine-tuned KB-SBERT | 89.34 | 83.22 | 0.8828 | 0.8931 | 20.7 |
| Fine-tuned DeBERTa | 89.23 | 83.14 | 0.8762 | 0.8918 | 13.9 |
| **Fusion Models** | | | | | |
| Late fusion | 88.59 | 84.95 | 0.8663 | 0.8828 | 2.7 |
| Fully trained Early fusion | 75.09 | 73.37 | 0.7443 | 0.7638 | 0.8 |
| Simple Early fusion | 87.10 | 82.38 | 0.8554 | 0.8714 | 2.7 |
| Complex Early fusion | **89.92** | **86.23** | **0.8834** | **0.8981** | 2.7 |
| Hierarchical Text | 88.81 | 82.68 | 0.8694 | 0.8863 | 3.1 |
| Hierarchical Features | 86.92 | 80.92 | 0.8494 | 0.8675 | 2.7 |
| **Other Models** | | | | | |
| Decoder-based Qwen2.5-3B | 83.90 | 78.19 | 0.8274 | 0.8369 | 0.1 |
| CLIP | 32.00 | 28.50 | 0.2083 | 0.2145 | 3.4 |
| Frozen BERT w/ class. head | 76.68 | 71.43 | 0.7374 | 0.7606 | 23.7 |
| Naive Bayes | 78.06 | 73.20 | 0.7612 | 0.7757 | **5379.0** |
| Human Benchmark | 82.60 | 82.60 | 0.8088 | 0.8415 | 0.2 |

Table 4.1: Performance comparison of different classification models on 15-class dataset.

## 4.2 Confusion Matrices

Here, the confusion matrices for some of the most interesting classifiers will be provided. A comprehensive view of all matrices, including precision and recall matrices can be found in the Appendix A.

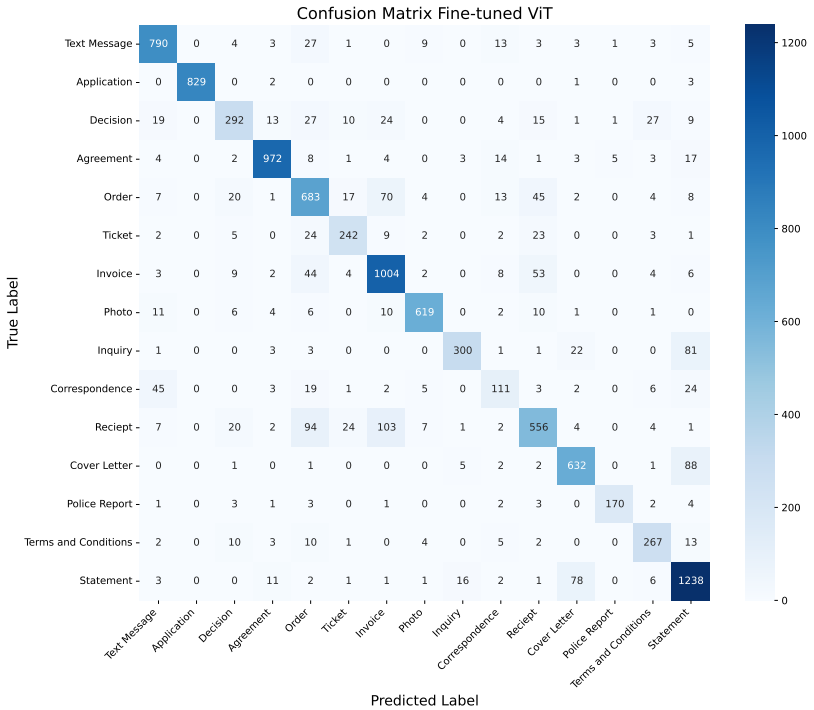



Figure 4.1: ViT confusion matrix

Figure 4.1 shows the confusion matrix for the ViT classifier. A general diagonal can be seen, although some outliers occur. The matrix shows how different classes are confused by

the predictor. For example, the overlap between "statement" and "cover letter" suggests that the classes are often mixed up.

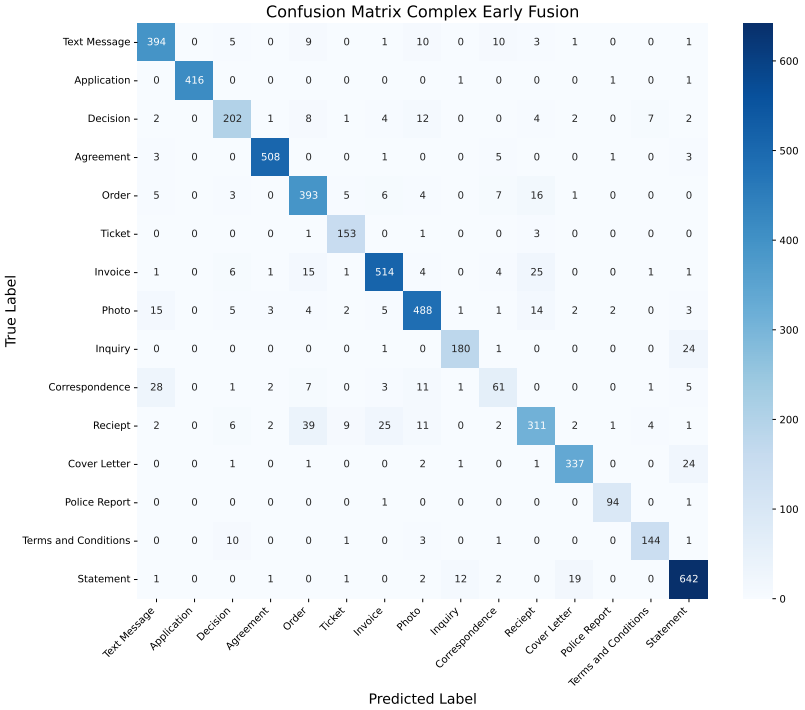



Figure 4.2: Complex early fusion confusion matrix

Figure 4.2 shows the confusion matrix for the complex early fusion model, which, as seen in Table 4.1, is the most accurate classifier tested. While the diagonal trend is more prominent, the matrix reveals how it struggles with certain classes more than others, such as "receipt".

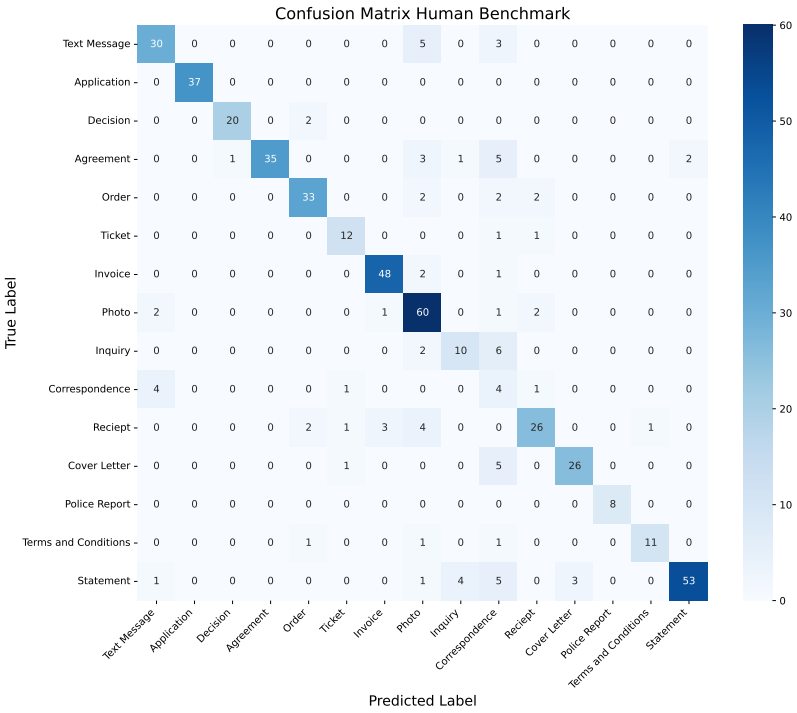



Figure 4.3: Human Benchmark confusion matrix

Figure 4.3 shows the confusion matrix for the human benchmark. While the diagonal trend still exists, the matrix shows how classes such as "correspondence" were more difficult to classify than others.

## 4.3 Per-Class Accuracy

Using the confusion matrices, it is possible to calculate the per-class accuracy. This is presented together with the support, which is total number of instances of that class in the dataset, and the F1 score.

Table 4.2 shows the per-class accuracies and F1 scores for the fine-tuned KB-SBERT model. The support column makes the class imbalance evident, although no clear correlation to accuracy can be seen.

| Per-class Performance Metrics KB-SBERT | | | |
|---|---|---|---|
| **Class** | **Accuracy (%)** | **Support** | **F1 Score** |
| Text Message | 89.36 | 423 | 0.898 |
| Application | 99.52 | 416 | 0.993 |
| Decision | 79.58 | 240 | 0.827 |
| Agreement | 96.95 | 525 | 0.977 |
| Order | 86.21 | 435 | 0.862 |
| Ticket | 91.56 | 154 | 0.895 |
| Invoice | 91.52 | 566 | 0.884 |
| Photo | 83.86 | 409 | 0.835 |
| Inquiry | 93.20 | 206 | 0.943 |
| Correspondence | 69.57 | 115 | 0.672 |
| Receipt | 71.43 | 413 | 0.751 |
| Cover Letter | 93.15 | 365 | 0.910 |
| Police Report | 96.77 | 93 | 0.947 |
| Terms and Conditions | 90.51 | 158 | 0.894 |
| Statement | 93.38 | 680 | 0.941 |

Table 4.2: Per-class Performance Metrics KB-SBERT

| Per-class Performance Metrics ViT | | | |
|---|---|---|---|
| **Class** | **Accuracy (%)** | **Support** | **F1 Score** |
| Text Message | 91.65 | 862 | 0.899 |
| Application | 99.28 | 835 | 0.996 |
| Decision | 66.06 | 442 | 0.717 |
| Agreement | 93.73 | 1037 | 0.945 |
| Order | 78.15 | 874 | 0.748 |
| Ticket | 77.32 | 313 | 0.787 |
| Invoice | 88.15 | 1139 | 0.848 |
| Photo | 92.39 | 670 | 0.936 |
| Inquiry | 72.82 | 412 | 0.814 |
| Correspondence | 50.23 | 221 | 0.552 |
| Receipt | 67.39 | 825 | 0.721 |
| Cover Letter | 86.34 | 732 | 0.853 |
| Police Report | 89.47 | 190 | 0.926 |
| Terms and Conditions | 84.23 | 317 | 0.824 |
| Statement | 91.03 | 1360 | 0.866 |

Table 4.3: Per-class Performance Metrics ViT

Table 4.3 shows the per-class metrics for the ViT. This table can be compared to Table 4.2, showcasing which similarities and differences a vision-based model has compared to a text-based model. For example, the ViT handles the "photo" class better, while the results are worse for more text-heavy classes such as "inquiry".

Table 4.4 shows per-class metrics for the complex early fusion model. The table shows how fusing the modalities can make it more accurate than either of the modalities by themselves for multiple classes, such as "order" or "ticket". However, this does not hold for every class.

| Per-class Performance Metrics Complex early fusion | | | |
|---|---|---|---|
| **Class** | **Accuracy (%)** | **Support** | **F1 Score** |
| Text Message | 90.78 | 434 | 0.890 |
| Application | 99.28 | 419 | 0.996 |
| Decision | 82.45 | 245 | 0.835 |
| Agreement | 97.50 | 521 | 0.978 |
| Order | 89.32 | 440 | 0.857 |
| Ticket | 96.84 | 158 | 0.924 |
| Invoice | 89.70 | 573 | 0.907 |
| Photo | 89.54 | 545 | 0.893 |
| Inquiry | 87.38 | 206 | 0.896 |
| Correspondence | 50.83 | 120 | 0.570 |
| Receipt | 74.94 | 415 | 0.785 |
| Cover Letter | 91.83 | 367 | 0.922 |
| Police Report | 97.92 | 96 | 0.964 |
| Terms and Conditions | 90.00 | 160 | 0.909 |
| Statement | 94.41 | 680 | 0.924 |

Table 4.4: Per-class Performance Metrics Complex early fusion

| Per-class Performance Metrics Human Benchmark | | | |
|---|---|---|---|
| **Class** | **Accuracy (%)** | **Support** | **F1 Score** |
| Text Message | 84.21 | 38 | 0.800 |
| Application | 100.00 | 37 | 1.000 |
| Decision | 77.27 | 22 | 0.930 |
| Agreement | 74.47 | 47 | 0.854 |
| Order | 89.74 | 39 | 0.857 |
| Ticket | 85.71 | 14 | 0.828 |
| Invoice | 92.16 | 51 | 0.932 |
| Photo | 84.85 | 66 | 0.822 |
| Inquiry | 50.00 | 18 | 0.606 |
| Correspondence | 30.00 | 10 | 0.182 |
| Receipt | 62.16 | 37 | 0.754 |
| Cover Letter | 75.00 | 32 | 0.852 |
| Police Report | 100.00 | 8 | 1.000 |
| Terms and Conditions | 50.00 | 14 | 0.846 |
| Statement | 74.63 | 67 | 0.869 |

Table 4.5: Per-class Performance Metrics Human Benchmark

Table 4.5 shows the per-class metrics for the human benchmark. Looking at the support column, it is clear that the dataset for this benchmark was smaller than the other tests. The table shows how human classifiers performed worse when classifying labels such as "correspondence" and "inquiry", while performing perfectly on "application" and "police report".

## 4.4 Model Scores

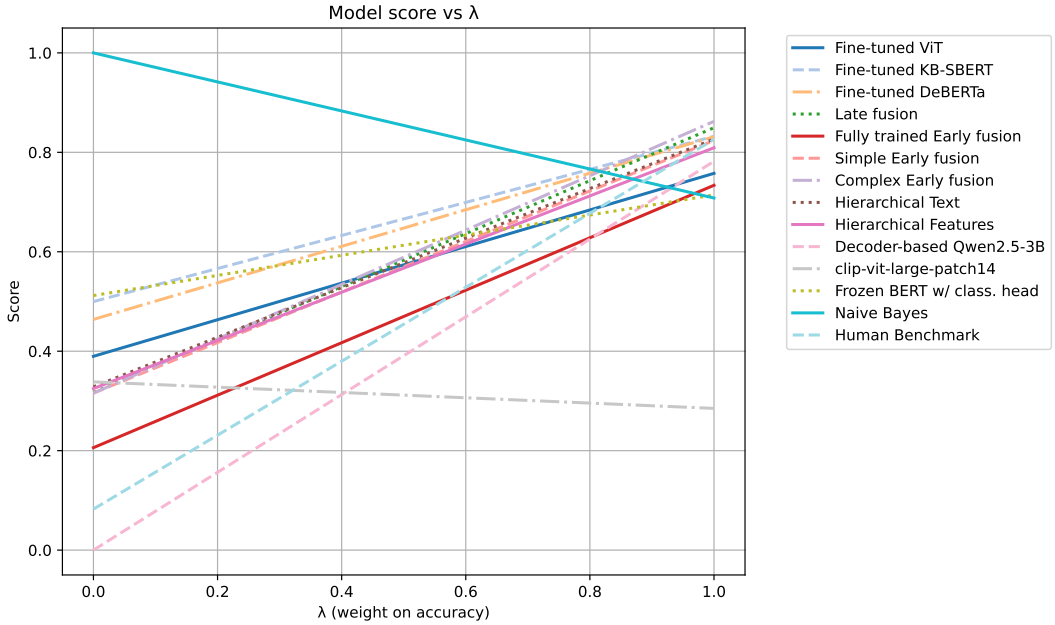



Figure 4.4: Scores for each model as the importance of accuracy against throughput increases.

In Figure 4.4, the model scores, calculated as described in Section 3.2.2, are illustrated for all models tested in this thesis. The plot shows how the Naive Bayes classifier remains the optimal choice when throughput is prioritized. The optimal model then becomes the fine-tuned KB-SBERT model for a short interval, before finally being replaced by the complex early fusion model, which achieved the highest accuracy. While specific values of this metric are not so informative, the comparison between them can still yield insights into the trade-off between accuracy and throughput, which can be useful when considering which solution to implement for specific applications. To better facilitate this, Figure 4.5 illustrates how the models are ranked against each other as the importance of accuracy increases. The fine-tuned KB-SBERT model remains a top-three ranking regardless of the trade-off, showcasing it as a strong choice in situations where on model is needed to cover multiple applications with different tradeoffs. In contrast, methods such as CLIP or the fully trained early fusion classifier fail to reach a top-five position, no matter how accuracy and throughput are prioritized.

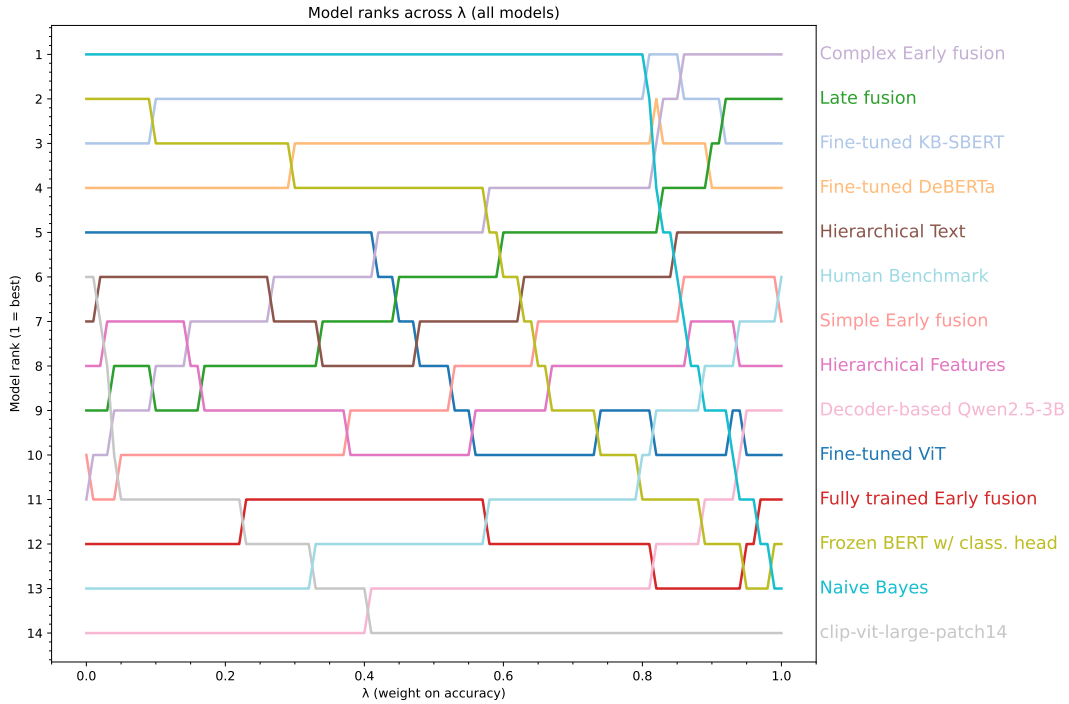



Figure 4.5: Rank of each model as the importance of accuracy against throughput increases.

## 4.5 Training and Fine-Tuning Times

In Table 4.6 the training and fine-tuning times for all non-base models are presented. The size of the video memory (VRAM) is also included since the training was performed on two different computers with different graphics cards, one using a Tesla T4 with 16 GiB of VRAM and the other using an RTX A2000 with 4 GiB of VRAM.

| Model | Epochs | Training size | VRAM (GiB) | Time (hh:mm) |
|---|---|---|---|---|
| Fine-tuned ViT | 5 | 20462 | 16 | 00:38 |
| Fine-tuned BERT | 5 | 20829 | 16 | 00:43 |
| Fully trained early fusion | 5 | 43025 | 4 | 49:30 |
| Complex Early fusion | 5 | 29292 | 4 | 05:29 |
| Fine-tuned Qwen2.5-3B | 5 | 7000 | 16 | 06:16 |

Table 4.6: Training details

The table illustrates how training time varies for models with different complexities. For example, the decoder-based model was trained for over six hours despite using a faster machine and significantly cutting the training size. The fully trained fusion model, which backpropagates the loss throughout the whole architecture as described in Section 3.4.3, was trained for a substantial amount of time.

# 5 Discussion

This chapter discusses the entire thesis, from dataset construction to model testing and results. This discussion should aid in answering the research questions.

## 5.1 Dataset

During this project, it has been assumed that all labels in the dataset have been correctly labeled and that a 100% accuracy is achievable. However, this assumption may not hold true, which means that no matter how much training and fine-tuning is performed, there may be an upper limit to achievable accuracy. This section discusses the dataset by investigating its structure and discussing the results of the human benchmark tests.

### 5.1.1 Dataset Imbalance

In Figure 3.2 the class distribution of the dataset is shown. It is apparent that several classes are underrepresented compared to others, such as "police report" (952) compared to "photo" (7356). This class imbalance could have been avoided from the beginning by, for example, clustering the small classes with some of the larger ones that are similar enough. A way to investigate if the class imbalance has a meaningful effect on the predictor results is to compare the weighted F1 score to the macro average F1 score, since a discrepancy between these metrics may indicate that the predictor favors the overrepresented or underrepresented classes.

The macro and weighted average F1 for the complex early fusion model are 0.8535 and 0.8713 respectively. This indicates that the model slightly favors overrepresented classes like "photo" over less frequent classes like "correspondence". This discrepancy could indicate that the model would perform better if more instances of the smaller classes were present in its training dataset, as this could teach it more robust underlying representations of these classes.

A way to investigate whether better class balance would directly affect the classifier performance was also explored by upsampling the original dataset so that each class had at least 3000 instances in the training set and then retraining the complex early fusion model on this upsampled dataset. Upsampling means duplicating the files under smaller labels in the training dataset and then training the model on this balanced dataset. Unfortunately, this did not result in an increase in accuracy, which is why it was not included in the final results. An issue

with upsampling is that the new added data is only a copy of the old data, which therefore does not teach the model any new information about the dataset.

Another way to investigate how the class imbalance in the dataset affects the classifier results is to calculate the per-class accuracy of the predictors, which can be seen in Table 4.2. These tables show the per-class accuracy for the top two predictors, along with the support which is the total number of instances for a specific class. Here, it can be seen that a low accuracy does not necessarily coincide with low support; for example, "police report" has a high accuracy with only 96 instances, while "photo" has an accuracy of 67% on 545 instances for the complex late fusion model.

Since the difference between weighted and macro F1 scores is small, and training on an upsampled dataset failed to improve accuracy, we can conclude that class imbalance isn't the primary factor limiting performance. This is further supported by the lack of clear correlation between per-class accuracy and support values, suggesting other issues are more significant barriers to achieving higher accuracy.

### 5.1.2 Investigating Class Partition with Human Benchmark

The dataset was also classified by two humans to see how their classifications compare to the models. Here, they achieved an accuracy of 82.6% which compares to a fine-tuned Qwen2.5-3B model with 83.9%. This relatively low score indicates that some instances of the dataset are difficult to classify, even for humans. It is important to note that the individuals who performed this classification were different from those who labeled the original dataset. Had the original labelers conducted this evaluation, they likely would have achieved a higher accuracy.

In the confusion matrix in Figure 4.3, it can be seen that the label "correspondence" was difficult to classify and was often mixed up with the "statement" and "text message" classes. This could be a result of the ambiguous name for the label, which may confuse the human predictors since the definition of correspondence is "communication by letters or email," which is very similar to "text message" and "statement" [41]. This issue could have been resolved by for example including a more detailed annotation guideline with the dataset, but since this was not included the predictors had to resort to the actual label names and contents of the documents when making their predictions.

When looking at the per-class accuracy table for the fine-tuned S-BERT and complex early fusion, we see that the accuracy for correspondence is 48.72% and 55.00% respectively, which both constitute the hardest label to predict. Since BERT doesn't consider the semantic meaning of label names when making predictions (as labels could be named anything), the "correspondence" class is inherently difficult to classify, disregarding the name of the class.

By comparing the per-class accuracy between models and human predictors, one can deduce that apart from "correspondence", the classes "receipt" and "statement" get an accuracy lower than the overall accuracy for that specific model. The low per-class accuracy indicates that these classes are more difficult to classify, regardless of the predictor. This tells us that something needs to be done with these classes to achieve an overall improvement in accuracy. For the "correspondence" class, the easiest alternative is probably to rework it by subdividing it or merging the class with some similar instance that the models often mix them up with (for example, the "text message" class). It is important to note that a suggestion to adjust the classes themselves should not be misunderstood as simply tweaking the labels to maximize accuracy. In practice, a perfect accuracy could be trivially achieved by simply merging all classes, making the process meaningless. Instead, the goal is to use this insight to look for semantically similar classes or classes that perhaps could be divided into subclasses, as the end goal is to achieve informative classifications for a downstream task. A merging of classes only makes sense if doing so doesn't undermine the real-world needs of the agency.

The "decision" class is one of the most frequent classes, but still manages to be hard to predict by the models and human classifiers. Because of this, adding more class instances

would probably not increase its accuracy. The confusion matrices do not reveal any consistent pattern of mix-ups between this class and others, providing few clues about the source of classification difficulty. One way to get a deeper understanding of the issue would be to perform a feature analysis to investigate which features are most important for distinguishing this class from others.

The "receipt" class presents challenges due to its mixed content of receipts, photos and online screenshots, which complicates text-only classification approaches. Additionally, there's notable confusion between this class and both the "invoice" and "order" classes, as evidenced in Figure 4.1. It would therefore be beneficial to further investigate this class and possibly combine it with any of the other two classes.

Finally, the "photo" class is problematic, not due to a low accuracy but because it often gets confused with other classes due to its broad nature. Many of the files under the "photo" class are pictures of random objects, but the class can also include pictures of documents and text messages. To improve accuracy, this class would need to be reworked so that all pictures of objects are separated and images of documents and messages are assigned to their corresponding classes. Apart from that, it is also the most difficult class to parse to text or convert to image as seen in Figure 3.1.

To conclude this section, the human benchmark proved that the dataset partition is not optimal and could be reworked. By investigating the confusion matrices and per-class accuracies, the four classes that seem to be the primary contributors to the reduced accuracy could be identified. By reworking these classes and further analyzing them, their per-class accuracy may improve, and the classifications themselves could prove more useful.

### 5.1.3 Preprocessing

The dataset contains pdf files, which have to be preprocessed in different ways depending on the task. Many of the text models require plain-text inputs, while vision based models expect fixed-size images. As seen in the tests in Section 3.1.4, not all files can be converted successfully.

The text conversion was performed using Optical Character Recognition (OCR) with the Tesseract engine [22]. While effective, this engine is not flawless. It often skips parts of words or introduces unintended characters due to document artifacts. For example, the "photo" class typically lacks any text, but the OCR engine attempts to extract any elements that resemble Latin characters, frequently resulting in mistakes. Additionally, photos commonly include timestamps of upload dates in the corners, which are captured as text. Although similar artifacts appear in other document classes, they often comprise the majority of text extracted from photos. As a result, it is possible that the models learn to associate brief, incoherent text strings containing dates with the "photo" class.

In this project, the entire dataset was preprocessed in advance, extracting all text and using it for training and inference later. Consequently, any noise introduced by the OCR engine became permanent in the dataset. If the OCR step had been part of the training loops, models would have been able to learn which parts of the documents were legacy and which were noise, which might have resulted in higher accuracies.

The image conversion was handled using the PyMuPDF library [3] to convert the first page to png. Other libraries were tested, and the choice was made based on speed and ease of implementation. When passed to a transformer model such as the ViT, the images are randomly cropped down to a size of 224x224 pixels, which means some information will be lost. It is worth noting that while data augmentation is also used for existing document classification models like DocBERT, the augmentations are much more subtle to reflect the fact that documents often come in a more standardized format [2]. Because of this, a ViT was fine-tuned on a dataset that had much smaller augmentations to ensure that the ViT does not get fine-tuned on a training set that does not fit its purpose. However, this model performed worse than the other model and will not be studied further.

## 5.2 Method Discussion

In this section, we look back at the choices made during implementation and consider how different choices may have affected the results.

### 5.2.1 Model Selection

During this project, many models have been loaded and tested. It is, therefore, important to discuss the model selection process and what assumptions could have been changed.

All pre-trained encoder models were selected based on previous research in this field. This approach may explain why several of the encoder models used are relatively old within the rapidly evolving field of AI. For instance, the paper introducing the Vision Transformer was published in 2020 [16]. If a more thorough deep dive into the HuggingFace library had been conducted, some more recent visual encoder models may have been discovered, which could improve the results.

While encoder models were subjected to the most thorough testing and modification in this work, the current trend among researchers is heavily focused on investigating decoder models. Although some effort was dedicated to researching and fine-tuning decoder models such as Qwen 2.5, their complete potential may not have been fully explored or realized. This was also partly motivated by the fact that decoder-based models often are much more hardware-intensive, both for fine-tuning and running inference. The process of fine-tuning a decoder, even when utilizing PEFT methods, is much more tedious then fine-tuning an encoder model, making testing it difficult.

### 5.2.2 Fine-Tuning Process

The fine-tuning processes for BERT, ViT, and Qwen followed the standard pipeline using the transformers Trainer class. For BERT, we utilized the hyperparameters recommended in the original paper [15]. The Vision Transformer (ViT) fine-tuning similarly adopted hyperparameters from its original implementation [16]. For Qwen 2.5, we employed parameter-efficient fine-tuning (PEFT) with LoRA adapters due to the model's larger size, which allowed for effective training while minimizing computational requirements. Across all models, limited time was allocated to hyperparameter optimization, as our focus was on architecture comparison rather than individual model performance maximization. More extensive hyperparameter tuning, especially for the more recently developed Qwen model, would likely have improved not only the individual model performances but also enhanced the fusion models that incorporated these base architectures. The decision to use default hyperparameters represents a practical compromise between resource constraints and performance optimization.

### 5.2.3 Fusion Model Construction

During the construction and testing of the fusion model, some decisions were made that may have affected the final results. For all fusion models except the fully trained one, the same set of CLS tokens was used throughout the entire training process. This was done since the task of extracting CLS-token was both time and resource-consuming. By extracting the CLS-tokens once and then training the fusion architecture on the same set of tokens for each iteration, this approach may risk teaching the model noise that is unique to that specific set of tokens. A better but more time-consuming option would have been to include the extraction process when fetching the dataset during training, so that each new epoch is presented with new CLS-tokens.

Another issue is that by freezing the BERT and ViT classifier weights during back-propagation, only the fusion module learns to merge information from the modalities, while each separate model has no idea that their features are going to be merged with some other representation. This means that the separate features are not being optimized for being

merged. This issue was solved by including both base classifiers in the backpropagation process but did unfortunately increase the training time substantially since the BERT and ViT models had to be trained while also extracting their CLS-tokens at the same time. Additionally, the training time increased significantly because this model utilized 80% of the dataset for training, compared to only 40% used when training the other early fusion models. If more time or better resources had been available, this alternative could have been more thoroughly investigated. However, as can be seen in Table 4.6, the training time was too long. Analysis of this table might raise questions about the decision not to utilize the machine with the better graphics card. This choice was necessitated by the disk space requirements of the fully trained fusion model, which required three copies of the dataset, exceeding the total available storage capacity of the faster machine.

### 5.2.4 Replicability

When conducting research as extensive as this thesis, it is essential to carefully document all decisions to ensure that the results can be fully replicated by other researchers. It is important to note, however, that some parts of our process cannot be replicated, specifically the fine-tuning of pre-trained models and the training of the fusion model. This is because these steps were performed using a dataset that is not publicly available.

Beyond this, all models and fine-tuning approaches are documented in the method chapter. For similar classification tasks with comparable datasets, the models described in this thesis could be fine-tuned following the same methodology. As hyperparameter optimization was beyond the scope of this work, each model was fine-tuned using only a single set of hyperparameters, which are provided in Appendix B.

The early fusion models can be reconstructed by following the methodology described under the complex early fusion section in the method chapter. The parameters used for the transformer blocks can also be found in the Appendix B.

### 5.2.5 Literature Review

To ensure that the literary foundation of this project is credible, as many peer-reviewed papers as possible were used. Out of the 60 references, 16 are not peer-reviewed. These 16 sources can be divided into four groups.

#### Technical Reports

These four references (GPT-4 Technical Report [1], GELU activation function note [28], Qwen 2.5 Technical Report [69], Generative Pre-Training description [48]) are technical publications released by research teams. While never peer-reviewed, these sources are heavily cited and provide the original details of the architectures they describe. We include them to ground our work in the precise implementation details used by the original authors of SOTA models.

#### Libraries and Documentation

The PyMuPDF library manual [3], Google Cloud Pricing Calculator [21], Tesseract OCR repository [22], and Hugging Face website [32] are official project documentation and tools used in this project. They are included here to ensure replicability, as they describe the tools used in this project.

#### Non-Academic Publications

Non-academic publications are research projects published outside the academic setting. Since these projects are not performed in connection to a university they do not necessarily follow the same rigorous peer-review procedure as in academia. Examples in this thesis

include: [46], a research article on the energy usage of AI commissioned by the company Pilz; [59], an investigation of the administrative costs for municipalities; and [63] investigates the share of different languages used on the internet.

[33] directly cites a company's website and [41] cites a dictionary for a specific word.

#### Developer Articles

Philip Gage's algorithm description [19], and the blog posts created by KBLab describing how the Swedish BERT variants were trained [53, 52], are articles posted without a peer-review process. These sources allow developers to describe models and algorithms without writing a formal report.

## 5.3 Results

This section examines the results from all predictors, providing context for the findings while highlighting the strengths and weaknesses of each classification model.

### 5.3.1 Model Complexity Trade Off

One of the research questions for this project was to investigate the trade-off between model complexity and performance. The complexity of a model is documented by measuring inference time and training/fine-tuning duration. Naturally, the times themselves are arbitrary since different machines have different capabilities. However, since Ida Infront provided the resources used during this project, the measurements can still offer valuable insight into what the training times would likely be if they choose to follow our implementation. Furthermore, since all tests were conducted on the same machine, the difference in time can be useful for comparison. To aid this comparison, each model has been ranked for different tradeoffs between accuracy and complexity. As seen in Figure 4.5, the optimal models to use changes drastically depending on the situation. When implementing a classification solution, one must address the importance of classification accuracy and inference time before reviewing which solution to consider.

#### Fusion Modules

The fusion models proved to be the most accurate for this classification task, as can be seen in Table 4.1. However, the overhead required to make fusion models work might be too high, as it includes fine-tuning two separate pre-trained models, creating a custom dataset with CLS tokens, and developing and training a novel fusion architecture. This point is further emphasized when considering the model accuracy from the fine-tuned KB-SBERT model, whose performance is very similar to the complex early fusion model. The process of fine-tuning a pre-trained encoder model is much simpler and requires far less theoretical knowledge of neural network architecture than creating a custom network.

The fully trained fusion model, which could potentially address issues caused by using two frozen fine-tuned classifiers, proved too computationally demanding for the available hardware in this project. With better hardware or additional time allocation, this approach might have demonstrated greater promise. Despite its greater complexity, this model offers substantial implementation advantages for new developers, as the entire training script operates automatically when provided with a labeled dataset. This contrasts with the complex early fusion model, which requires fine-tuning two separate classifiers and then extracting their CLS-tokens on 40% of the dataset, a factor that must be considered when evaluating overall model complexity.

The hierarchical models, while promising, could not compete with the unimodal KB-SBERT model. This is likely because the architecture of the hierarchical models didn't allow

for predictions to be made on data where text could not be retrieved. While this was never an option in the case of the concatenated text version, the feature-engineered hierarchical architecture could benefit from a revision, allowing the ViT's initial prediction to lead to a final prediction even when the text model fails. The text concatenation model could also improve from different prompting strategies. For example, instead of simply prepending the raw output of the ViT, the BERT model could be given context about what the ViT is guessing.

#### Decoder-Based Classifiers

In addition to the encoder-based experiments, this study implemented a decoder-based classification model using the Qwen2.5-3B backbone [69]. As shown in Table 4.1, the model achieved an accuracy of 83.90%, placing it between the frozen and the fully fine-tuned KB-SBERT. This can be seen as a proof of concept, as different architectures and fine-tuning strategies might yield better result. While the 3B parameters make this model the largest one tested in this study, it is still relatively small among decoder models [1]. Larger models such as the 32B or 72B Qwen variants could potentially leverage more knowledge and achieve higher accuracies. However, inference throughput was only 0.08 files/s, roughly two orders of magnitude slower than the encoder-based models. As seen in Table 4.6, training time was also significant, and hardware requirements were greater than any other tested model. The significant training time and hardware requirements limited this model's training to only a subset of the available data. Training on more data is likely to yield better results. Although encoder-based models outperformed the decoder model, this experiment highlights the potential of decoder architectures and a need for further study of larger models or alternative classification strategies. Despite respectable accuracy, the computational overhead of decoder-based classifiers limits their practicality for high-volume government document processing.

### 5.3.2 Fine-Tune Results vs. Base Results

In this project, one of the main challenges was to incorporate domain-specific Swedish terminology into the classification models. This was done through fine-tuning. As seen in Table 4.1, fine-tuning led to a significant increase in accuracy. The fine-tuned KB-SBERT model achieved a high accuracy of 89.34%. In contrast, the same KB-SBERT model without fine-tuning with a trained classification head on top achieved an accuracy of 76.68%. In this case, the encoder had not learned any domain-specific knowledge and only output general-purpose embeddings, which then had to be separated into the different classes by the 2-Layer MLP head. To ensure that the fault wasn't at the classification head, multiple different classification heads were tested, such as Gaussian Naive Bayes, SVM, and MLPs with a different number of layers. Of all the configurations, the 2-layer MLP performed the best. The significant impact on accuracy from only 43 minutes of fine-tuning, as seen in Table 4.6, shows that, an encoder model such as KB-SBERT is adaptable to specific domains. If the domain is in another low-resource language, KB-SBERT could be substituted with a multilingual model such as DeBERTa, which showed competitive results. This extends to all fusion architectures as well since they utilize KB-SBERT. If sufficient hardware is unavailable for fine-tuning, one way to show the model the domain-specific terminology without resource-intensive training is to use a classical machine learning algorithm, such as Naive Bayes. As described in Section 2.4.1, this classifier bases its decisions on the relative frequency of words in each class. This makes it especially capable in domains with highly standardized documents. For example, most documents of the class "application" follow a well-defined structure with specific phrases that appear in nearly every case. This allows the model to classify the document without actually considering any of the words together. As a result, the classifier can perform well without understanding the meaning of the text or considering how words relate to one another.

### 5.3.3 Fusion Results

As demonstrated in Table 4.1, the complex early fusion performed the best in terms of both model accuracy and dataset accuracy. We can get some deeper insights into why this is the case by looking at the per-class accuracy for this model in Table 4.4 and comparing it with the per-class accuracy for the other classifiers in Table 4.2, 4.5, and 4.3. It is evident that all models struggle with the same two document categories, while consistently performing well on the same two easiest classes. Complex fusion still struggles with "correspondence" and "recipe", indicating that these classes have something inherently wrong with them, as discussed in the dataset discussion. Complex fusion shows a substantial increase for the "ticket" class compared to only using the fine-tuned KB-SBERT, with an increase in accuracy of almost 8 percentage points. The "ticket" class contains both pictures of physical tickets and screenshots of digital tickets, so the usage of both the visual and textual aspects for this specific class may contribute to the increase in accuracy. The same goes for the "photo" class, with an increase in accuracy of about 5 percentage points, which better aligns with the ViT accuracy for that class. When comparing it to the ViT we can see a notable increase in "decision", "police report", and "terms and conditions", all of these being easier to differentiate by reading the text. The behavior of combining the best attributes from two classifiers that are looking at different modalities was exactly the reason why this architecture was constructed, and it seems to work when using complex early fusion. This model proves the capacity to combine the best traits from each separate classifier. This is even more apparent when looking at the "dataset accuracy" in Table 4.1, where complex early fusion strongly outperforms the fine-tuned encoder models, which is reasonable since complex early fusion can use both extracted text and extracted images while the encoder models only rely on one of them.

To investigate if the complex fusion technique is too complicated for the task, a simple early fusion model was also constructed where the fusion step only has one trainable parameter, dictating how much weight to put on each modality. In table 4.1 we can see that this model is worse than both the complex fusion model and also the late fusion model, which implies that complex fusion manages to balance the modality representations in a way that is not possible through weighted fusion and also that weighted fusion is worse at selecting a modality than just using KB-SBERT and falling back on ViT.

Finally, we can investigate the corresponding F1 scores for the fusion models. Here, we notice that the imbalance in classifying minority classes is almost the same for all model architectures since the difference between macro average and weighted average is similar. The results indicate that while the classifiers show a slight preference for majority classes over minority classes, this bias is not significant enough to necessitate re-balancing the dataset. This is especially evident when compared to the Human Benchmark, where the imbalance is even greater, indicating that these minority classes may be more difficult to classify.

## 5.4 The Work in a Wider Context

This thesis on different classification techniques for Swedish government documents sits at the intersection of several important research domains in artificial intelligence and NLP. While our experiments focused on Swedish government documents specifically, the approaches developed here have implications that extend beyond this particular application. The challenges encountered and solutions proposed touch upon broader issues in NLP for low-resource languages and multi-modal representation learning.

The performance of the fine-tuned KB-SBERT demonstrates that even for low-resource languages with limited text corpora and fewer pre-trained models, strong classification results can be achieved when working with an adequately sized labeled dataset in the target language. What this implies is that the construction of a novel pre-trained encoder model is not necessary since the existing ones are sufficient for domain specific adaptation. This finding has significant implications for global NLP applications, suggesting that organiza-

tions working with low-resource languages can leverage existing models through fine-tuning rather than investing in costly training of language-specific models from scratch.

The results from the fusion models demonstrate that while implementing a multi-modal fusion architecture requires additional computational overhead, it can lead to improvements in predictive accuracy when the dataset contains information distributed across different modalities. This finding has broader implications for document processing systems beyond government agencies, particularly in fields such as healthcare, legal services, and financial institutions where documents often contain both textual and visual elements that carry complementary information. The approach developed in this thesis could be adapted to enhance classification accuracy in these domains, potentially improving workflow efficiency and information retrieval across multiple industries.

# 6 Conclusion

In this section we answer the research questions and also discuss what future work can be conducted as an extension of our findings.

**How can domain-specific terminology and contextual knowledge be incorporated into PLMs to improve classification accuracy for Swedish government documents?**

This study demonstrated that models achieved higher classification accuracy when trained on specific domains compared to using generic pre-trained embeddings. This indicates that the domain-specific terminology and contextual knowledge from the dataset could be captured by the models during fine-tuning. For the Swedish domain, this study concludes that the KB-SBERT model performs well as a base model. For similar types of data in other low-resource languages, DeBERTa is a strong multilingual contender. To further improve accuracy, utilizing fine-tuned decoder-based models shows significant potential. Although resource-intensive, larger models are likely to be competitive to the encoder-based solutions. Classification accuracy can also benefit from including contextual information in the visual modality, with a fusion model achieving the highest accuracy in the study, seen in Table 4.1.

**What strategies can balance model accuracy and computational efficiency for deploying LLMs locally in Swedish government agencies?**

When considering deploying automated solutions locally, computational efficiency and ease of implementation is essential. This study used throughput during inference to represent computational efficiency. The best performance was achieved by the Naive Bayes classifier, which was two orders of magnitude faster than any transformer-based model. The encoder-based models generally perform one order of magnitude better than fusion architectures, and two orders of magnitude faster than the decoder-based model. An analysis of the trade-off between accuracy and throughput determined that the optimal models are the complex early fusion model, followed by the fine-tuned KB-SBERT model, followed by the Naive Bayes model, in order of accuracy importance relative to throughput.

**How does a multi-modal approach combining visual and textual elements enhance document representation compared to uni-modal approaches relying solely on either visual or textual information?**

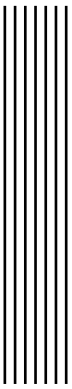
The multi-modal fusion architecture achieved the best results out of all the tested classifiers. By analyzing the per-class accuracy for the different classifiers, it is evident that complex fusion manages to utilize the best properties from both modalities, which contributes to its increased accuracy. The fusion architecture displays better model accuracy than all predictors and also superior dataset accuracy, indicating that the complex early fusion model architecture is the best fit for the dataset. The high accuracy did however come at a significant overhead cost for creating and using the model, requiring fine-tuning of two separate models, extraction of CLS tokens, and development of a custom fusion architecture that demands greater computational resources during both training and inference.

## 6.1 Future Work

To further analyze the performance of the proposed fusion architectures, comparing the results in this report to tests on other datasets would be insightful. This would reveal whether architectural decisions matter over specific domains, or if trends are consistent among broader types of data.

Another area that would benefit from further study is decoder-based classifiers. This study included a decoder-based classifier as a baseline for comparison with encoder-based models. The results revealed their potential as classifiers in low-resource language, single-label multiclass setting. Future work could examine how model size impacts classification accuracy and throughput in decoder-based transformer models. Additional fine-tuning methods and decoder constraints would also be insightful for further study.

Finally, fully-trained early fusion architecture shows promise but would need much more extensive testing, requiring more computational resources to achieve similar results to complex fusion. In theory, the fully trained architecture would need less manual intervention, as it would simultaneously train the BERT weights, ViT weights, and fusion weights during back-propagation, with both pre-trained models being fine-tuned specifically for the purpose of representation fusion. While this architecture would demand more computational resources during training, it could also offer streamlined efficiency during inference and potentially enable a more sophisticated multi-modal understanding.



# Bibliography


[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. "Gpt-4 technical report". In: *arXiv preprint arXiv:2303.08774* (2023).

[2] Ashutosh Adhikari, Achyudh Ram, Raphael Tang, William L. Hamilton, and Jimmy Lin. "Exploring the Limits of Simple Learners in Knowledge Distillation for Document Classification with DocBERT". In: *Proceedings of the 5th Workshop on Representation Learning for NLP*. Ed. by Spandana Gella, Johannes Welbl, Marek Rei, Fabio Petroni, Patrick Lewis, Emma Strubell, Minjoon Seo, and Hannaneh Hajishirzi. Online: Association for Computational Linguistics, July 2020, pp. 72–77. DOI: 10.18653/v1/2020.repl4nlp-1.10. URL: https://aclanthology.org/2020.repl4nlp-1.10/.

[3] Artifex. *PyMuPDF Library*. https://github.com/pymupdf/PyMuPDF. Accessed: 2025-03-13. 2025.

[4] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *3rd International Conference on Learning Representations (ICLR)*. 2016. arXiv: 1409.0473 [cs.CL]. URL: https://arxiv.org/abs/1409.0473.

[5] Souhail Bakkali, Zuheng Ming, Mickael Coustaty, and Marcal Rusinol. "Visual and Textual Deep Feature Fusion for Document Image Classification". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*. June 2020.

[6] Thomas Bayes. "LII. An essay towards solving a problem in the doctrine of chances. By the late Rev. Mr. Bayes, FRS communicated by Mr. Price, in a letter to John Canton, AMFR S". In: *Philosophical transactions of the Royal Society of London* 53 (1763), pp. 370–418.

[7] Iman Ben Zaken and Ido Dagan. "BitFit: Simple Parameter-efficient Fine-tuning for Transformers". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021.

[8] Alberto Benayas, Miguel Angel Sicilia, and Marçal Mora-Cantallops. "A comparative analysis of encoder only and decoder only models in intent classification and sentiment analysis: Navigating the trade-offs in model size and performance". In: *Language Resources and Evaluation* (2024), pp. 1–24.

[9] Gerlof Bouma. "Normalized (pointwise) mutual information in collocation extraction". In: *Proceedings of GSCL* 30 (2009), pp. 31–40.

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[11] Kenneth Ward Church, Zeyu Chen, and Yanjun Ma. "Emerging trends: A gentle introduction to fine-tuning". In: *Natural Language Engineering* 27.6 (2021), pp. 763–778.

[12] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=r1xMH1BtvB.

[13] Benjamin Clavié, Alexandru Ciceu, Frederick Naylor, Guillaume Soulié, and Thomas Brightwell. "Large language models in the workplace: A case study on prompt engineering for job type classification". In: *International conference on applications of natural language to information systems*. Springer. 2023, pp. 3–17.

[14] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., 2023, pp. 10088–10115. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/1feb87871436031bdc0f2beaa62a049b-Paper-Conference.pdf.

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423.

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations*. 2021. URL: https://openreview.net/forum?id=YicbFdNTTy.

[17] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. *ViT-base-patch16-224*. https://huggingface.co/google/vit-base-patch16-224. Accessed: July 11, 2025. 2021.

[18] Konrad Gadzicki, Razieh Khamsehashari, and Christoph Zetzsche. "Early vs Late Fusion in Multimodal Convolutional Neural Networks". In: *2020 IEEE 23rd International Conference on Information Fusion (FUSION)*. 2020, pp. 1–6. DOI: 10.23919/FUSION45008.2020.9190246.

[19] Philip Gage. "A new algorithm for data compression". In: *C Users J.* 12.2 (Feb. 1994), pp. 23–38. ISSN: 0898-9788.

[20] Ankita Gandhi, Kinjal Adhvaryu, Soujanya Poria, Erik Cambria, and Amir Hussain. "Multimodal sentiment analysis: A systematic review of history, datasets, multimodal fusion methods, applications, challenges and future directions". In: *Information Fusion* 91 (2023), pp. 424–444.

[21] Google. *Google Cloud Pricing Calculator*. Accessed: 2025-03-11. n.d. URL: https://cloud.google.com/products/calculator.

[22] Google. *Tesseract OCR Engine*. Accessed: 2025-03-13. 2024. URL: https://github.com/tesseract-ocr/tesseract.

[23] Cyril Goutte and Eric Gaussier. "A probabilistic interpretation of precision, recall and F-score, with implication for evaluation". In: *European conference on information retrieval*. Springer. 2005, pp. 345–359.

[24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[25] Pengcheng He, Jianfeng Gao, and Weizhu Chen. "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=sE7-XhLxHA.

[26] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. "Deberta: Decoding-Enhanced Bert With Disentangled Attention". In: *International Conference on Learning Representations*. 2021. URL: https://openreview.net/forum?id=XPZIaotutsD.

[27] Pengcheng He, Xiaodong Liu, Jianfeng Wang, and Jialong Gao. *DeBERTa-v3-base*. https://huggingface.co/microsoft/deberta-v3-base. Accessed: July 11, 2025. 2021.

[28] Dan Hendrycks and Kevin Gimpel. *Gaussian Error Linear Units (GELUs)*. 2023. arXiv: 1606.08415 [cs.LG]. URL: https://arxiv.org/abs/1606.08415.

[29] Daniel Holmer and Arne Jönsson. "Comparing the Performance of Various Swedish BERT Models for Classification". In: *Eighth Swedish Language Technology Conference (SLTC2020)*. Organized by University of Gothenburg, Sweden. University of Gothenburg. 2020.

[30] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. "Parameter-Efficient Transfer Learning for NLP". In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, Sept. 2019, pp. 2790–2799. URL: https://proceedings.mlr.press/v97/houlsby19a.html.

[31] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9.

[32] Hugging Face. *Hugging Face*. Accessed: 2025-05-02. 2024. URL: https://huggingface.co/.

[33] Ida Infront. *Ida Infront - Digitalization with security and efficiency*. Accessed: 2025-04-03. 2025. URL: https://idainfront.se/.

[34] Sanjay Kariyappa, Freddy Lecue, Saumitra Mishra, Christopher Pond, Daniele Magazzeni, and Manuela Veloso. "Progressive Inference: Explaining Decoder-Only Sequence Classification Models Using Intermediate Predictions". In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, 21–27 Jul 2024, pp. 23238–23255. URL: https://proceedings.mlr.press/v235/kariyappa24a.html.

[35] KBLab. *sentence-bert-swedish-cased*. https://huggingface.co/KBLab/sentence-bert-swedish-cased. Accessed: July 11, 2025. 2021.

[36] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).

[37] Liunian Harold Li, Mark Yatskar, Da Yin, Cho-Jui Hsieh, and Kai-Wei Chang. "What Does BERT with Vision Look At?" In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault. Online: Association for Computational Linguistics, July 2020, pp. 5265–5275. DOI: 10.18653/v1/2020.acl-main.469. URL: https://aclanthology.org/2020.acl-main.469/.

[38] Yanzeng Li, Bowen Yu, Xue Mengge, and Tingwen Liu. "Enhancing Pre-trained Chinese Character Representation with Word-aligned Attention". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault. Online: Association for Computational Linguistics, July 2020, pp. 3442–3448. DOI: 10.18653/v1/2020.acl-main.315. URL: https://aclanthology.org/2020.acl-main.315/.

[39] Jiasen Lu, Dhruv Batra, Devi Parikh, and Stefan Lee. "ViLBERT: pretraining task-agnostic visiolinguistic representations for vision-and-language tasks". In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[40] Sakshi Mahendru and Tejul Pandit. "SecureNet: A Comparative Study of DeBERTa and Large Language Models for Phishing Detection". In: *2024 IEEE 7th International Conference on Big Data and Artificial Intelligence (BDAI)*. 2024, pp. 160–169. DOI: 10.1109/BDAI62182.2024.10692765.

[41] Merriam-Webster. *Correspondence*. Accessed: 2025-04-22. n.d. URL: https://www.merriam-webster.com/dictionary/correspondence.

[42] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. "Distributed Representations of Words and Phrases and their Compositionality". In: *Advances in Neural Information Processing Systems*. Ed. by C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger. Vol. 26. Curran Associates, Inc., 2013. URL: https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf.

[43] David Navon. "Forest before trees: The precedence of global features in visual perception". In: *Cognitive Psychology* 9.3 (1977), pp. 353–383. ISSN: 0010-0285. DOI: https://doi.org/10.1016/0010-0285(77)90012-3. URL: https://www.sciencedirect.com/science/article/pii/0010028577900123.

[44] Thanh Nguyen, Campbell Wilson, and Janis Dalins. "Fine-Tuning Llama 2 Large Language Models for Detecting Online Sexual Predatory Chats and Abusive Texts". In: *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning – ESANN 2024*. Jan. 2024, pp. 613–618. DOI: 10.14428/esann/2024.ES2024-222.

[45] Alejandro Peña, Aythami Morales, Julian Fierrez, Ignacio Serna, Javier Ortega-Garcia, Íñigo Puente, Jorge Cordova, and Gonzalo Cordova. "Leveraging Large Language Models for Topic Classification in the Domain of Public Affairs". In: *International Conference on Document Analysis and Recognition*. Springer. 2023, pp. 20–33.

[46] Konstantin F. Pilz, Yusuf Mahmood, and Lennart Heim. *AI's Power Requirements Under Exponential Growth: Extrapolating AI Data Center Power Demand and Assessing Its Potential Impact on U.S. Competitiveness*. Research Report RRA3572-1. RAND Corporation, 2025. URL: https://www.rand.org/pubs/research_reports/RRA3572-1.html.

[47] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. "Learning transferable visual models from natural language supervision". In: *International conference on machine learning*. PmLR. 2021, pp. 8748–8763.

[48] Alec Radford and Karthik Narasimhan. "Improving Language Understanding by Generative Pre-Training". In: 2018. URL: https://api.semanticscholar.org/CorpusID:49313245.

[49] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. "SQuAD: 100,000+ Questions for Machine Comprehension of Text". In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Ed. by Jian Su, Kevin Duh, and Xavier Carreras. Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 2383–2392. DOI: 10.18653/v1/D16-1264. URL: https://aclanthology.org/D16-1264/.

[50] Nils Reimers and Iryna Gurevych. "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Ed. by Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu. Online: Association for Computational Linguistics, Nov. 2020, pp. 4512–4525. DOI: 10.18653/v1/2020.emnlp-main.365. URL: https://aclanthology.org/2020.emnlp-main.365/.

[51] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.

[52] Faton Rekathati. *Introducing a Swedish Sentence Transformer*. Aug. 23, 2021. URL: https://kb-labb.github.io/posts/2021-08-23-a-swedish-sentence-transformer/.

[53] Faton Rekathati. *Swedish Sentence Transformer 2.0*. Jan. 16, 2023. URL: https://kb-labb.github.io/posts/2023-01-16-sentence-transformer-20/.

[54] Manon Reusens, Alexander Stevens, Jonathan Tonglet, Johannes De Smedt, Wouter Verbeke, Seppe vanden Broucke, and Bart Baesens. "Evaluating text classification: A benchmark study". In: *Expert Systems with Applications* 254 (2024), p. 124302. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2024.124302. URL: https://www.sciencedirect.com/science/article/pii/S0957417424011680.

[55] Sennrich Rico, Haddow Barry, and Birch Alexandra. "Neural Machine Translation of Rare Words with Subword Units". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2016), p. 1715. DOI: 10.18653/v1/p16-1162. URL: https://cir.nii.ac.jp/crid/1363670320499938816.

[56] Timo Schick and Hinrich Schütze. "It's Not Just Size That Matters: Small Language Models Are Also Few-Shot Learners". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou. Online: Association for Computational Linguistics, June 2021, pp. 2339–2352. DOI: 10.18653/v1/2021.naacl-main.185. URL: https://aclanthology.org/2021.naacl-main.185/.

[57] Mike Schuster and Kaisuke Nakajima. "Japanese and Korean voice search". In: *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2012, pp. 5149–5152. DOI: 10.1109/ICASSP.2012.6289079.

[58] Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. "Compute Trends Across Three Eras of Machine Learning". In: *2022 International Joint Conference on Neural Networks (IJCNN)*. 2022, pp. 1–8. DOI: 10.1109/IJCNN55064.2022.9891914.

[59] Statskontoret. *Administrativa kostnader i kommunsektorn: En analys av statens styrning av kommuner och regioner*. 2022:9. Published May 31, 2022. Statskontoret, May 2022. URL: https://www.statskontoret.se/siteassets/rapporter-pdf/2022/2022_9---utskriftsversion.pdf.

[60] Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. "Revisiting unreasonable effectiveness of data in deep learning era". In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 843–852.

[61] Kamal Taha, Paul D. Yoo, Chan Yeun, Dirar Homouz, and Aya Taha. "A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights". In: *Computer Science Review* 54 (2024), p. 100664. ISSN: 1574-0137. DOI: https://doi.org/10.1016/j.cosrev.2024.100664. URL: https://www.sciencedirect.com/science/article/pii/S1574013724000480.

[62] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010. ISBN: 9781510860964. DOI: 10.48550/arXiv.1706.03762.

[63] W3Techs. *Usage Statistics and Market Share of Content Languages for Websites*. Accessed: 2024-11-29. 2024. URL: https://w3techs.com/technologies/overview/content_language.

[64] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=rJ4km2R5t7.

[65] Karl Weiss, Taghi M Khoshgoftaar, and DingDing Wang. "A survey of transfer learning". In: *Journal of Big data* 3 (2016), pp. 1–40.

[66] Dawid Wisniewski, Antoni Solarski, and Artur Nowakowski. "Exploring the Feasibility of Multilingual Grammatical Error Correction with a Single LLM up to 9B parameters: A Comparative Study of 17 Models". In: *arXiv preprint arXiv:2505.06004* (2025).

[67] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation". In: *Transactions of the Association for Computational Linguistics* 4 (2016), pp. 339–351. DOI: 10.1162/tacl_a_00065.

[68] Ao Xiang, Zongqing Qi, Han Wang, Qin Yang, and Danqing Ma. "A multimodal fusion network for student emotion recognition based on transformer and tensor product". In: *2024 IEEE 2nd International Conference on Sensors, Electronics and Computer Engineering (ICSECE)*. IEEE. 2024, pp. 1–4.

[69] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. *Qwen2.5 Technical Report*. 2025. arXiv: 2412.15115 [cs.CL]. URL: https://arxiv.org/abs/2412.15115.

[70] Chao Zhang, Zichao Yang, Xiaodong He, and Li Deng. "Multimodal intelligence: Representation learning, information fusion, and applications". In: *IEEE Journal of Selected Topics in Signal Processing* 14.3 (2020), pp. 478–493.

[71] Haode Zhang, Yuwei Zhang, Li-Ming Zhan, Jiaxin Chen, Guangyuan Shi, Albert Y.S. Lam, and Xiao-Ming Wu. "Effectiveness of Pre-training for Few-shot Intent Classification". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. Ed. by Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih. Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 1114–1120. DOI: 10.18653/v1/2021.findings-emnlp.96. URL: https://aclanthology.org/2021.findings-emnlp.96/.

[72] Rengui Zhang, Jin Wang, and Xuejie Zhang. "YNU-HPCC at SemEval-2024 Task 2: Applying DeBERTa-v3-large to Safe Biomedical Natural Language Inference for Clinical Trials". In: *Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024)*. Ed. by Atul Kr. Ojha, A. Seza Doğruöz, Harish Tayyar Madabushi, Giovanni Da San Martino, Sara Rosenthal, and Aiala Rosá. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 785–791. DOI: 10.18653/v1/2024.semeval-1.112. URL: https://aclanthology.org/2024.semeval-1.112/.

[73] Shenglin Zhang, Pengtian Zhu, Minghua Ma, Jiagang Wang, Yongqian Sun, Dongwen Li, Jingyu Wang, Qianying Guo, Xiaolei Hua, Lin Zhu, and Dan Pei. " Enhanced Fine-Tuning of Lightweight Domain-Specific QA Model Based on Large Language Models ". In: *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. Los Alamitos, CA, USA: IEEE Computer Society, Oct. 2024, pp. 61–66. DOI: 10.1109/ISSREW63542.2024.00048. URL: https://doi.ieeecomputersociety.org/10.1109/ISSREW63542.2024.00048.

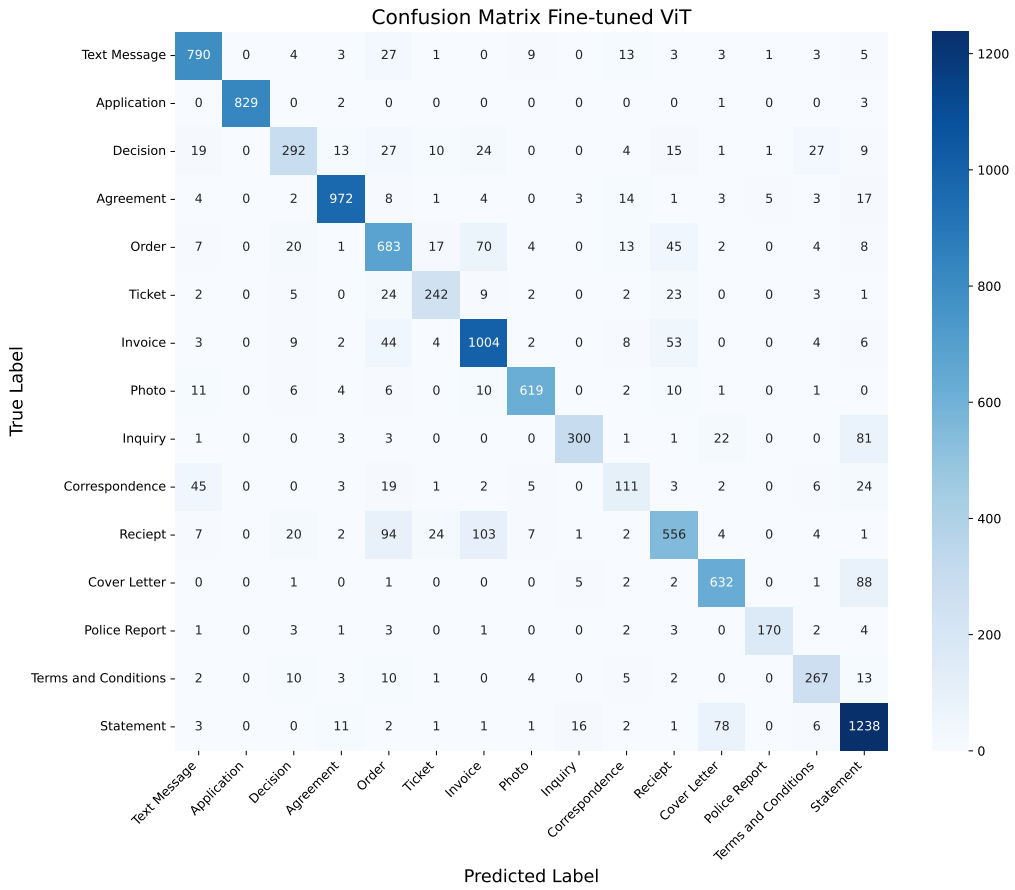


# A Appendix A - Confusion matrices

This appendix contains confusion matrices of all models tested in this study. It also presents precision, recall, and F1-score for each class.

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.883 | 0.916 | 0.899 |
| Application | 1.000 | 0.993 | 0.996 |
| Decision | 0.785 | 0.661 | 0.717 |
| Agreement | 0.953 | 0.937 | 0.945 |
| Order | 0.718 | 0.781 | 0.748 |
| Ticket | 0.801 | 0.773 | 0.787 |
| Invoice | 0.818 | 0.881 | 0.848 |
| Photo | 0.948 | 0.924 | 0.936 |
| Inquiry | 0.923 | 0.728 | 0.814 |
| Correspondence | 0.613 | 0.502 | 0.552 |
| Receipt | 0.774 | 0.674 | 0.721 |
| Cover Letter | 0.844 | 0.863 | 0.853 |
| Police Report | 0.960 | 0.895 | 0.926 |
| Terms and Conditions | 0.807 | 0.842 | 0.824 |
| Statement | 0.826 | 0.910 | 0.866 |

(a) Precision, Recall, and F1-Score per class

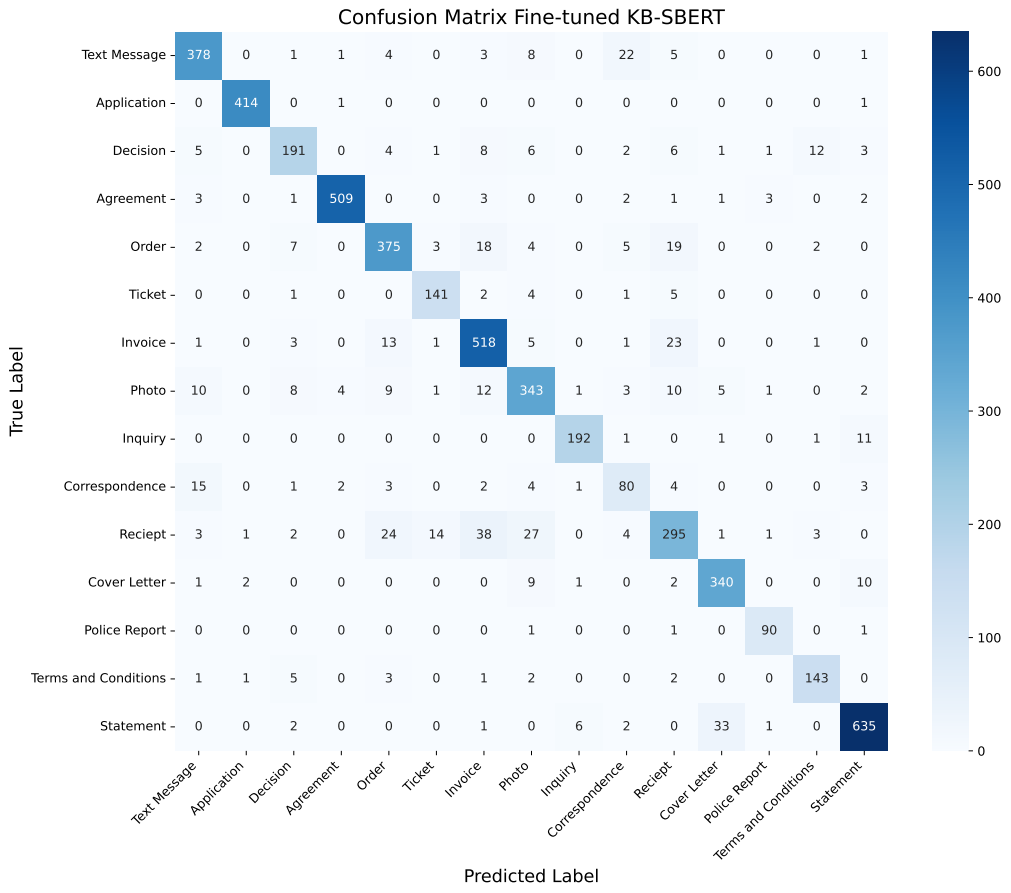



(b) Confusion Matrix

Figure A.1: Confusion Matrix, Precision, Recall and F1 for ViT

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.902 | 0.894 | 0.898 |
| Application | 0.990 | 0.995 | 0.993 |
| Decision | 0.860 | 0.796 | 0.827 |
| Agreement | 0.985 | 0.970 | 0.977 |
| Order | 0.862 | 0.862 | 0.862 |
| Ticket | 0.876 | 0.916 | 0.895 |
| Invoice | 0.855 | 0.915 | 0.884 |
| Photo | 0.831 | 0.839 | 0.835 |
| Inquiry | 0.955 | 0.932 | 0.943 |
| Correspondence | 0.650 | 0.696 | 0.672 |
| Receipt | 0.791 | 0.714 | 0.751 |
| Cover Letter | 0.890 | 0.932 | 0.910 |
| Police Report | 0.928 | 0.968 | 0.947 |
| Terms and Conditions | 0.883 | 0.905 | 0.894 |
| Statement | 0.949 | 0.934 | 0.941 |

(a) Precision, Recall, and F1-Score per class

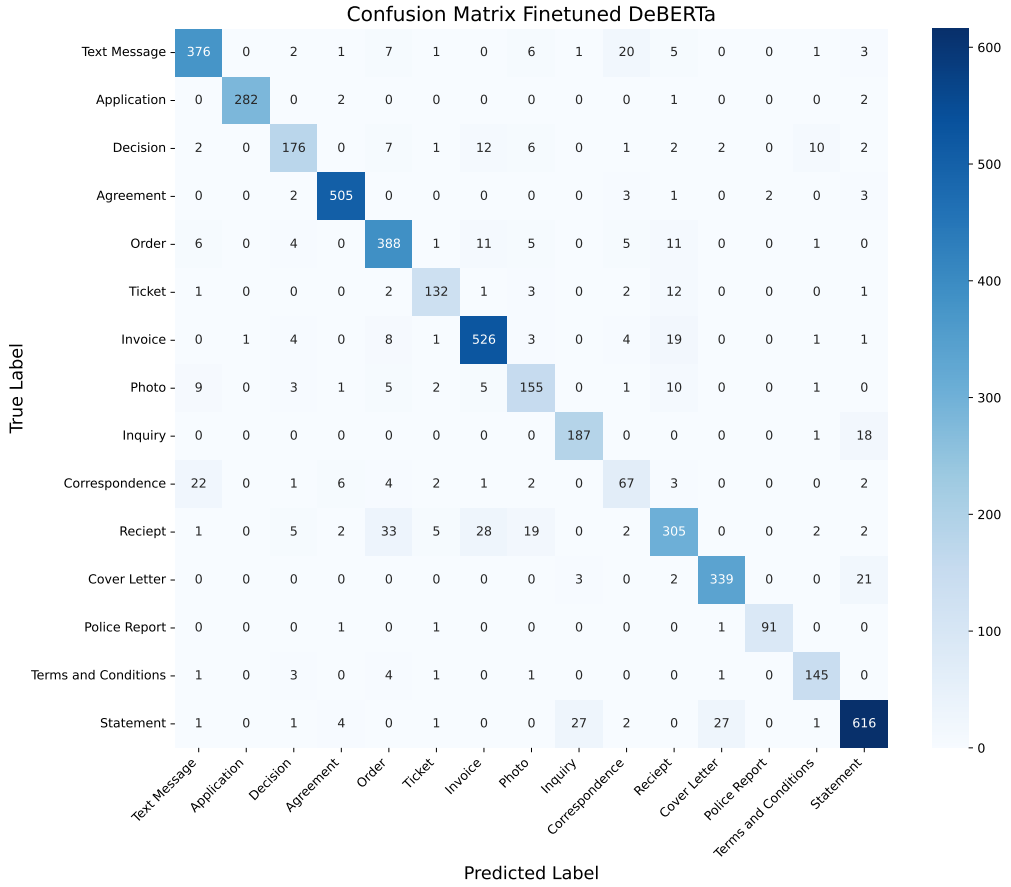



(b) Confusion Matrix

Figure A.2: Confusion Matrix, Precision, Recall and F1 for KB-SBERT

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.897 | 0.889 | 0.893 |
| Application | 0.996 | 0.983 | 0.989 |
| Decision | 0.876 | 0.796 | 0.834 |
| Agreement | 0.967 | 0.979 | 0.973 |
| Order | 0.847 | 0.898 | 0.872 |
| Ticket | 0.892 | 0.857 | 0.874 |
| Invoice | 0.901 | 0.926 | 0.913 |
| Photo | 0.775 | 0.807 | 0.791 |
| Inquiry | 0.858 | 0.908 | 0.882 |
| Correspondence | 0.626 | 0.609 | 0.618 |
| Receipt | 0.822 | 0.755 | 0.787 |
| Cover Letter | 0.916 | 0.929 | 0.922 |
| Police Report | 0.978 | 0.968 | 0.973 |
| Terms and Conditions | 0.890 | 0.929 | 0.909 |
| Statement | 0.918 | 0.906 | 0.912 |

(a) Precision, Recall, and F1-Score per class

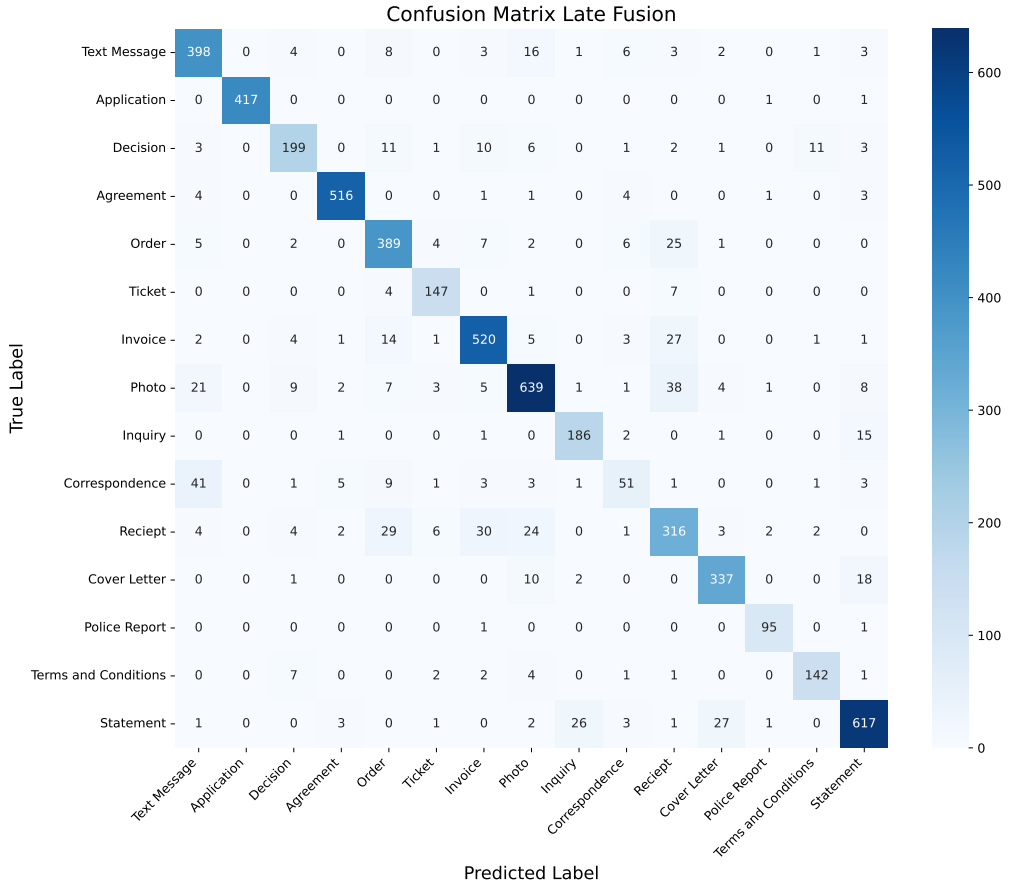



(b) Confusion Matrix

Figure A.3: Confusion Matrix, Precision, Recall and F1 for DeBERTa

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.831 | 0.894 | 0.861 |
| Application | 1.000 | 0.995 | 0.998 |
| Decision | 0.861 | 0.802 | 0.831 |
| Agreement | 0.974 | 0.974 | 0.974 |
| Order | 0.826 | 0.882 | 0.853 |
| Ticket | 0.886 | 0.925 | 0.905 |
| Invoice | 0.892 | 0.898 | 0.895 |
| Photo | 0.896 | 0.865 | 0.880 |
| Inquiry | 0.857 | 0.903 | 0.879 |
| Correspondence | 0.646 | 0.425 | 0.513 |
| Receipt | 0.751 | 0.747 | 0.749 |
| Cover Letter | 0.896 | 0.916 | 0.906 |
| Police Report | 0.941 | 0.979 | 0.960 |
| Terms and Conditions | 0.899 | 0.887 | 0.893 |
| Statement | 0.915 | 0.905 | 0.910 |

(a) Precision, Recall, and F1-Score per class

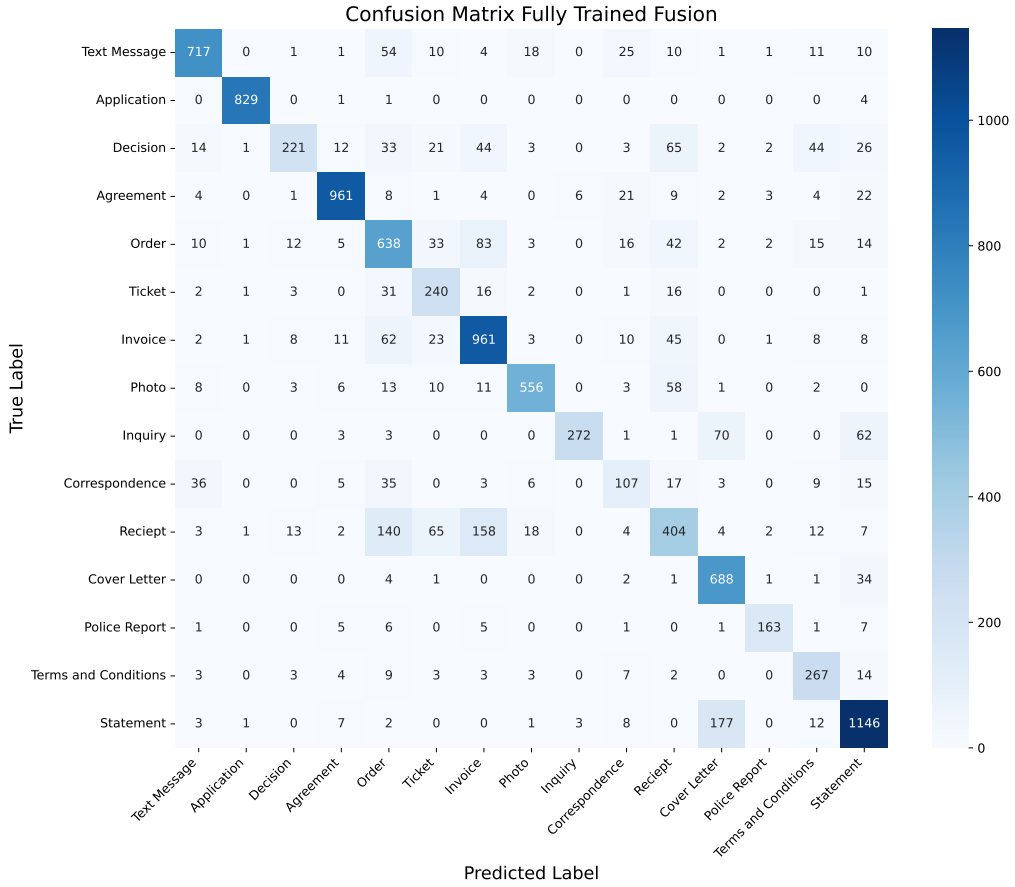



(b) Confusion Matrix

Figure A.4: Confusion Matrix, Precision, Recall and F1 for Late fusion model

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.893 | 0.831 | 0.861 |
| Application | 0.993 | 0.993 | 0.993 |
| Decision | 0.834 | 0.450 | 0.585 |
| Agreement | 0.939 | 0.919 | 0.929 |
| Order | 0.614 | 0.728 | 0.666 |
| Ticket | 0.590 | 0.767 | 0.667 |
| Invoice | 0.744 | 0.841 | 0.789 |
| Photo | 0.907 | 0.829 | 0.866 |
| Inquiry | 0.968 | 0.660 | 0.785 |
| Correspondence | 0.512 | 0.453 | 0.481 |
| Receipt | 0.603 | 0.485 | 0.538 |
| Cover Letter | 0.723 | 0.940 | 0.818 |
| Police Report | 0.931 | 0.858 | 0.893 |
| Terms and Conditions | 0.692 | 0.840 | 0.759 |
| Statement | 0.836 | 0.843 | 0.840 |

(a) Precision, Recall, and F1-Score per class

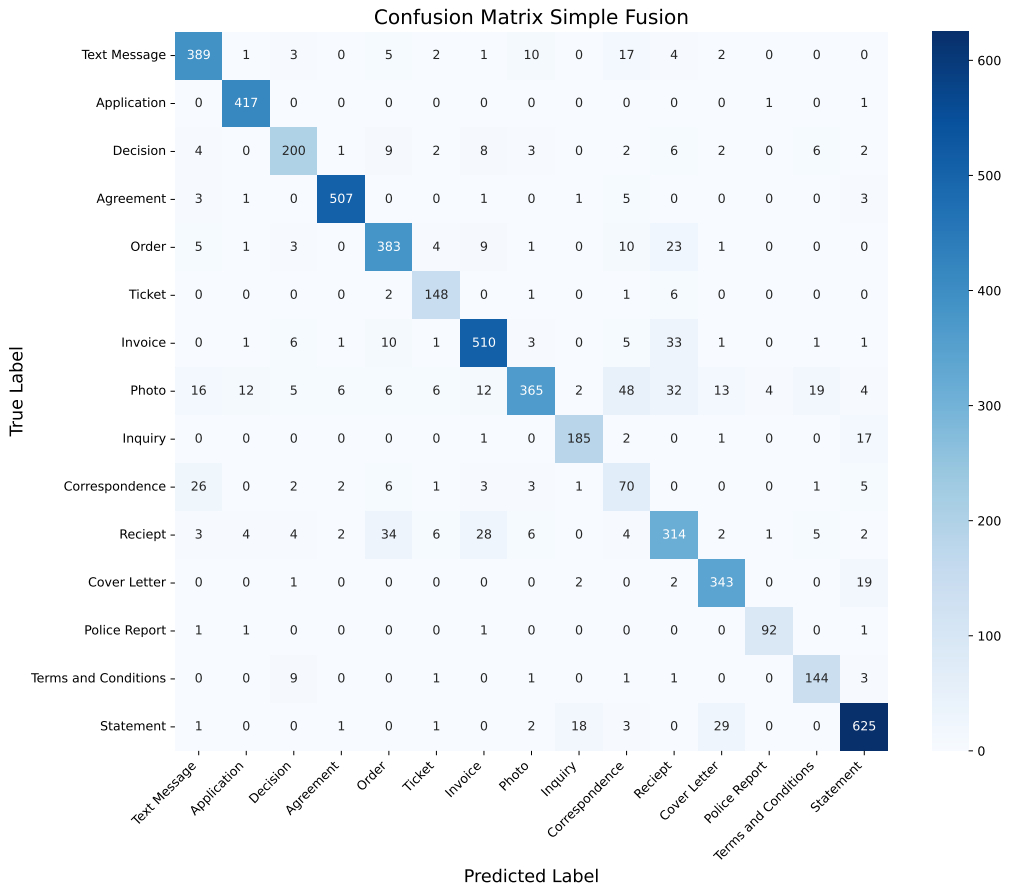



(b) Confusion Matrix

Figure A.5: Confusion Matrix, Precision, Recall and F1 for Fully trained Early fusion model

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.868 | 0.896 | 0.882 |
| Application | 0.952 | 0.995 | 0.973 |
| Decision | 0.858 | 0.816 | 0.837 |
| Agreement | 0.975 | 0.973 | 0.974 |
| Order | 0.842 | 0.870 | 0.856 |
| Ticket | 0.860 | 0.937 | 0.897 |
| Invoice | 0.889 | 0.890 | 0.889 |
| Photo | 0.924 | 0.664 | 0.772 |
| Inquiry | 0.885 | 0.898 | 0.892 |
| Correspondence | 0.417 | 0.583 | 0.486 |
| Receipt | 0.746 | 0.757 | 0.751 |
| Cover Letter | 0.871 | 0.935 | 0.901 |
| Police Report | 0.939 | 0.958 | 0.948 |
| Terms and Conditions | 0.818 | 0.900 | 0.857 |
| Statement | 0.915 | 0.919 | 0.917 |

(a) Precision, Recall, and F1-Score per class

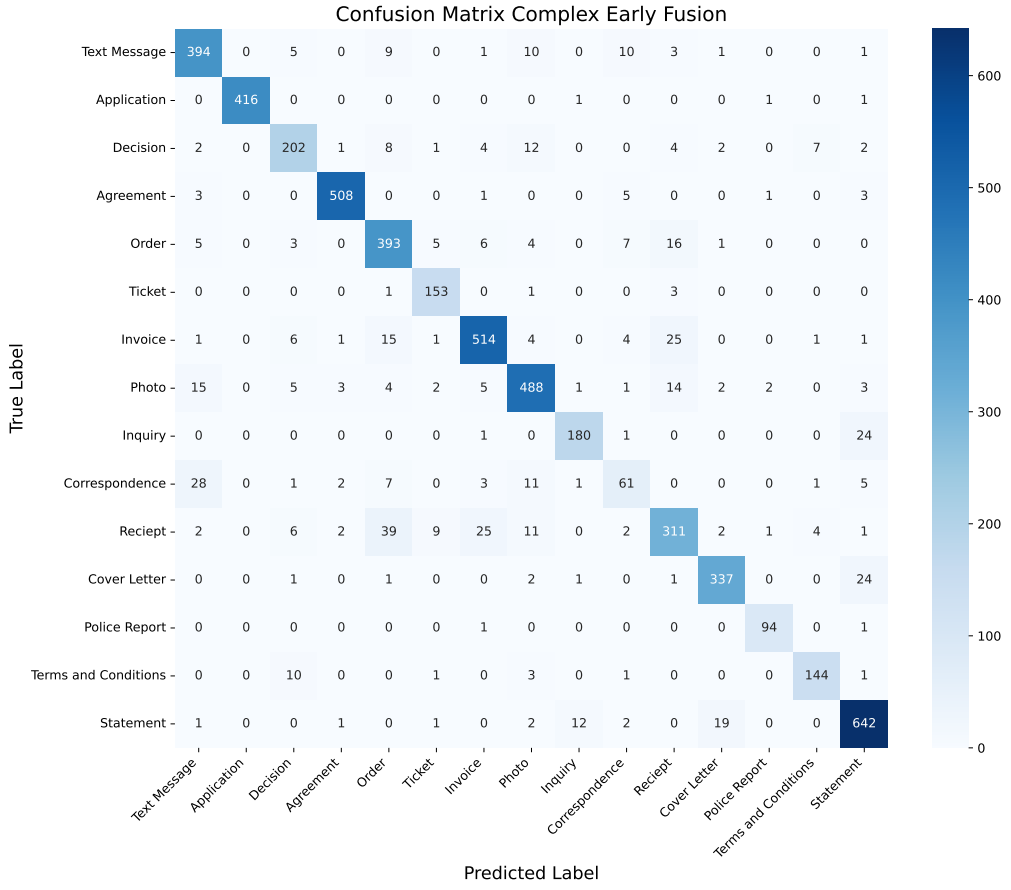



(b) Confusion Matrix

Figure A.6: Confusion Matrix, Precision, Recall and F1 for Simple Early fusion model

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.874 | 0.908 | 0.890 |
| Application | 1.000 | 0.993 | 0.996 |
| Decision | 0.845 | 0.824 | 0.835 |
| Agreement | 0.981 | 0.975 | 0.978 |
| Order | 0.824 | 0.893 | 0.857 |
| Ticket | 0.884 | 0.968 | 0.924 |
| Invoice | 0.916 | 0.897 | 0.907 |
| Photo | 0.891 | 0.895 | 0.893 |
| Inquiry | 0.918 | 0.874 | 0.896 |
| Correspondence | 0.649 | 0.508 | 0.570 |
| Receipt | 0.825 | 0.749 | 0.785 |
| Cover Letter | 0.926 | 0.918 | 0.922 |
| Police Report | 0.949 | 0.979 | 0.964 |
| Terms and Conditions | 0.917 | 0.900 | 0.909 |
| Statement | 0.906 | 0.944 | 0.924 |

(a) Precision, Recall, and F1-Score per class

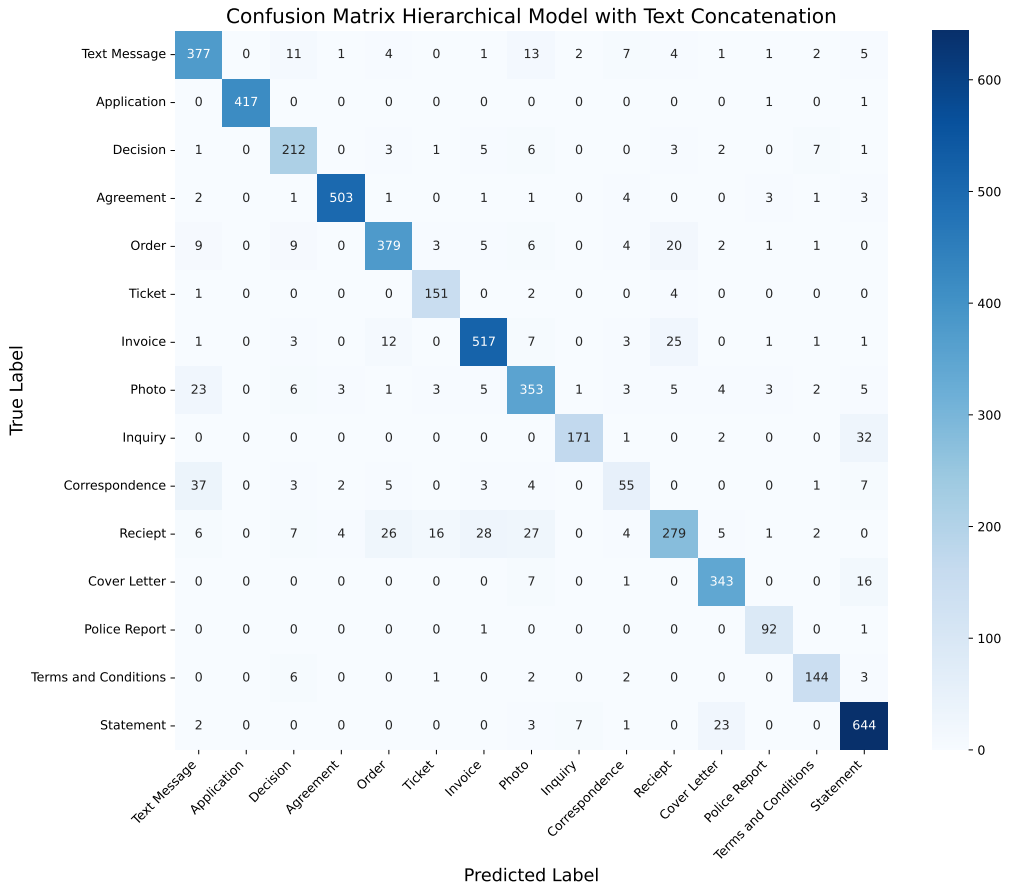



(b) Confusion Matrix

Figure A.7: Confusion Matrix, Precision, Recall and F1 for Complex early fusion model

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.821 | 0.879 | 0.849 |
| Application | 1.000 | 0.995 | 0.998 |
| Decision | 0.822 | 0.880 | 0.850 |
| Agreement | 0.981 | 0.967 | 0.974 |
| Order | 0.879 | 0.863 | 0.871 |
| Ticket | 0.863 | 0.956 | 0.907 |
| Invoice | 0.913 | 0.905 | 0.909 |
| Photo | 0.819 | 0.847 | 0.833 |
| Inquiry | 0.945 | 0.830 | 0.884 |
| Correspondence | 0.647 | 0.470 | 0.545 |
| Receipt | 0.821 | 0.689 | 0.749 |
| Cover Letter | 0.898 | 0.935 | 0.916 |
| Police Report | 0.893 | 0.979 | 0.934 |
| Terms and Conditions | 0.894 | 0.911 | 0.903 |
| Statement | 0.896 | 0.947 | 0.921 |

(a) Precision, Recall, and F1-Score per class

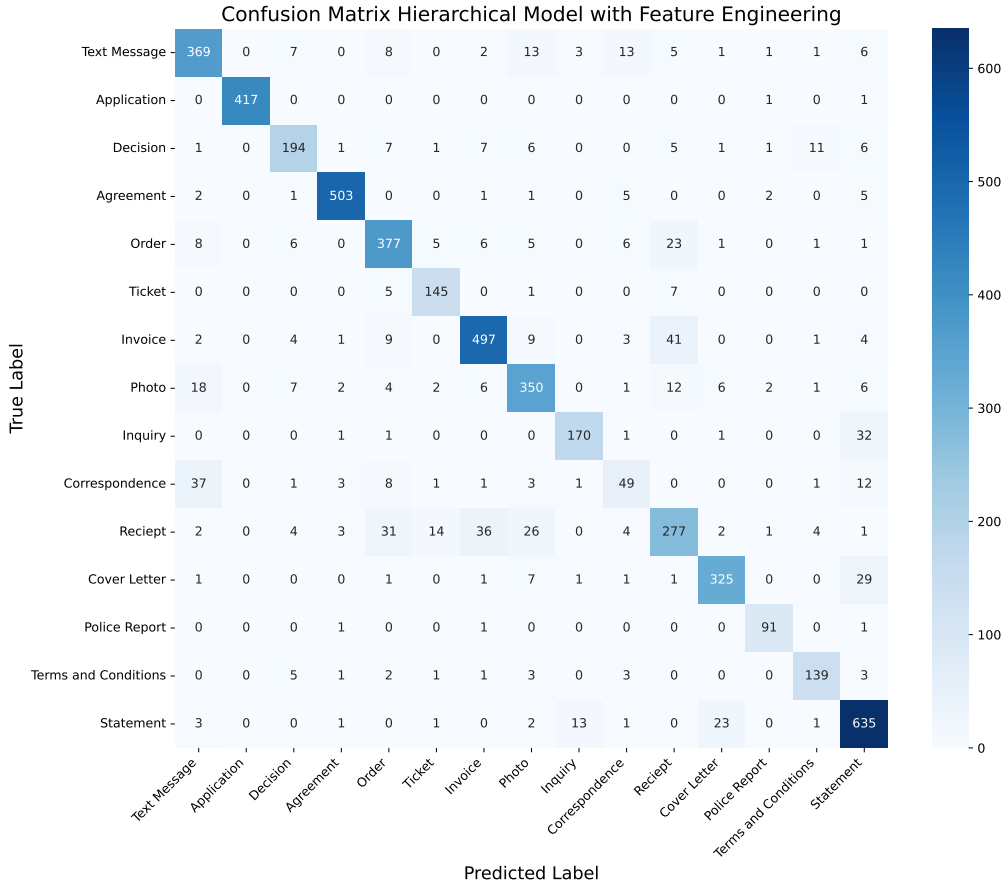



(b) Confusion Matrix

Figure A.8: Confusion Matrix, Precision, Recall and F1 for Hierarchical model with text concatenation

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.833 | 0.860 | 0.846 |
| Application | 1.000 | 0.995 | 0.998 |
| Decision | 0.847 | 0.805 | 0.826 |
| Agreement | 0.973 | 0.967 | 0.970 |
| Order | 0.832 | 0.859 | 0.845 |
| Ticket | 0.853 | 0.918 | 0.884 |
| Invoice | 0.889 | 0.870 | 0.880 |
| Photo | 0.822 | 0.839 | 0.830 |
| Inquiry | 0.904 | 0.825 | 0.863 |
| Correspondence | 0.563 | 0.419 | 0.480 |
| Receipt | 0.747 | 0.684 | 0.714 |
| Cover Letter | 0.903 | 0.886 | 0.894 |
| Police Report | 0.919 | 0.968 | 0.943 |
| Terms and Conditions | 0.869 | 0.880 | 0.874 |
| Statement | 0.856 | 0.934 | 0.893 |

(a) Precision, Recall, and F1-Score per class

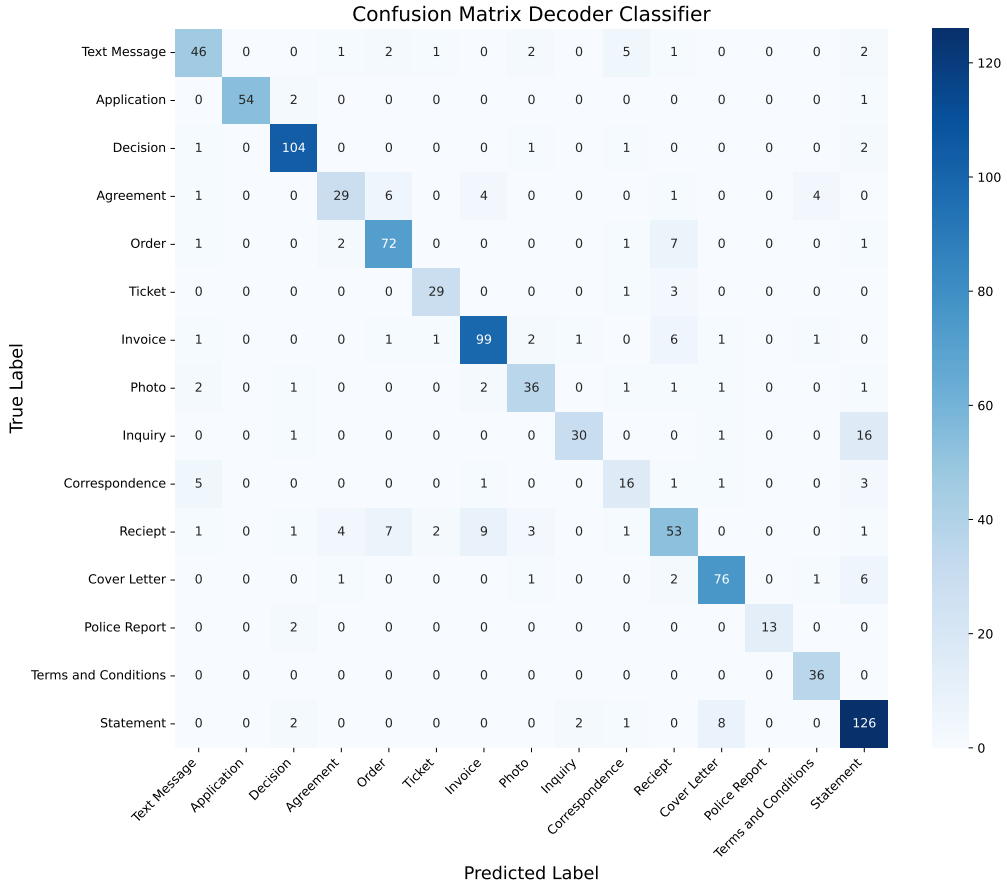



(b) Confusion Matrix

Figure A.9: Confusion Matrix, Precision, Recall and F1 for Hierarchical model with feature engineering

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.793 | 0.767 | 0.780 |
| Application | 1.000 | 0.947 | 0.973 |
| Decision | 0.920 | 0.954 | 0.937 |
| Agreement | 0.784 | 0.644 | 0.707 |
| Order | 0.818 | 0.857 | 0.837 |
| Ticket | 0.879 | 0.879 | 0.879 |
| Invoice | 0.861 | 0.876 | 0.868 |
| Photo | 0.800 | 0.800 | 0.800 |
| Inquiry | 0.909 | 0.625 | 0.741 |
| Correspondence | 0.593 | 0.593 | 0.593 |
| Receipt | 0.707 | 0.646 | 0.675 |
| Cover Letter | 0.864 | 0.874 | 0.869 |
| Police Report | 1.000 | 0.867 | 0.929 |
| Terms and Conditions | 0.857 | 1.000 | 0.923 |
| Statement | 0.792 | 0.906 | 0.846 |

(a) Precision, Recall, and F1-Score per class

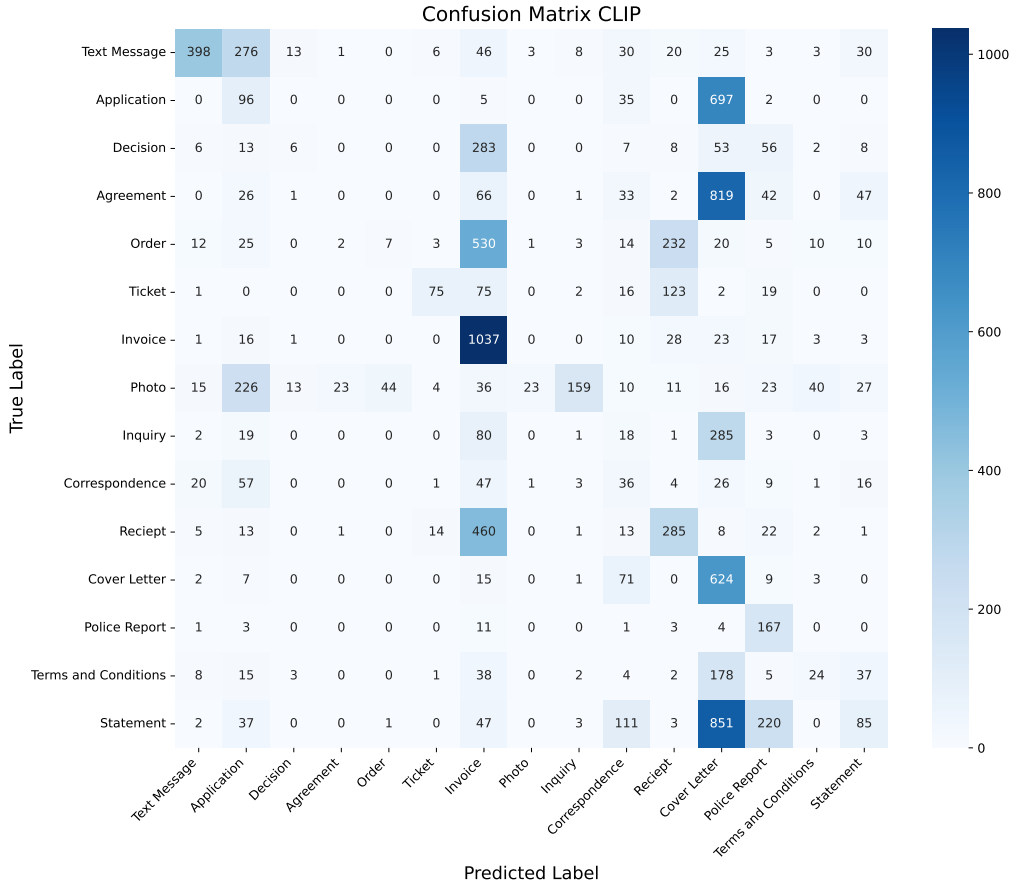



(b) Confusion Matrix

Figure A.10: Confusion Matrix, Precision, Recall and F1 for Decoder model (Qwen2.5-3B)

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.841 | 0.462 | 0.596 |
| Application | 0.116 | 0.115 | 0.115 |
| Decision | 0.162 | 0.014 | 0.025 |
| Agreement | 0.000 | 0.000 | nan |
| Order | 0.135 | 0.008 | 0.015 |
| Ticket | 0.721 | 0.240 | 0.360 |
| Invoice | 0.374 | 0.910 | 0.530 |
| Photo | 0.821 | 0.034 | 0.066 |
| Inquiry | 0.005 | 0.002 | 0.003 |
| Correspondence | 0.088 | 0.163 | 0.114 |
| Receipt | 0.395 | 0.345 | 0.368 |
| Cover Letter | 0.172 | 0.852 | 0.286 |
| Police Report | 0.277 | 0.879 | 0.422 |
| Terms and Conditions | 0.273 | 0.076 | 0.119 |
| Statement | 0.318 | 0.062 | 0.104 |

(a) Precision, Recall, and F1-Score per class

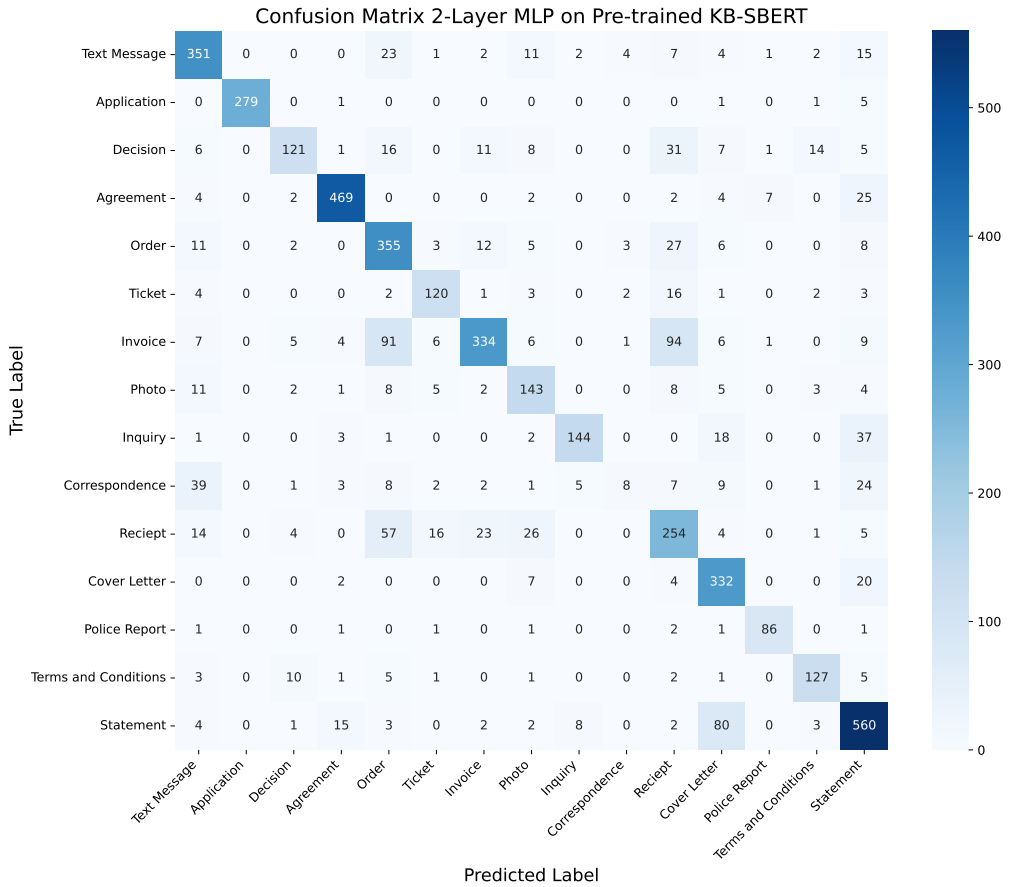



(b) Confusion Matrix

Figure A.11: Confusion Matrix, Precision, Recall and F1 for CLIP model

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.770 | 0.830 | 0.799 |
| Application | 1.000 | 0.972 | 0.986 |
| Decision | 0.818 | 0.548 | 0.656 |
| Agreement | 0.936 | 0.911 | 0.923 |
| Order | 0.624 | 0.822 | 0.709 |
| Ticket | 0.774 | 0.779 | 0.777 |
| Invoice | 0.859 | 0.592 | 0.701 |
| Photo | 0.656 | 0.745 | 0.698 |
| Inquiry | 0.906 | 0.699 | 0.789 |
| Correspondence | 0.444 | 0.073 | 0.125 |
| Receipt | 0.557 | 0.629 | 0.591 |
| Cover Letter | 0.693 | 0.910 | 0.787 |
| Police Report | 0.896 | 0.915 | 0.905 |
| Terms and Conditions | 0.825 | 0.814 | 0.819 |
| Statement | 0.771 | 0.824 | 0.797 |

(a) Precision, Recall, and F1-Score per class

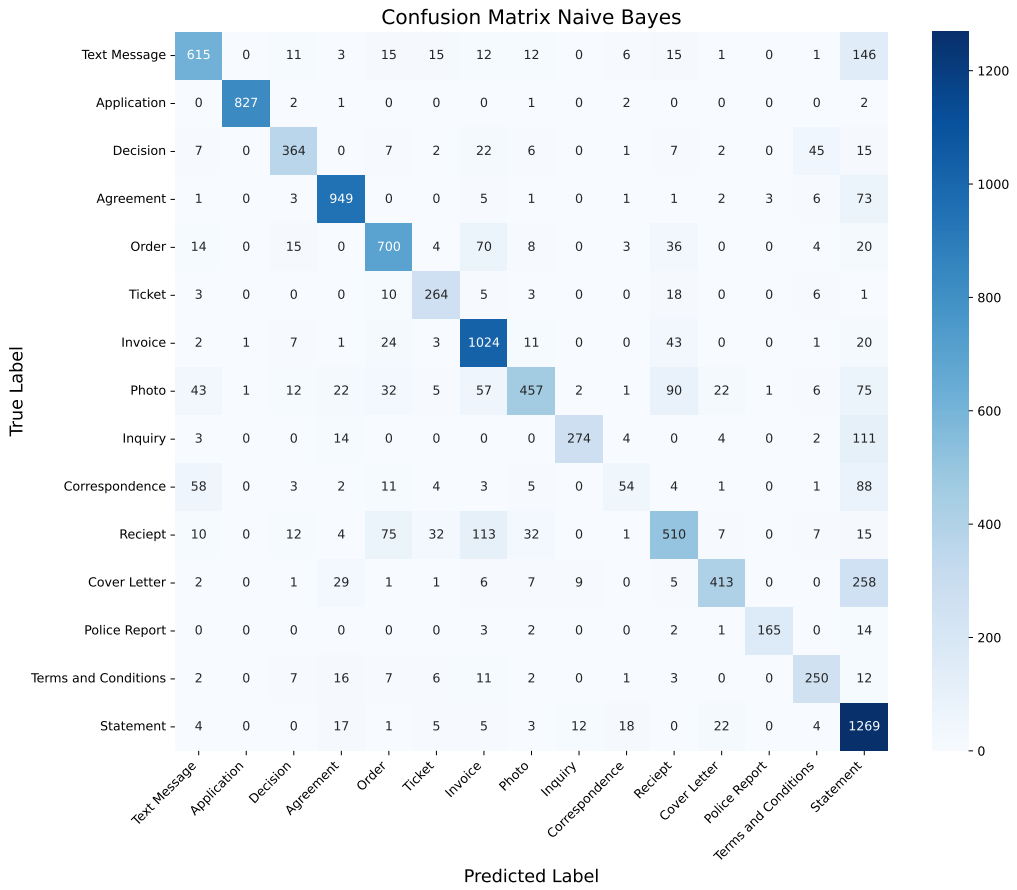



(b) Confusion Matrix

Figure A.12: Confusion Matrix, Precision, Recall and F1 for Frozen KB-SBERT model with 2-layer MLP classification head

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.805 | 0.722 | 0.761 |
| Application | 0.998 | 0.990 | 0.994 |
| Decision | 0.833 | 0.762 | 0.796 |
| Agreement | 0.897 | 0.908 | 0.903 |
| Order | 0.793 | 0.801 | 0.797 |
| Ticket | 0.774 | 0.852 | 0.811 |
| Invoice | 0.766 | 0.901 | 0.828 |
| Photo | 0.831 | 0.553 | 0.664 |
| Inquiry | 0.923 | 0.665 | 0.773 |
| Correspondence | 0.587 | 0.231 | 0.331 |
| Receipt | 0.695 | 0.623 | 0.657 |
| Cover Letter | 0.869 | 0.564 | 0.684 |
| Police Report | 0.976 | 0.882 | 0.927 |
| Terms and Conditions | 0.751 | 0.789 | 0.769 |
| Statement | 0.599 | 0.933 | 0.730 |

(a) Precision, Recall, and F1-Score per class

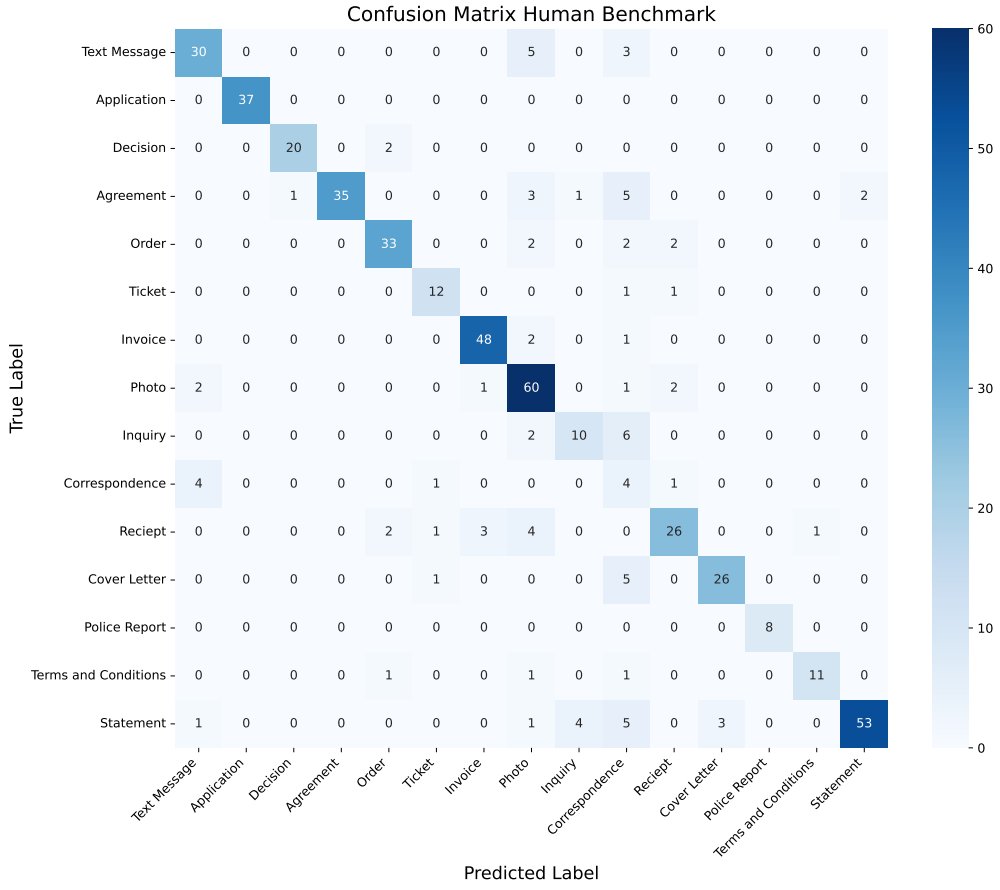



(b) Confusion Matrix

Figure A.13: Confusion Matrix, Precision, Recall and F1 for Naive Bayes classifier

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Text Message | 0.811 | 0.789 | 0.800 |
| Application | 1.000 | 1.000 | 1.000 |
| Decision | 0.952 | 0.909 | 0.930 |
| Agreement | 1.000 | 0.745 | 0.854 |
| Order | 0.868 | 0.846 | 0.857 |
| Ticket | 0.800 | 0.857 | 0.828 |
| Invoice | 0.923 | 0.941 | 0.932 |
| Photo | 0.750 | 0.909 | 0.822 |
| Inquiry | 0.667 | 0.556 | 0.606 |
| Correspondence | 0.118 | 0.400 | 0.182 |
| Receipt | 0.812 | 0.703 | 0.754 |
| Cover Letter | 0.897 | 0.812 | 0.852 |
| Police Report | 1.000 | 1.000 | 1.000 |
| Terms and Conditions | 0.917 | 0.786 | 0.846 |
| Statement | 0.964 | 0.791 | 0.869 |

(a) Precision, Recall, and F1-Score per class




(b) Confusion Matrix

Figure A.14: Confusion Matrix, Precision, Recall and F1 for Human benchmark

# B Appendix B - Hyperparameters

Hyperparameters for fine-tuning the Vision transformer and KB-SBERT, as well as parameters used for constructing fusion architecture.

Table B.1: Training Hyperparameters KB-SBERT

| **Hyperparameter** | **Value** |
|---|---|
| num_train_epochs | 5 |
| per_device_train_batch_size | 64 |
| per_device_eval_batch_size | 64 |
| eval_strategy | steps |
| save_strategy | steps |
| eval_steps | 500 |
| learning_rate | 2e-5 |
| fp16 | True |
| dataloader_num_workers | 4 |
| dataloader_pin_memory | True |
| gradient_accumulation_steps | 1 |
| max_grad_norm | 1.0 |

Table B.2: Training Hyperparameters ViT

| **Hyperparameter** | **Value** |
|---|---|
| num_train_epochs | 5 |
| per_device_train_batch_size | 64 |
| per_device_eval_batch_size | 64 |
| evaluation_strategy | epoch |
| save_strategy | epoch |
| learning_rate | 2e-5 |
| dataloader_num_workers | 4 |
| fp16 | True |
| dataloader_pin_memory | True |
| gradient_accumulation_steps | 1 |
| max_grad_norm | 1.0 |

Table B.3: Training Hyperparameters Complex Early fusion

| Hyperparameter | Value |
|---|---|
| num_train_epochs | 5 |
| per_device_train_batch_size | 4 |
| per_device_eval_batch_size | 1 |
| evaluation_strategy | epoch |
| save_strategy | epoch |
| dataloader_num_workers | 0 |
| dataloader_pin_memory | True |
| num_heads | 8 |
| num_layers | 6 |

Table B.4: Training Hyperparameters Qwen2.5-3B

| Hyperparameter | Value |
|---|---|
| num_train_epochs | 5 |
| evaluation_strategy | epoch |
| save_strategy | epoch |
| learning_rate | 2e-5 |
| warmup_ratio | 0.1 |
| lr_scheduler_type | cosine |
| weight_decay | 0.01 |
| optim | adamw_torch |
| seed | 42 |
| data_seed | 42 |
| metric_for_best_model | accuracy |
| label_smoothing_factor | 0.0 |
| max_grad_norm | 1.0 |
| dataloader_num_workers | 4 |
| gradient_accumulation_steps | 1 |